UNRESTRICTED UNIONS OF FRONTIER-ONE AND FULL FIRST-ORDER-REWRITABLE RULES HAVE FINITE PARTITIONWIDTH

THOMAS FELLER, TIM S. LYON, PIOTR OSTROPOLSKI-NALEWAJA, SEBASTIAN RUDOLPH

Editorial scope. Only the titled novel framework/extension counted. Prior ICDT and Blumensath results, easy closures, support lemmas and direct corollaries uncounted.

Allowed standard prerequisites (editorial).

- First-order and second-order logic, compactness, homomorphisms, MSO interpretation, Rabin tree theorem and existential-rule chase. All original definitions and original bbl retained. Prior Blumensath partitionwidth and earlier ICDT grid results remain cited prerequisites, not new tasks.

1. Introduction

The task of checking logical entailment is at the core of logic in computer science, with wide applications in many related fields including symbolic AI, databases, and verification, but also impacting neighboring disciplines like mathematics and philosophy. Automating any kind of logical reasoning requires a profound understanding of the underlying entailment problem's computational properties, and in particular, its decidability status. The fact that entailment in many natural logical formalisms is undecidable has spawned extensive research activities toward the identification of expressive yet decidable fragments and generally toward a better understanding of generic principles underlying the divide between decidability and undecidability [BGG97, Var96, Grä01, PH23]. This article contributes to this endeavor.

The concept of logical entailment appears to be the essence and primordial motivation of the study of logic. It embodies the judgment, typically noted down as $\Phi \models \Psi$, that a certain statement Ψ is an inevitable consequence of another statement Φ, established through formal structural analysis. Starting out as a primarily philosophical endeavor millenia ago [AriCE], the study of logic(al entailment) underwent substantial mathematical formalization and rigorization in the late 19th and early 20th century [Boo54, Fre79, Tar43]. With the advent of computing devices and computer science, the algorithmization of logical reasoning and the computational characteristics of entailment problems became central topics. Today, logical entailment is a notion widely employed throughout computer science, including but not restricted to the following areas:

- **Formal Methods**: This discipline is concerned with describing dynamic systems and ascertaining their properties. If the dynamic system – typically specified as a state-transition system – is fully specified (e.g., by means of a process algebra), the task of deciding if it satisfies some property (such as safety or lifeness) is a variant of model checking (and explicitly referred to as such).

Date: arxiv:2304.06348v5; CC BY 4.0.

- **Databases**: The primary task considered in the database (DB) area is that of query answering, which, in its plain form, can be logically described as finding variable assignments (the *query answers*) for a logical formula (the *query*) which make that formula true over a given finite structure (the *database*). If the logic used as query language permits to directly refer to database elements, the query answering problem can be reduced to validity of logical sentences (*Boolean queries*) – and hence to classical logical model checking.

  More advanced database settings, however, deal with the phenomenon of incomplete information, where the database does not carry the full knowledge about the state of affairs and rather is to be seen as a logical description which allows for several realizations (that is, models). Such ambiguity may arise from absence of certain information (e.g., when following the open world assumption) or from providing higher-level logical constraints alongside factual data (as in *deductive databases*). Determining which query answers are *certain answers*, or which boolean queries are satisfied in all models then amounts to an entailment problem.

  Another way in which entailment arises in the area of databases is in static analysis of queries and views. The popular problem of query containment asks if every answer for Q_1 is an answer for Q_2 over every database.
- **Classical Knowledge Representation**: Classical logic-based knowledge representation (KR) is a subdiscipline of artificial intelligence that attempts to represent world knowledge as finite logical theories – referred to as *knowledge bases* – of some logical formalism – sometimes called *ontology language*, with the purpose of establishing a clear semantics and harnessing automated inferencing techniques to provide reasoning support. Rooted in mathematical logic, a fundamental task considered in classical KR is that of *axiom entailment*, asking if some logical sentence (referred to as an axiom) of the formalism is a consequence of the considered knowledge base. In most cases, the considered logical language supports negation on the sentence level, whereby the axiom entailment problem can be reduced to that of *knowledge base satisfiability*. On the other hand, knowledge base satisfiability is usually reducible to axiom entailment by asking if the knowledge base entails a contradictory sentence – provided such a sentence exists in the considered formalism. Prominent KR formalisms are description logics and rule-based languages.
- **Contemporary Knowledge Representation**: Over the recent two or three decades, the DB and KR fields have converged significantly. One one hand, the DB side has developed an increased interest into taking available background knowledge (and the implicit knowledge implied by it) into account when querying data – which is ever more important in view of the rise of many open, less curated data sources e.g. on the Web. On the other hand, the KR community became aware of the necessity of distinguishing between the formalism used for specifying the background knowledge (the ontology language) and that for expressing information needs (the query language). The enlarged intersection between DB and KR fields that might be referred to as *contemporary KR* is primarily concerned with the task of *ontology-based query answering* (OBQA), that is posing queries to knowledge bases under the certain answer semantics. At the core of this task is, again, an entailment problem.

More formally, in this article, we will consider the following setting: Assuming two classes $\mathfrak{F}$ and $\mathfrak{Q}$ of logical sentences, the decision problem of interest is specified as follows:

Problem: ENTAILMENT$(\mathfrak{F}, \mathfrak{Q})$
Input: $\Phi \in \mathfrak{F}$, $\Psi \in \mathfrak{Q}$.
Output: YES, if $\Phi \models \Psi$, NO otherwise.

We will refer to $\mathfrak{F}$ as a *specification language*, to $\mathfrak{Q}$ as a *query language*, and we sometimes call the above entailment problem "query entailment" or "querying", as is common in database theory and knowledge representation. We assume $\mathfrak{F}$ and $\mathfrak{Q}$ both rely on a common model theory based upon relational structures of arbitrary cardinality.[1] More specifically, in this article, we focus on the setting where $\mathfrak{F}$ is a fragment of first-order logic (FO) and $\mathfrak{Q}$ is a fragment of universal second-order logic ($\forall$SO). There are good reasons for focusing on this particular – yet still very general – shape of entailments:

- Given the choice of $\mathfrak{Q}$, the considered problem occurs to be a special case of entailment in second-order logic. However, entailment problems of this shape can be reduced to first-order entailment in a rather straighforward way, which ensures semidecidability and allows one to restrict one's attention to countable models (see Section 3).
- The reason for posing the problem as an entailment $\Phi \models \Psi$, rather than the (un)satisfiability of $\Phi \wedge \neg\Psi$, is that in many scenarios encountered in databases, knowledge representation, and other practical settings, the specification language and the query language differ significantly so that the reduction to (un)satisfiability does not generally produce a sentence that would readily fall into a known decidable fragment [CGL98, CEO09, RG10, OS12, BGO14].
- Many contemporary query languages (starting from simple types of path queries used in graph databases [FLS98] and reaching up to highly expressive query formalisms like disjunctive datalog [EGM97]) go well beyond FO, but can be straightforwardly expressed in $\forall$SO (see Section 5).
- The two other "archetypical" decision problems in logic can be obtained as special cases of ENTAILMENT$(\mathfrak{F}, \mathfrak{Q})$: picking $\mathfrak{Q} = \{\text{false}\}$ yields (un)satisfiability checking in $\mathfrak{F}$, whereas choosing $\mathfrak{F} = \{\text{true}\}$ amounts to the validity problem in $\mathfrak{Q}$.

The described problem in its unconstrained form, ENTAILMENT$(\text{FO}, \forall\text{SO})$, is merely semidecidable, that is, there exists an algorithm that successfully terminates in case $\Phi \models \Psi$. Thus, in order to arrive at a decidable setting, one needs to further constrain $\mathfrak{F}$ and $\mathfrak{Q}$ in a way that ensures co-semidecidability, i.e. the existence of a procedure successfully terminating whenever $\Phi \not\models \Psi$. We propose to achieve this by exploiting the existence of *countermodels* (that is, models of Φ that are not models of Ψ and hence witness the non-entailment) which are of a particular, simple shape. To gauge "structural difficulty", a variety of functions, usually referred to as *width measures* have been proposed and have proven beneficial for a wide range of diverse purposes (graph-theoretic, model-theoretic, algorithmic, and others).

Example 1.1. As an easy example, consider the folklore observation that the satisfiability problem of a FO fragment $\mathfrak{F}$ is decidable whenever it exhibits the

[1] In particular, this article will not be concerned with finite-model semantics and the notion of finite entailment.

finite-model property (fmp), meaning that every satisfiable sentence $\Phi \in \mathfrak{F}$ has a finite model: In that case, unsatisfiability of Φ can be detected by enumerating all its consequences and stopping if *false* is produced, while satisfiability of Φ can be spotted by enumerating all finite structures, checking modelhood, and stopping if one finite model of Φ is encountered – then, both semidecision algorithms run in parallel yield a decision procedure.

Returning to our framework, note that the statement *"$\mathfrak{F}$ has the fmp"* can be paraphrased into *"*$\textsc{Entailment}(\mathfrak{F}, \{\text{false}\})$ *has the finite-domainsize-countermodel property"*, and we see that the function just returning the domain size of a structure can be seen as a width measure useful for our purpose.

When it comes to characterizing logics that are guaranteed to have strucutrally simple models, the *treewidth* measure [RS84] has been well-established as another useful tool, e.g., for logics of the guarded family [ANvB98, Grä99]. More recently, treewidth and *cliquewidth* of universal models have been investigated in the context of existential rules [BLMS11, CGK13, FLOR23].[2]

These considerations make it seem worthwhile to investigate – on a more abstract level – under which conditions decidability of $\textsc{Entailment}(\mathfrak{F}, \mathfrak{Q})$ follows from its *"finite-width-countermodel property"* for some kind of width. This is what we set out to do in this paper. In short, our contributions are the following:

- In Section 3 and Section 4, we establish a generic framework for showing decidability of $\textsc{Entailment}(\mathfrak{F}, \mathfrak{Q})$ based on the existence of countermodels of finite width, and discuss how it can be used to obtain and generalize miscellaneous decidability results from the literature (where the existing proofs sometimes even employ an ad hoc version of our generic argument).
- In Section 5, we single out the lucrative special case where the specification language warrants finitely universal model sets of finite width, yielding decidability of entailment for a vast class of homomorphism-closed queries of high expressivity. In our view, this special case strikes a very favorable balance between the expressivity of the specification language and that of the query language.
- Section 6 exposes (a suitably reformulated and simplified version of) Blumensath's *partitionwidth* [Blu03, Blu06] as a very powerful width measure, superseding all others previously considered in this context, and exhibiting the extremely convenient property of being robust under diverse model-theoretic transformations.
- From Section 7 on, we apply our framework in the area of existential rules, by defining *finite-partitionwidth sets* of rules (Section 8). We show that this class of rulesets allows for the incorporation of various types of datalog layers (in the sense of stratification), yielding a flexible toolset for the analysis and creation of rulesets with favorable querying properties (Section 9). We also discuss the somewhat complicated case of rulesets whose decidable querying is based on first-order rewritability (Section 10).

Our work draws from various subfields of computer science and mathematics. To reduce the cognitive load, many of the concepts used will be defined in the place they are needed rather than in the preliminaries; we will make clear if these notions rely on earlier work. The results presented are novel unless clearly stated otherwise. To improve readability, we defer some technical details and most proofs to the appendix. Several of the provided examples refer to previously established

[2]Specifics will be discussed in detail in later sections.

first-order fragments, also including formalisms from the description logic (DL) family. As these observations follow from general properties of these logics rather than their technical details, we refrain from introducing each and every formalism in meticulous detail. In the appendix we do, however provide a concise overview of the syntactic fragments of first- and second-order logic used (Appendix A.1) and a slightly more comprehensive layout of syntax and semantics of DLs (Appendix A.2). For a full treatise of these logics, we refer the interested reader to the standard literature provided inside these appendices.

2. Preliminaries

Tuples and shufflings. We denote finite sequences $a_1, \ldots, a_n$ of objects, also referred to as *(n-)tuples*, by corresponding bold font symbols $\boldsymbol{a}$ and write $|\boldsymbol{a}|$ to refer to a tuple's length n. For tuples $\boldsymbol{a} = a_1, \ldots, a_n$ and $\boldsymbol{b} = b_1, \ldots, b_m$ as well as $I \subseteq \{1, \ldots, n+m\}$ with $|I| = n$, we define the *I-shuffling* of $\boldsymbol{a}$ and $\boldsymbol{b}$, denoted $\boldsymbol{a} +\!\!+_I \boldsymbol{b}$, as the interleaving combination of $\boldsymbol{a}$ and $\boldsymbol{b}$ where I indicates the positions in the resulting $(n+m)$-tuple where the entries of $\boldsymbol{a}$ go. Formally, $\boldsymbol{a} +\!\!+_I \boldsymbol{b}$ is the tuple $\boldsymbol{c} = c_1, \ldots, c_{n+m}$ where

$$c_i = \begin{cases} a_{|\{1,\ldots,i\} \cap I|} & \text{if } i \in I, \\ b_{|\{1,\ldots,i\} \setminus I|} & \text{if } i \notin I. \end{cases}$$

For instance, $a_1 a_2 +\!\!+_{\{2,5\}} b_1 b_2 b_3 = b_1 a_1 b_2 b_3 a_2$. We lift these operations to sets of tuples as usual, that is, if A and B are two sets of tuples, then $A +\!\!+_I B = \{\boldsymbol{a} +\!\!+_I \boldsymbol{b} \mid \boldsymbol{a} \in A \text{ and } \boldsymbol{b} \in B\}$.

Syntax and formulae. We let $\mathbf{T}$ denote a set of *terms*, defined as the union of three countably infinite, mutually disjoint sets of *constants* $\mathbf{C}$, *nulls* $\mathbf{N}$, and *variables* $\mathbf{V}$. We use $\mathtt{a}$, $\mathtt{b}$, $\mathtt{c}$, ... (occasionally annotated) to denote constants, and use x, y, z, ... (occasionally annotated) to denote both nulls and variables. A *signature* Σ is a finite set of *predicate symbols* (called *predicates*), which are capitalized ($\mathtt{E}$, $\mathtt{R}$, ...) with the exception of the special, universal unary predicate $\top$, assumed to include every term. Throughout the paper, we assume a fixed finite signature Σ, unless stated otherwise. For each predicate $\mathtt{R} \in \Sigma$, we denote its *arity* with $\mathsf{ar}(\mathtt{R})$. For a finite set of predicates Σ, we denote by $\mathsf{ar}(\Sigma)$ the maximum over $\mathsf{ar}(\mathtt{R})$ for all $\mathtt{R} \in \Sigma$. We speak of *binary signatures* whenever $\mathsf{ar}(\Sigma) \leq 2$. An *atom* over Σ is an expression of the form $\mathtt{R}(\boldsymbol{t})$, where $\mathtt{R} \in \Sigma$ and $\boldsymbol{t}$ is an $\mathsf{ar}(\mathtt{R})$-tuple of terms. A *ground atom* is an atom $\mathtt{R}(\boldsymbol{t})$, where $\boldsymbol{t}$ consists only of constants. An *instance* $\mathcal{I}$ over Σ is a (possibly countably infinite) set of atoms over constants and nulls, whereas a *database* $\mathcal{D}$ is a finite set of ground atoms. The *active domain* $\mathsf{dom}(\mathcal{I})$ of an instance $\mathcal{I}$ is the set of terms (i.e., constants and nulls) appearing in the atoms of $\mathcal{I}$. Clearly, databases and instances can be seen as model-theoretic structures,[3] so we use $\models$ to indicate satisfaction of logical formulae as usual. We assume the reader is familiar with the syntax and semantics of first- and second-order logic. We will use upper case greek letters (Φ, Ψ, ...) to denote sentences (i.e., formulae without free variables) and lower case ones (φ, ψ, ...) to denote open formulae. We write $\varphi(\boldsymbol{x})$ to indicate that the variable tuple $\boldsymbol{x}$ corresponds to the free variables of φ. By slight abuse of notation, we write $\mathcal{I} \models \varphi(\boldsymbol{t})$ instead of $\mathcal{I}[\boldsymbol{x} \mapsto \boldsymbol{t}] \models \varphi(\boldsymbol{x})$ for tuples $\boldsymbol{t}$ of terms with $|\boldsymbol{t}| = |\boldsymbol{x}|$. We refer to classes of sentences as *fragments* and denote them by gothic letters ($\mathfrak{F}$, $\mathfrak{Q}$, $\mathfrak{L}$, ...) or, when known and fixed, by their usual abbreviations (FO, SO, MSO, GSO, ...). An overview of the fragments discussed in this article is provided in the appendix.

[3] The ensuing model theory enforces the unique name assumption and allows for constants to be uninterpreted. These design choices warrant a simpler and homogenized presentation without affecting the results.

Homomorphisms and substitutions. A *substitution* is a partial function over $\mathbf{T}$. A *homomorphism* from a set of atoms $\mathcal{X}$ to a set of atoms $\mathcal{Y}$ is a substitution h from the terms of $\mathcal{X}$ to the terms of $\mathcal{Y}$ that satisfies: (i) $\mathtt{R}(h(t_1), \dots, h(t_n)) \in \mathcal{Y}$, for all $\mathtt{R}(t_1, \dots, t_n) \in \mathcal{X}$, and (ii) $h(\mathtt{a}) = \mathtt{a}$, for each $\mathtt{a} \in \mathbf{C}$. A homomorphism h is an *isomorphism iff* it is bijective and h^{-1} is also a homomorphism. $\mathcal{X}$ and $\mathcal{Y}$ are *isomorphic* (written $\mathcal{X} \cong \mathcal{Y}$) *iff* an isomorphism from $\mathcal{X}$ to $\mathcal{Y}$ exists.

(Unions of) conjunctive queries and their satisfaction. A *conjunctive query* (CQ) is a formula $q(\boldsymbol{y}) = \exists \boldsymbol{x}.\varphi(\boldsymbol{x}, \boldsymbol{y})$ with $\varphi(\boldsymbol{x}, \boldsymbol{y})$ being a non-empty conjunction (sometimes construed as a set) of atoms over the variables from $\boldsymbol{x}, \boldsymbol{y}$ and constants. The variables from $\boldsymbol{y}$ are called *free*. A *Boolean* CQ (or BCQ) is a CQ with no free variables. A *union of conjunctive queries* (UCQ) $\psi(\boldsymbol{y})$ is a disjunction of CQs with free variables $\boldsymbol{y}$. We will treat UCQs as sets of CQs. Satisfaction of a BCQ $q = \exists \boldsymbol{x}.\varphi(\boldsymbol{x})$ in an instance $\mathcal{I}$, written $\mathcal{I} \models q$, coincides with the existence of a homomorphism from $\varphi(\boldsymbol{x})$ to $\mathcal{I}$. As per standard FO semantics, $\mathcal{I}$ satisfies a union of BCQs, if it satisfies at least one of its disjuncts. Given a FO sentence Φ, we say that it is *finitely controllable* if for every BCQ q with $\Phi \not\models q$ there exists a finite model of Φ that does not satisfy q. An FO fragment is called finitely controllable if all of its sentences are.

3. Semidecidability via Reduction to FO

As noted in the introduction, our general approach to establish a procedure that decides whether the entailment $\Phi \models \Psi$ holds is to couple two complementary semi-decision procedures: the first terminates successfully whenever $\Phi \models \Psi$ and the second does so whenever $\Phi \not\models \Psi$. In this section, we discuss the former, which is the easier one. Recall that we are interested in entailments of the form $\Phi \models \Psi$, where Φ is a FO sentence and Ψ is a $\forall$SO sentence, both over some finite signature Σ. That is, Ψ has the shape $\forall \Sigma'.\Psi^*$ where Ψ^* is a FO sentence over the signature $\Sigma \cup \Sigma'$ (with Σ' being disjoint from Σ and consisting of relation names that are "repurposed" as second-order variables in Ψ).

Observation 3.1. $\Phi \models \Psi$ *iff* $\Phi \models \Psi^*$.

Proof. Recalling $\Psi = \forall \Sigma'.\Psi^*$, we note that $\Phi \models \forall \Sigma'.\Psi^*$ holds iff $\Phi \wedge \neg \forall \Sigma'.\Psi^*$ is unsatisfiable. Now consider $\Phi \wedge \neg \forall \Sigma'.\Psi^* \equiv \Phi \wedge \exists \Sigma'.\neg \Psi^* \equiv \exists \Sigma'.(\Phi \wedge \neg \Psi^*)$, and note that $\exists \Sigma'.(\Phi \wedge \neg \Psi^*)$ is unsatisfiable whenever $\Phi \wedge \neg \Psi^*$ is (through second-order Skolemization). The latter coincides with $\Phi \models \Psi^*$, so we have shown the claimed fact. □

Thanks to this insight, we immediately obtain semidecidability of Entailment(FO,$\forall$SO): Exploiting the completeness of FO [Göd29], we can enumerate all FO consequences of Φ and terminate whenever Ψ^* is produced.

Said relation to FO also helps with co-semidecidability: as every model of $\Phi \wedge \neg \Psi^*$ can immediately serve as a witness of $\Phi \not\models \Psi$, applying the downward part of Löwenheim-Skolem [Sko29] confirms that it suffices to focus one's search for countermodels on instances with a countable domain.

4. Co-Semidecidability via Width Restrictions

In this section, we provide a generic argument by means of which co-semidecidability (and consequently decidability) of Entailment can be established in situations where "structurally well-behaved" countermodels exist. As yardsticks for "structural well-behavedness", we will employ various measures that were originally defined (only) for finite graphs, yet, can be generalized to countable instances.

width measure	SO-friendly	GSO-friendly	MSO-friendly	FO-friendly
expansion	yes	yes	yes	yes
treewidth	no	yes	yes	yes
cliquewidth	no	no	yes	yes
partitionwidth	no	no	yes	yes

TABLE 1. Width notions discussed in this article and their friendliness toward certain logics.

Definition 4.1 (width measures and their properties)**.** A *width measure* is a function w mapping every countable instance over some finite signature to a value from $\mathbb{N} \cup \{\infty\}$. An instance $\mathcal{I}$ is called w-finite if $\mathsf{w}(\mathcal{I}) \in \mathbb{N}$. A width measure w *generalizes* a width measure $\widetilde{\mathsf{w}}$ *iff* every $\widetilde{\mathsf{w}}$-finite instance is w-finite. A fragment $\mathfrak{F}$ has the *finite-*w*-model property iff* every satisfiable $\Phi \in \mathfrak{F}$ has a w-finite model. Given a fragment $\mathfrak{L}$, a width measure w is called $\mathfrak{L}$*-friendly iff* there exists an algorithm that, taking a number $n \in \mathbb{N}$ as input, enumerates all $\Xi \in \mathfrak{L}$ that have a model $\mathcal{I}$ with $\mathsf{w}(\mathcal{I}) \leq n$. ◇

Observe that $\mathfrak{L}$-friendliness implies $\mathfrak{L}'$-friendliness for every $\mathfrak{L}' \subseteq \mathfrak{L}$. Next, we will briefly review known width notions and corresponding friendliness results. A summary can be found in Table 1 which also includes the notion of partitionwidth, which will be defined and thoroughly discussed in Section 6.

Expansion (domain size). As a very simple example, consider the *expansion* function[4] $\mathsf{expansion} : \mathcal{I} \mapsto |\mathsf{dom}(\mathcal{I})|$, mapping every countable instance to the size of its domain. Clearly, the finite-$\mathsf{expansion}$-model property is just the well-known finite model property. Moreover, $\mathsf{expansion}$ is an SO-friendly width measure: there are only finitely many instances with n elements (up to isomorphism) which can be computed and checked.

Treewidth. A somewhat less trivial example of a width measure is the popular notion of *treewidth* [RS84].

Definition 4.2 (treewidth)**.** A *tree decomposition* of an instance $\mathcal{I}$ is a (potentially infinite) tree $T = (V, E)$, where:

- $V \subseteq 2^{\mathsf{dom}(\mathcal{I})}$, i.e., each node $X \in V$ is a set of terms of $\mathcal{I}$,
- $\bigcup_{X \in V} X = \mathsf{dom}(\mathcal{I})$,
- $\mathtt{R}(t_1, \ldots, t_n) \in \mathcal{I}$ implies $\{t_1, \ldots, t_n\} \subseteq X$ for some $X \in V$,
- for each term t in $\mathcal{I}$, the subgraph of T consisting of the nodes X with $t \in X$ is connected.

The *treewidth* of a tree decomposition $T = (V, E)$ is set to $\max_{X \in V} |X| - 1$ if such a finite maximum exists and ∞ otherwise. The *treewidth* of an instance $\mathcal{I}$, denoted $\mathsf{tw}(\mathcal{I})$, is the minimal treewidth among all its tree decompositions. ◇

Treewidth is a GSO-friendly width measure [CE12, Thm. 9.18] and generalizes $\mathsf{expansion}$.

[4]The name is chosen for homogeneity, so it fits the pre-existing notion of *finite expansion sets* discussed in Section 8. There is no relationship to the notion of expansion in graph theory.

Cliquewidth. Yet another example of a width measure is the notion of *cliquewidth* [FLOR23], denoted cw, whose technical description is somewhat involved and constitutes a special case of the to-be-introduced notion of partitionwidth, so we defer the formal definition to Section 6.4. The version of cliquewidth employed by us works for arbitrary countable structures properly generalizes earlier notions of cliquewidth for finite directed edge-labelled graphs [CE12] and countable unlabelled undirected graphs [Cou04], as well as Grohe and Turán's cliquewidth notion for finite instances of arbitrary arity [GT04]. Cliquewidth is MSO-friendly [FLOR23, Thm. 11] and generalizes $\mathsf{expansion}$, whereas neither of cw and tw generalizes the other (see also Figure 1 in Section 6.4).

The next two definitions capture the two core ingredients of our framework. The first ingredient is the guaranteed existence of width-finite countermodels.

Definition 4.3 (width-based controllability)**.** Let w be a width measure and let $\mathfrak{Q}$ be a fragment. A fragment $\mathfrak{F}$ is called w-*controllable for* $\mathfrak{Q}$ (briefly noted as $\mathrm{Cont}_{\mathsf{w}}(\mathfrak{F}, \mathfrak{Q})$) *iff* for every $\Psi \in \mathfrak{Q}$ and $\Phi \in \mathfrak{F}$ with $\Phi \not\models \Psi$, there exists a w-finite model $\mathcal{I}^* \models \Phi$ with $\mathcal{I}^* \not\models \Psi$. ◇

It follows from the above definitions that $\mathrm{Cont}_{\mathsf{w}}(\mathfrak{F}, \mathfrak{Q})$ implies $\mathrm{Cont}_{\widetilde{\mathsf{w}}}(\mathfrak{F}', \mathfrak{Q}')$ whenever $\widetilde{\mathsf{w}}$ generalizes w, $\mathfrak{F}' \subseteq \mathfrak{F}$, and $\mathfrak{Q}' \subseteq \mathfrak{Q}$. The other ingredient deals with the existence of an equivalent translation between fragments.

Definition 4.4 (effective expressibility)**.** Given fragments $\mathfrak{L}$ and $\mathfrak{L}'$, we call $\mathfrak{L}$ *effectively* $\mathfrak{L}'$*-expressible iff* there exists some computable function $\mathsf{eqtrans} : \mathfrak{L} \to \mathfrak{L}'$ that satisfies $\Psi \equiv \mathsf{eqtrans}(\Psi)$. ◇

We next formulate this article's core decidability result.

Theorem 4.5. *Let* w *be a width measure,* $\mathfrak{F}$ *an FO fragment, and* $\mathfrak{Q}$ *a* $\forall$*SO fragment such that* $\mathfrak{F}$ *is* w*-controllable for* $\mathfrak{Q}$*. If there exists some fragment* $\mathfrak{L}$ *such that* w *is* $\mathfrak{L}$*-friendly and the fragment* $\{\Phi \wedge \neg\Psi \mid \Phi \in \mathfrak{F}, \Psi \in \mathfrak{Q}\}$ *is effectively* $\mathfrak{L}$*-expressible, then* $\textsc{Entailment}(\mathfrak{F}, \mathfrak{Q})$ *is decidable.*

Proof. Detecting $\Phi \models \Psi$ is semidecidable, as discussed earlier. Hence it remains to establish a semidecision procedure for $\Phi \not\models \Psi$. Thanks to the assumption that Φ is from a fragment that is w-controllable for $\mathfrak{Q}$ with $\Psi \in \mathfrak{Q}$, it follows by definition that there exists a w-finite model $\mathcal{I}^* \models \Phi$ with $\mathcal{I}^* \not\models \Psi$. Note that such a "countermodel" $\mathcal{I}^*$ can be characterized by the $\mathfrak{L}$-sentence $\mathsf{eqtrans}(\Phi \wedge \neg\Psi)$. Therefore, the following procedure will terminate, witnessing non-entailment: Enumerate all natural numbers in their natural order and for each number n run a parallel thread with the enumeration algorithm from Definition 4.1 with input n (the algorithm is guaranteed to exist due to $\mathfrak{L}$-friendliness of w). Terminate as soon as one thread produces $\mathsf{eqtrans}(\Phi \wedge \neg\Psi)$. □

Table 2 displays the corresponding decision algorithm. We note that the parallel execution of countably infinitely many "threads" in (2) can be realized by a technique commonly known as *dovetailing*: execute one computation step for the first thread, then execute one step for threads 1 and 2, then one step for each of threads 1,2, and 3, etc. This yields a fair execution, meaning that every computation step of every thread will be executed after a finite amount of time. In particular, this execution strategy ensures that the parallel branch (2) of the algorithm terminates whenever a width-finite model of $\mathsf{eqtrans}(\Phi \wedge \neg\Psi)$ exists. One should note that, beyond mere decidability, the generic framework does not give rise to any informative upper complexity bounds.

[5]Obviously, decidability entails the existence of a total recursive upper-bound function obtained from exhaustively evaluating all size-bounded inputs.

Algorithm: EntailmentCheck($\mathsf{w}, \mathsf{eqtrans}$)
Requires: $\mathrm{Cont}_{\mathsf{w}}(\mathfrak{F}, \mathfrak{Q})$, $\mathfrak{L}$-friendly w, $\mathsf{eqtrans}$
Input: $\Phi \in \mathfrak{F} \subseteq \mathrm{FO}$, $\forall\Sigma'.\Psi^* \in \mathfrak{Q} \subseteq \forall\mathrm{SO}$.
Output: YES, if $\Phi \models \Psi$, NO otherwise.

Run the following in parallel:
(1) enumerate all $\Theta \in \mathrm{FO}$ with $\Phi \models \Theta$
  if $\Theta = \Psi^*$, terminate with YES,
  next Θ.
(2) for all $n \in \mathbb{N}$, run the following in parallel:
  enumerate all $\Xi \in \mathfrak{L}$ having model $\mathcal{I}$ with $\mathsf{w}(\mathcal{I}) \leq n$,
  if $\Xi = \mathsf{eqtrans}(\Phi \wedge \neg\Psi)$, terminate with NO,
  next Ξ.

TABLE 2. Entailment decision algorithm discussed in Theorem 4.5.

Example 4.6. The proof of the previously long-standing open problem of decidability of UCQ entailment in the description logic $\mathcal{ALCHOIQ}b$ [RG10] can be seen as an application of Theorem 4.5 with $\mathfrak{F} = \mathcal{ALCHOIQ}b$ and $\mathfrak{Q} = \mathrm{UCQ}$, where $\mathfrak{L} = \mathrm{FO}$ and $\mathsf{w} = \mathsf{tw}$, i.e., treewidth is the used width notion. The most intricate part of this result amounts to showing that $\mathcal{ALCHOIQ}b$ is tw-controllable for UCQ, via an elaborate countermodel construction, producing so-called *quasi-forest models*, which are known to be tw-finite. To this day, no informative upper complexity bound for $\textsc{Entailment}(\mathcal{ALCHOIQ}b, \mathrm{UCQ})$ is known.[5]

Remark 4.7. The attentive reader might wonder whether the proposed setting is unnecessarily complicated: Wouldn't it be possible to use the established reduction from the second-order entailment $\Phi \models \Psi$ with $\Psi = \forall\Sigma'.\Psi^*$ to the first-order entailment $\Phi \models \Psi^*$ for the co-semidecision part, too? This would allow one to always pick $\mathfrak{L} = \mathrm{FO}$ and $\mathsf{eqtrans} : \Phi \wedge \neg\Psi \mapsto \Phi \wedge \neg\Psi^*$. The answer is no: while $\Phi \not\models \Psi$ coincides with $\Phi \not\models \Psi^*$, the former may have a width-finite countermodel while the latter does not.

For example, let Φ be the sentence $\exists z.\forall x.\exists y.\mathtt{S}(x, y)$, expressing that there is at least one element and that every element has an $\mathtt{S}$-successor, and let Ψ be a query checking for the existence of a directed $\mathtt{S}$-cycle: $\Psi = \forall\mathtt{P}.\Psi^*$ where Ψ^* is

$$\big(\forall xy.(\mathtt{S}(x,y) \rightarrow \mathtt{P}(x,y)) \wedge \forall xyz.(\mathtt{P}(x,y) \wedge \mathtt{P}(y,z) \rightarrow \mathtt{P}(x,z))\big) \;\rightarrow\; \exists x.\mathtt{P}(x,x).$$

Then we can see that $\Phi \not\models \Psi$ is witnessed by the countermodel $\mathcal{I} = \{\mathtt{S}(n, n{+}1) \mid n \in \mathbb{N}\}$ which has treewidth of 1, whereas any countermodel witnessing $\Phi \not\models \Psi^*$ has infinite treewidth. To see this, note that $\mathcal{I} \cup \{\mathtt{P}(n, m) \mid n < m \in \mathbb{N}\}$ is a countermodel that has infinite treewidth and it is minimal in the sense that it admits an injective homomorphism into any other countermodel, whence treewidth-infinity carries over.

Remark 4.8. The attentive and well-informed reader might question our restriction of $\mathfrak{F}$ to fragments of FO. Indeed, all the width notions w considered in this paper share the property that they are not only MSO-friendly, but in fact, for any given $n \in \mathbb{N}$, it is even decidable if an MSO sentence Φ has a model $\mathcal{I}$

with $\mathsf{w}(\mathcal{I}) \leq n$ or not. So could we not exploit this property to design a decision procedure for ENTAILMENT$(\mathfrak{F}, \mathfrak{Q})$ without reverting to completeness of FO as laid out in Section 3? In that case we could significantly extend our setting by letting $\mathfrak{F}$ be some fragment of MSO (rather than just FO) and $\mathfrak{Q}$ be a selection of MSO sentences preceded by a prefix of (higher-ary) universal second-order quantifiers. Unfortunately, however, this is generally not possible. For this approach to work, we would need to know an a-priori upper bound on a potential countermodel's width, that is, a computable function $\mathsf{maxwidth} : \mathfrak{F} \times \mathfrak{Q} \to \mathbb{N}$ satisfying that, for every $\Phi \in \mathfrak{F}$, $\Psi \in \mathfrak{Q}$ with $\Phi \not\models \Psi$, there exists a corresponding countermodel $\mathcal{I}$ with $\mathsf{w}(\mathcal{I}) \leq \mathsf{maxwidth}(\Phi, \Psi)$. While in many cases such a function is easy to establish (e.g., for GF), this does not hold in general. In some cases, such a function does provably not exist, even if $\mathfrak{F}$ is w-controllable for $\mathfrak{Q}$. As a basic example, let w be $\mathsf{expansion}$ (i.e., the domain size), let $\mathfrak{Q} = \{\mathit{false}\}$ and let $\mathfrak{F}$ consist of all sentences of the form $\Phi \wedge \Phi'$ where Φ is an arbitrary FO sentence not using the binary predicate $\mathtt{S}$, while Φ' is the fixed MSO sentence (using some MSO-expressible "macros" to enhance readability)

$$\exists x, y. \Big(\big(\neg\exists v. \mathtt{S}(v, x)\big) \wedge \big(\neg\exists v. \mathtt{S}(y, v)\big) \wedge \big(\forall z{\neq}y. \exists! z'. \mathtt{S}(z, z')\big) \wedge \big(\forall z{\neq}x. \exists! z'. \mathtt{S}(z', z)\big) \\ \wedge \big(\forall v. \forall X{\ni}x. (\forall z, z'. X(z) \wedge \mathtt{S}(z, z') \to X(z')) \;\to\; X(v)\big) \\ \wedge \big(\forall v. \forall X{\ni}v. (\forall z, z'. X(z) \wedge \mathtt{S}(z, z') \to X(z')) \;\to\; X(y)\big)\Big),$$

which forces every model's domain to be a finite total order with successor relation $\mathtt{S}$. Due to this, every countermodel for $\Phi \wedge \Phi' \models \mathit{false}$ must have finite domain size and consequently we have established that $\mathfrak{F}$ is $\mathsf{expansion}$-controllable for $\{\mathit{false}\}$. Now, toward a contradiction, suppose a computable function $\mathsf{maxwidth}$ as above existed for our case. Then we could use it to decide finite unsatisfiability of any $\mathtt{S}$-free FO sentence Φ by deciding the entailment $\Phi \wedge \Phi' \models \mathit{false}$ using the strategy sketched above. Yet, finite satisfiability of FO sentences is known to be undecidable by Trakhtenbrot's Theorem [Tra50].

Please note that the ENTAILMENTCHECK algorithm from Table 2 could also be run on a "best effort basis" with very little context information: for every Φ, Ψ, and $\mathfrak{L}$-translation of Ψ, one could execute it using the most general known width notion w available (with partitionwidth being the recommended choice). Whenever the algorithm terminates, it will provide the correct answer. It will only fail to terminate in case $\Phi \not\models \Psi$ with every countermodel having infinite width.

We demonstrate the generality of our width-related notion of controllability by giving a few examples of scenarios covered by it. $\mathfrak{F}$ is w-controllable for $\mathfrak{Q}$ in the following cases:

- $\mathfrak{F}$ has the finite-w model property and $\mathfrak{Q}$ contains only unsatisfiable sentences. This coincides with the task of (un)satisfiability checking in $\mathfrak{F}$ and (for any width notion discussed here) subsumes all fragments known to have the finite-model property, which, among others, is the case for all description logics subsumed by $\mathcal{SROI}b_s$, the majority of the prefix classes of FO with a decidable satisfiability problem [BGG97], the two-variable fragment of FO (FO^2 [Mor75, GKV97]), the guarded fragment (GF [ANvB98, Grä99]), the unary negation fragment (UNF [StC13]), the guarded negation fragment (GNF [BCS15]) and the triguarded fragment (TGF [RS18, KR21]). An interesting case in this respect is the counting two-variable fragment of FO ($\mathcal{C}^2$ [GOR97]). It is known to not have the finite-model property and can be easily seen to not have the finite-tw model

property either. We conjecture, however, that it has the finite-cw-model property.

- w generalizes expansion, $\mathfrak{Q}$ contains only conjunctive queries and $\mathfrak{F}$ is finitely controllable. This applies to description logics subsumed by $\mathcal{ALCHOIb}$, and to GF, UNF, and GNF [BGO14]. Contrariwise, CQ entailment for FO^2, $\mathcal{C}^2$, and TGF is known to be undecidable [Ros07], so these logics must fail to be w-controllable for CQs and any FO-friendly w.

In the next section, we discuss an important further case, by means of which we can establish controllability for query languages way beyond (U)CQs.

5. The Case of Homomorphism-Closed Queries

While the presented framework provides a uniform way of showing decidability of entailment problems, establishing the preconditions of Theorem 4.5, and in particular controllability, can be arbitrarily difficult and may require ad hoc proof techniques in many cases. It is therefore desirable to single out sufficient conditions for controllability that cover comprehensive specification and query languages. It turns out that fragments which are "homomorphically benign" in a certain sense strike a good balance in that respect.

Definition 5.1 ((finite) universality, homomorphism-closedness)**.** Let $\mathcal{I}$ be a set of models of a sentence Φ. $\mathcal{I}$ is *universal*, *iff* for every model $\mathcal{J} \models \Phi$ there is some $\mathcal{I} \in \mathcal{I}$ that maps homomorphically into $\mathcal{J}$. $\mathcal{I}$ is *finitely universal*, *iff* for every model $\mathcal{J} \models \Phi$ there is some $\mathcal{I} \in \mathcal{I}$ such that every finite subset of $\mathcal{I}$ maps homomorphically into $\mathcal{J}$. In case $\mathcal{I} = \{\mathcal{I}\}$, we call $\mathcal{I}$ a *(finitely) universal model* of Φ.

A sentence Ψ is called *homomorphism-closed iff* whenever a model $\mathcal{I} \models \Psi$ admits a homomorphism into some instance $\mathcal{J}$, then $\mathcal{J}$ is also a model of Ψ. ◇

Clearly, every universal model (set) is also finitely universal. The following Theorem is folklore for universal models and well-known for universal model sets [BMMP16]. The generalization using finite universality is novel and a bit more involved.

Theorem 5.2 (finite universality and homclosedness)**.** *Let Φ be an FO sentence having some finitely universal model set $\mathcal{I}$ and let Ψ be a homomorphism-closed $\forall$SO sentence. Then, $\Phi \not\models \Psi$ iff there exists a model $\mathcal{I} \in \mathcal{I}$ with $\mathcal{I} \not\models \Psi$.*

The proof of this theorem (see appendix) hinges on the insight that every model of some homomorphism-closed ∀SO sentence has a finite submodel. This same insight can also be utilized to show the following statement (also, see the appendix for the proof), which can be understood as a sort of interpolation result and might be interesting in itself.

Lemma 5.3. *Let Φ be a FO sentence and Ψ a homomorphism-closed $\forall$SO sentence. Then, $\Phi \models \Psi$ holds iff there exists a union of BCQs $Q = \bigvee_{i=1}^{n} q_i$ satisfying $\Phi \models Q$ and $Q \models \Psi$.*

Theorem 5.2 ensures that for homomorphism-closed queries, the search for countermodels can be confined to any finitely universal model set of Φ. That means, if some finitely universal model set can be guaranteed to consist only of structurally well-behaved models, we can ensure controllability.[6]

[6]We note that finite universality is demonstrably better than universality: there are known cases where Φ has a finitely universal model of finite width yet any universal model has infinite width [BMR23].

Theorem 5.4. *Let* w *be a width measure. Let* $\mathfrak{Q}$ *be a* $\forall$*SO fragment of homomorphism-closed sentences and let* $\mathfrak{F}$ *be a FO fragment of which every sentence exhibits some finitely universal model set containing only* w*-finite structures. Then,* $\mathfrak{F}$ *is* w*-controllable for* $\mathfrak{Q}$*.*

Proof. Consider arbitrary $\Psi \in \mathfrak{Q}$ and $\Phi \in \mathfrak{F}$ with $\Phi \not\models \Psi$. By assumption, there exists a finitely universal model set $\mathcal{J}$ of Φ containing only w-finite structures and by Theorem 5.2, there must be some $\mathcal{I}^* \in \mathcal{J}$ with $\mathcal{I}^* \not\models \Psi$. Thus $\mathfrak{F}$ is w-controllable for $\mathfrak{Q}$. □

Combining Theorem 5.4 and Theorem 4.5, we directly obtain the subsequent corollary.

Corollary 5.5. *Let* w *be a width measure,* $\mathfrak{Q}$ *a* $\forall$*SO fragment of homomorphism-closed sentences, and* $\mathfrak{F}$ *an FO fragment whose every sentence has a finitely universal model set of only* w*-finite instances. If there is a fragment* $\mathfrak{L}$ *such that* w *is* $\mathfrak{L}$*-friendly and the fragment* $\{\Phi \wedge \neg\Psi \mid \Phi \in \mathfrak{F}, \Psi \in \mathfrak{Q}\}$ *is effectively* $\mathfrak{L}$*-expressible, then* $\textsc{Entailment}(\mathfrak{F}, \mathfrak{Q})$ *is decidable.*

With this corollary in place, we will next review the expressivity boundaries of the query languages for which decidability can be warranted, depending on the underlying width measures' friendliness properties. Note that there is a trade-off: less general width measures allow for less expressive choices for the specification language $\mathfrak{F}$ but – due to being generally friendlier – admit more expressive query languages $\mathfrak{Q}$.

MSO-friendly width measures. In this case (which comprises all width measures discussed in this article), the most expressive query language is the semantically defined class $\mathfrak{Q}$ of all *h*omomorphism-closed *u*niversal *s*econd-*o*rder / *m*onadic *s*econd-*o*rder *q*ueries – henceforth abbreviated as HUSOMSOQs (pronounced "hue-somm-socks"). This begs the question how comprehensive and expressive this class actually is and, in particular, which popular syntactically defined query languages it encompasses. An effective syntax characterizing the entire class is still elusive, but it subsumes two very comprehensive and expressive logical fragments (and through these many other query languages):

- **Nested monadically defined queries.** This well-investigated fragment (abbreviated NEMODEQs) constitutes a very expressive yet computationally manageable formalism, admitting polytime translations into MSO and datalog (and hence homomorphism-closed ∀SO) [RK13]. We will refrain from recalling the somewhat involved syntax here and instead highlight that NEMODEQs subsume (unions of) Boolean CQs, *monadic datalog queries* [CGKV88], *(unions of) conjunctive 2-way regular path queries* ((U)C2RPQs) [FLS98], and nested versions thereof (e.g. *regular queries* [RRV17] as well as others [BKR14]).
- **Monadic disjunctive datalog queries.** As monadic disjunctive datalog (MDDlog) queries (see Section A) are "natively" contained both in ∀SO and in MSO, we can pick the identity function for `eqtrans` to witness effective MSO-expressibility. Homomorphism-closedness follows from the contained-ness in disjunctive datalog. MDDlog queries can verify complex properties such as an instance's non-membership in arbitrary finite-domain CSPs (e.g., non-3-colorability), but are also known to capture many ontology-mediated queries based on description logics [BtCLW14, FKL19].

In terms of expressivity, the classes of MDDlog queries and NEMODEQs are incomparable (unlike NEMODEQ, MDDlog cannot recognize graphs with directed cycles;

unlike MDDlog, NEMODEQ cannot recognize non-3-colorable graphs). The semantic intersection of the two classes contains UCQs and monadic datalog queries.

For comprehensive investigations into FO fragments that allow for decidable HUSOMSOQ entailment on the grounds of MSO-friendly width measures, we point the reader to Example 6.26 and Example 7.1 later in the article.

GSO-friendly width measures. In this case, where treewidth is the prominent corresponding width notion, we can establish decidability for all homomorphism-closed queries that are simultaneously expressible in ∀SO and GSO. A syntactic characterization of this entire class seems not within reach, despite some interesting results regarding the characterization of homomorphism-closed GSO in terms of infinite-domain CSPs [BKR21, Cor. 21]. The arguably most expressive known effectively GSO-expressible homomorphism-closed query languages in ∀SO are *frontier-guarded disjunctive datalog* (FGDDlog) on one hand, and on the other hand *nested frontier-guarded flag & check queries*, sometimes shortened to *nested guarded queries* and usually abbreviated by GQ^+ [BKR15].

FGDDlog is syntactically contained in ∀SO and its expressibility in GSO is a straightforward consequence of GSO's alternative characterization via the guarded semantics. The homomorphism-closedness of FGDDlog queries follows from their inclusion in disjunctive datalog. FGDDlog contains all MDDlog queries (see above) and their subclasses.

GQ^+ is expressible in datalog (hence homomorphism-closed and in ∀SO) [BKR15] and has also been shown to be expressible in GSO [BKR21, Prop. 47]. GQ^+ subsumes the class of NEMODEQs (see above) and all its subclasses.

Example 5.6. For numerous logical formalisms, the existence of treewidth-finite (finitely) universal model sets can be established through well-known model-transformations: For GF, UNF, and GNF as well as a variety of very expressive description logics [BHLS17, Rud11], specific techniques commonly referred to as *unravelling* transform an arbitrary model $\mathcal{I}$ into a treewidth-finite model $\mathcal{J}$ that homomorphically maps into $\mathcal{I}$ [GHO02, BBV16, BR19, BR23, CEO09, BKR14]. Thus, due to GSO-friendliness of treewidth, Corollary 5.5 ensures in one sweep decidability of entailment of all effectively GSO-expressible homomorphism-closed ∀SO queries for all these formalisms.

Consequently, if $\mathfrak{F}$ is any of GF, UNF, GNF, $\mathcal{ALCHOI}b$, $\mathcal{ALCHIQ}b$, $\mathcal{ALCHOQ}b$, and $\mathfrak{Q}$ is some effectively GSO-expressible fragment of homomorphism-closed ∀SO queries, then $\mathfrak{F}$ is treewidth-controllable for $\mathfrak{Q}$ and thus $\textsc{Entailment}(\mathfrak{F}, \mathfrak{Q})$ is decidable.[7]

While for simple instances of $\mathfrak{Q}$ like CQs or queries with regular path expressions, decidability of $\textsc{Entailment}(\mathfrak{F}, \mathfrak{Q})$ has been known for all these instances of $\mathfrak{F}$, our result not only provides a generic common ground for these prior results, but also generalizes them to the above-mentioned more expressive query languages of FGDDlog and GQ^+.

SO-friendly width measures. In this case, the semantic definition of the most expressive eligible query language simplifies to that of all homomorphism-closed ∀SO sentences. A very expressive syntactically defined query language that falls in this category is *disjunctive datalog* (DDlog) [EGM97], which includes all of datalog (Dlog). To the best of our knowledge, no syntactic characterization of

[7]It should be noted that this corollary does not cover the case $\mathfrak{F} = \mathcal{ALCHOIQ}b$ from Example 4.6, since the employed construction to convert an arbitrary countermodel $\mathcal{I}$ into a finite-treewidth countermodel $\mathcal{J}$ does not ensure that $\mathcal{J}$ homomorphically maps into $\mathcal{I}$. In fact, the decidability of the entailment of queries beyond UCQs from $\mathcal{ALCHOIQ}b$ is wide open.

homomorphism-closed ∀SO has been established (whereas one has been proposed for homomorphism-closed SO as a whole [BFKR21, Cor. 61]). So far, we are not aware of any query in homomorphism-closed ∀SO that is provably not expressible in DDlog, but we still find it unlikely that DDlog captures all of homomorphism-closed ∀SO.

6. Partitionwidth: Definition and Properties

The notion of *partitionwidth* by Blumensath [Blu03, Blu06] is a parameterized measure originally developed for model-theoretic structures of arbitrary cardinality and with countable signatures. Its purpose is to classify such structures according to the "simplicity" of their MSO theory. For the sake of understandability, we will refrain from introducing the notion in its full generality and confine ourselves to a restricted and notation-adjusted version applicable to countable instances over finite signatures, which is sufficient for our purposes.

Before going into the full formal details in Section 6.1, let us provide an intuitive description of the underlying high-level ideas. The notion of some instance's partitionwidth is based on the idea that we iteratively construct or assemble said instance from its singleton elements (e.g., terms). To this end, we utilize *colors* of various arities (mostly denoted with lowercase greek latters κ, μ, λ) which we dynamically assign to element tuples of the appropriate length. Initially, each singleton element e is assigned the original unary color o_1, the pair (e, e) the original binary color o_2, and so forth. During the assembly process, color assignments can be extended and manipulated in various ways. However, at any time, any tuple can only carry at most one color. Similar to the mechanism of assigning colors to tuples, it is also possible to assign them predicates from the underlying signature. As opposed to colors, such predicate assignments are immutable and several distinct predicates may be assigned to the same tuple of elements.

In some more detail, the assembly of a colored instance is carried out through the repeated execution of construction operations each of which is one of the following:

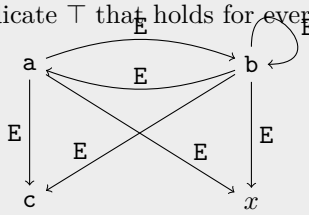
- A nullary operation, denoted $\texttt{New}_{\texttt{c}}$ for some constant name `c`, creating a one-element instance containing the term `c`, with the original color assignments.
- A nullary operation, denoted $\texttt{New}_{*}$, creating a one-element instance with an "unnamed" (i.e., non-constant) element, with the original color assignments.
- A binary operation, denoted by the infix operator $\oplus_{Ins}$, where *Ins* is a finite set of *instructions*. The operation takes two colored instances as input, creates their *disjoint union*, and executes color and predicate modifications according to the instructions contained in *Ins*. The different types of instructions and their effects are as follows:
  - color-adding: the instruction $\lambda\texttt{-Add}^{I}_{\kappa\mu}$ assigns the color λ to any tuple obtained by I-shuffling a κ-colored tuple from the first input instance with a μ-colored tuple from the second,
  - re-coloring: the instruction $\lambda\texttt{-ReCol}^{0}_{\kappa}$ changes the color of all κ-colored tuples from the first input instance to λ (in the same way, the instruction $\lambda\texttt{-ReCol}^{1}_{\kappa}$ re-colors tuples originating from the second input instance),
  - predicate-adding: the instruction $\texttt{R-Add}^{I}_{\kappa\mu}$ assigns the predicate `R` to any tuple obtained by I-shuffling a κ-colored tuple of the first instance with a μ-colored tuple of the second,
  - predicate-materializing: the instruction $\texttt{R-Add}^{0}_{\kappa}$ assigns the predicate `R` to every κ-colored tuple from the first input instance (likewise, the instruction $\texttt{R-Add}^{1}_{\kappa}$ materializes predicates out of colored tuples originating from the second input instance).

After the described assembly process over colored instances has finished, we obtain the resulting instance we wanted to build by "forgetting" all colors and only keeping the predicates. Then, an instance's partitionwidth is determined by the minimal set of colors required to create it in the way described above. In the finite case, an instance's "assembly plan" can be conveniently described by an algebraic expression built from the operators introduced above. Such an expression corresponds to a syntax tree.

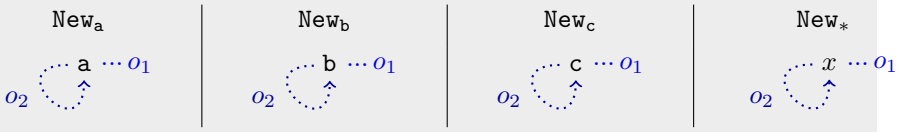
Example 6.1. Consider the finite instance

$$\mathcal{I}_{\text{ex}} = \{\mathtt{E}(\mathtt{a},\mathtt{b}), \mathtt{E}(\mathtt{a},\mathtt{c}), \mathtt{E}(\mathtt{a},x), \mathtt{E}(\mathtt{b},\mathtt{a}), \mathtt{E}(\mathtt{b},\mathtt{b}), \mathtt{E}(\mathtt{b},\mathtt{c}), \mathtt{E}(\mathtt{b},x)\},$$

over $\Sigma = \{\mathtt{E}\}$ with just one binary predicate $\mathtt{E}$, whose active domain consists of the three constants $\mathtt{a}$, $\mathtt{b}$, and $\mathtt{c}$, as well as a null x. The instance can be graphically depicted as follows (here and in the following, we omit displaying the universal unary predicate $\top$ that holds for every term).



Assume we want to construct $\mathcal{I}_{\text{ex}}$ (up to isomorphism) via an assembly process of the kind described above. There are several ways of doing so, but in any case, one has to start by creating one-term-instances by means of appropriate $\mathtt{New}$ commands. So we obtain the following four colored instances (we use dotted lines for color assignments):

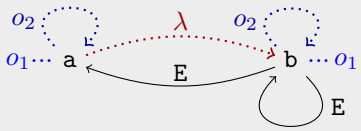


Each such initial instance is endowed with the original colors o_1 (unary) and o_2 (binary). The name of the null introduced by $\mathtt{New}_*$ is irrelevant (recall that we only aim at constructing the above instance up to isomorphism), we just chose x for convenience. It is, however, required that every "call" of $\mathtt{New}_*$ introduces a null that is "fresh", that is, not otherwise used in other independently constructed assembly components.

Proceeding with the assembly process, we decide to combine the colored instance produced by $\mathtt{New}_\mathtt{a}$ with that produced by $\mathtt{New}_\mathtt{b}$ by taking their disjoint union and concurrently executing the following three instructions:

- $\lambda\text{-}\mathtt{Add}^{\{1\}}_{o_1 o_1}$ – assigning the binary color λ to all pairs of terms whose first component is a o_1-colored term of the $\mathtt{New}_\mathtt{a}$ instance and whose second component is a o_1-colored term of the $\mathtt{New}_\mathtt{b}$ instance; the only pair meeting these conditions in our case is $(\mathtt{a},\mathtt{b})$.
- $\mathtt{E}\text{-}\mathtt{Add}^{\{2\}}_{o_1 o_1}$ – assigning the binary predicate $\mathtt{E}$ to all pairs of terms whose first component is a o_1-colored term of the $\mathtt{New}_\mathtt{b}$ instance and whose second component is a o_1-colored term of the $\mathtt{New}_\mathtt{a}$-instance; this way, the atom $\mathtt{E}(\mathtt{b},\mathtt{a})$ is introduced.
- $\mathtt{E}\text{-}\mathtt{Add}^{1}_{o_2}$ – assigning the binary predicate $\mathtt{E}$ to all o_2-colored pairs of terms of the $\mathtt{New}_\mathtt{b}$-instance; this way, the atom $\mathtt{E}(\mathtt{b},\mathtt{b})$ is introduced.

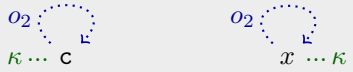
The result of this composite operation is the colored instance depicted right below which is described by the expression $\mathtt{New}_\mathtt{a} \oplus_{\{\lambda\text{-}\mathtt{Add}^{\{1\}}_{o_1 o_1}, \mathtt{E}\text{-}\mathtt{Add}^{1}_{o_2}, \mathtt{E}\text{-}\mathtt{Add}^{\{2\}}_{o_1 o_1}\}} \mathtt{New}_\mathtt{b}$.



Next, we choose to combine the colored instances described by $\texttt{New}_\texttt{c}$ and $\texttt{New}_*$ by, again, taking their disjoint union and then concurrently executing the following two instructions:

- $\kappa\text{-}\texttt{ReCol}^0_{o_1}$ – changing the color of any o_1-colored term from the $\texttt{New}_\texttt{c}$-instance to κ; the unary color of $\texttt{c}$ is now κ.
- $\kappa\text{-}\texttt{ReCol}^1_{o_1}$ – likewise changing the color of any o_1-colored term from the $\texttt{New}_*$-instance to κ; the unary color of x is now κ as well.

The result of this composite operation is the colored instance depicted right below which is described by the expression $\texttt{New}_\texttt{c} \oplus_{\{\kappa\text{-}\texttt{ReCol}^0_{o_1}, \kappa\text{-}\texttt{ReCol}^1_{o_1}\}} \texttt{New}_*$.



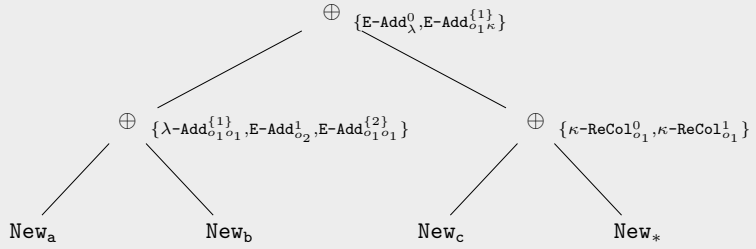
Finally, we combine the two just created colored structures, represented by the expressions $\texttt{New}_\texttt{a} \oplus_{\{\lambda\text{-}\texttt{Add}^{\{1\}}_{o_1 o_1}, \texttt{E-Add}^1_{o_2}, \texttt{E-Add}^{\{2\}}_{o_1 o_1}\}} \texttt{New}_\texttt{b}$ and $\texttt{New}_\texttt{c} \oplus_{\{\kappa\text{-}\texttt{ReCol}^0_{o_1}, \kappa\text{-}\texttt{ReCol}^1_{o_1}\}} \texttt{New}_*$, by taking their disjoint union and concurrently executing the following two instructions:

- $\texttt{E-Add}^0_\lambda$ – assigning the binary predicate $\texttt{E}$ to all λ-colored pairs of terms from the former instance; this way, the atom $\texttt{E}(\texttt{a}, \texttt{b})$ is introduced.
- $\texttt{E-Add}^{\{1\}}_{o_1 \kappa}$ – assigning the binary predicate $\texttt{E}$ to all pairs of terms whose first component is an o_1-colored term of the former instance and whose second component is a κ-colored term of the latter instance; this way, the atoms $\texttt{E}(\texttt{a}, \texttt{c})$, $\texttt{E}(\texttt{a}, x)$, $\texttt{E}(\texttt{b}, \texttt{c})$, and $\texttt{E}(\texttt{b}, x)$ are introduced in one go.

The final result of this operation is a colored structure, described by the expression

$$(\texttt{New}_\texttt{a} \oplus_{\{\lambda\text{-}\texttt{Add}^{\{1\}}_{o_1 o_1}, \texttt{E-Add}^1_{o_2}, \texttt{E-Add}^{\{2\}}_{o_1 o_1}\}} \texttt{New}_\texttt{b}) \oplus_{\{\texttt{E-Add}^0_\lambda, \texttt{E-Add}^{\{1\}}_{o_1 \kappa}\}} (\texttt{New}_\texttt{c} \oplus_{\{\kappa\text{-}\texttt{ReCol}^0_{o_1}, \kappa\text{-}\texttt{ReCol}^1_{o_1}\}} \texttt{New}_*)$$

which – when disregarding the coloring – is isomorphic to the initially given instance $\mathcal{I}_{\text{ex}}$. The fact that 4 colors needed for the construction implies that $\mathcal{I}_{\text{ex}}$'s partitionwidth is at most 4. Arguably, the structure of the "assembly expression" above can be made more transparent by displaying its syntax tree below. Also, thinking of assembly expressions as binary trees will be crucial for the formal developments in the following sections.



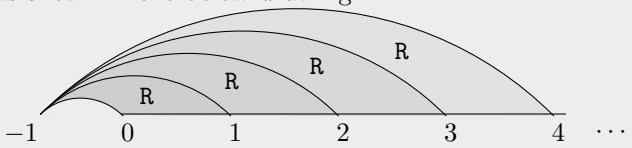
For finite instances, as in the example just discussed, the underlying ideas are very much in line with the notion of inductive definitions and should be easy to comprehend. Opposed to this, for infinite instances, the much more elusive idea

of an "infinite assembly plan" has to be implemented by resorting to assembly expressions that correspond to infinite, "unfounded" syntax trees. Such non-standard expressions describe construction processes that have an end but no proper beginning. According to our experience, this concept is cognitively hard to grasp and it takes a while to develop good and reliable intuitions about it. We will precede the formal treatment with an example to help developing the appropriate mindset, but certain aspects of the example will have to remain sketchy until the rigid formal development in the subsequent section.

Example 6.2. We consider the signature $\Sigma = \{\mathtt{R}\}$, which just contains the ternary predicate $\mathtt{R}$, and the infinite, constant-free instance

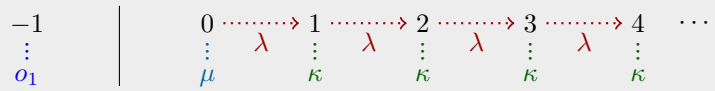
$$\mathcal{I}_{\text{tern}} = \{\mathtt{R}(-1, n, n{+}1) \mid n \in \mathbb{N}\},$$

where, for the sake of simplicity, we use the integers -1, 0, 1, 2, ... as nulls from $\mathbf{N}$. That is, $\mathcal{I}_{\text{tern}}$ contains the atoms $\mathtt{R}(-1, 0, 1)$, $\mathtt{R}(-1, 1, 2)$, $\mathtt{R}(-1, 2, 3)$, and so forth, as shown in the below drawing.



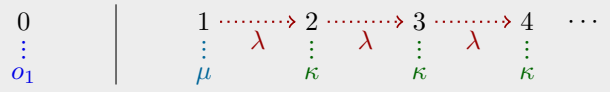
Due to the already mentioned peculiarity that assembling $\mathcal{I}_{\text{tern}}$ does not have a proper starting point, we will argue "backward" by starting from $\mathcal{I}_{\text{tern}}$ and discussing how it could have been created through a combination of colored instances, then asking the same question for it's constituents, etc.

To this end, we first note that $\mathcal{I}_{\text{tern}}$ could have arisen from the two colored instances



by taking their disjoint union and executing the instruction $\mathtt{R\text{-}Add}^{\{1\}}_{o_1\lambda}$ – assigning the ternary predicate $\mathtt{R}$ to all triples of terms whose first component is an o_1-colored term of the left instance and whose second and third component taken together as a pair carry the color λ in the right instance; this way, all the infinitely many atoms of $\mathcal{I}_{\text{tern}}$ are introduced at once.

While the left of the two input instances can be directly created through a $\mathtt{New}_*$ call (in this case, the only original color needed is the unary color o_1), the right input instance is itself infinite. For now we will denote the yet-to-be determined assembly expression for the right instance with E, whence the assembly expression of $\mathcal{I}_{\text{tern}}$ becomes $\mathtt{New}_* \oplus_{\{\mathtt{R\text{-}Add}^{\{1\}}_{o_1\lambda}\}} E$. In our quest to reverse-engineer the E-instance further, we find that it can be garnered from the two colored instances



by, once more, taking their disjoint union and then concurrently executing the following three instructions:

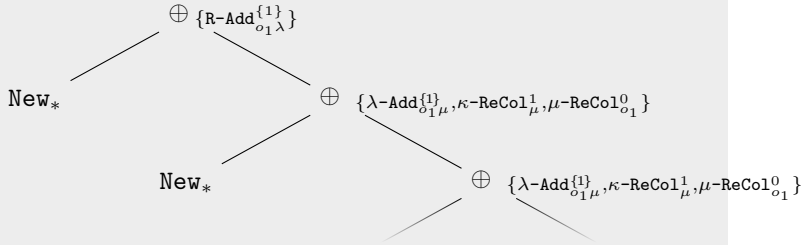
- $\lambda\mathtt{\text{-}Add}^{\{1\}}_{o_1\mu}$ – assigning the binary color λ to all pairs of terms whose first component is a o_1-colored term of the left instance and whose second component is a μ-colored term of the right instance; the only pair meeting these conditions in our case is $(0, 1)$.

- κ-$\mathtt{ReCol}^1_\mu$ – changing the color of any μ-colored term from the right instance to κ.
- μ-$\mathtt{ReCol}^0_{o_1}$ – changing the color of any o_1-colored term from the left instance to μ.

We find that – once more – the left instance can be created via $\mathtt{New}_*$, while the right instance is isomorphic to the E-instance we just disassembled. As the concrete names of the nulls used are irrelevant, we find that the assembly expression E creates the right instance as well. It is easy to see that the disassembly step can be created over and over again ad infinitum, where every null from $\{0, 1, \ldots\}$ will at some stage be obtained via a $\mathtt{New}_*$ call. We also note that, given the observation above, the expression E must satisfy the "recursive equation"

$$E = \mathtt{New}_* \oplus_{\{\lambda\text{-}\mathtt{Add}^{\{1\}}_{o_1\mu},\kappa\text{-}\mathtt{ReCol}^1_\mu,\mu\text{-}\mathtt{ReCol}^0_{o_1}\}} E.$$

While it is clear that this requirement cannot be realized by any finite string, one may choose to extend the setting to include "unfounded" expressions of infinite depth. Such expressions are difficult to grasp as linear sequences of symbols (at best, one might envision E as the result of expanding E with the right hand side of the above recursive equation infinitely often). But they may be easier to understand by means of the corresponding syntax trees. Below we depict the initial part of an (infinite) syntax tree representing $\mathcal{I}_{\text{tern}}$, stating that the subtree rooted in the root's right child coincides with the subtree rooted in the root's right-right grandchild, giving rise to a periodic infinite downward branch.



We find that, despite being infinite, the assembly expression only makes use of the four colors o_1, μ, κ, and λ. This allows us to conclude that the partitionwidth of $\mathcal{I}_{\text{tern}}$ is at most 4.

The following section will provide full formal details on partitionwidth. In particular, we will formalize the hitherto rather sketchy idea of a potentially infinite "assembly-plan-encoding syntax trees" by defining such a tree as a countably infinite instance of a particular shape.

6.1. **Formal Definition of Partitionwidth of Countable Instances.**

Definition 6.3 (Update Instructions)**.** Let Σ be a finite signature and let $\mathbb{L}$ be a finite set of *colors*, where every $\lambda \in \mathbb{L}$ is assigned an arity $\mathsf{ar}(\lambda) \in \mathbb{N}_+$. The set $\mathrm{Instr}(\mathbb{L}, \Sigma)$ of *update instructions over* $(\mathbb{L}, \Sigma)$ consists of the following expressions:

- λ-$\mathtt{Add}^I_{\kappa\mu}$ for $\kappa, \mu, \lambda \in \mathbb{L}$ with $\mathsf{ar}(\kappa) + \mathsf{ar}(\mu) = \mathsf{ar}(\lambda)$, $I \subsetneq \{1, ..., \mathsf{ar}(\lambda)\}$, and $\mathsf{ar}(\kappa) = |I|$,
- λ-$\mathtt{ReCol}^i_\kappa$ for $i \in \{0, 1\}$ and $\lambda, \kappa \in \mathbb{L}$ with $\mathsf{ar}(\lambda) = \mathsf{ar}(\kappa)$,
- $\mathtt{R}$-$\mathtt{Add}^I_{\kappa\mu}$ for $\kappa, \mu \in \mathbb{L}$ and $\mathtt{R} \in \Sigma$ with $\mathsf{ar}(\kappa) + \mathsf{ar}(\mu) = \mathsf{ar}(\mathtt{R})$, $I \subsetneq \{1, ..., \mathsf{ar}(\mathtt{R})\}$, and $\mathsf{ar}(\kappa) = |I|$,
- $\mathtt{R}$-$\mathtt{Add}^i_\kappa$ for $i \in \{0, 1\}$, $\kappa \in \mathbb{L}$ and $\mathtt{R} \in \Sigma$ with $\mathsf{ar}(\mathtt{R}) = \mathsf{ar}(\kappa)$.

Two instructions of the form $\lambda\text{-}\mathtt{Add}^I_{\kappa\mu}$ or $\lambda\text{-}\mathtt{ReCol}^i_\kappa$ will be called *incompatible*, if they disagree on λ while all other parameters coincide. $\diamond$

Note that since $\mathbb{L}$ and Σ are all assumed to be finite, the set $\mathrm{Instr}(\mathbb{L}, \Sigma)$ is finite as well.

Example 6.4. Exemplars of the instructions just formally introduced have already been seen in action informally in Example 6.1 and Example 6.2, which should facilitate understanding their purpose. To exemplify the intended effect of instructions of the type $\lambda\texttt{-Add}^I_{\kappa\mu}$ (and, in an analogue manner, $\texttt{R-Add}^I_{\kappa\mu}$), consider the following case. When executing the instruction $\lambda\texttt{-Add}^{\{2,4\}}_{\kappa\mu}$ on two colored instances, where

- the first colored instance has domain $\{\texttt{a},\texttt{b},\texttt{c}\}$ wherein the pairs $(\texttt{a},\texttt{a})$, $(\texttt{a},\texttt{b})$, and $(\texttt{a},\texttt{c})$ carry the binary color κ and no tuple carries the quaternary color λ, and
- the second colored instance has domain $\{\texttt{d},\texttt{e},\texttt{f}\}$ wherein the pairs $(\texttt{d},\texttt{e})$ and $(\texttt{e},\texttt{f})$ carry the binary color μ and no tuple carries the quaternary color λ,

the result is an instance with domain $\{\texttt{a},\texttt{b},\texttt{c},\texttt{d},\texttt{e},\texttt{f}\}$ carrying all colors and predicates of the input instances and, additionally, the color λ is assigned to the following quadruples:

$$(\texttt{d},\texttt{a},\texttt{e},\texttt{a}),\quad (\texttt{e},\texttt{a},\texttt{f},\texttt{a}),\quad (\texttt{d},\texttt{a},\texttt{e},\texttt{b}),\quad (\texttt{e},\texttt{a},\texttt{f},\texttt{b}),\quad (\texttt{d},\texttt{a},\texttt{e},\texttt{c}),\quad (\texttt{e},\texttt{a},\texttt{f},\texttt{c}),$$

which are obtained from combining every κ-pair from the first colored instance with every μ-pair from the second colored instance by putting the first and second entry of the κ-pair in positions 2 and 4, respectively, while putting the first and second entry of the μ-pair in the remaining positions 1 and 3, respectively. Note that the "internal position-order" of each contributing pair is preserved.

As indicated above, the idea of "instance assembly expressions" – meant to serve as blueprints for the instance construction – plays a central role in our development. As such "assembly expressions" may be infinite, it is helpful to encode them by their corresponding syntax trees. Such syntax trees will themselves take the form of instances of a very particular tree-like shape. The next definition specifies how an infinite binary tree, whose nodes might possibly carry some "markings" or "adornments", will be expressed as an instance.

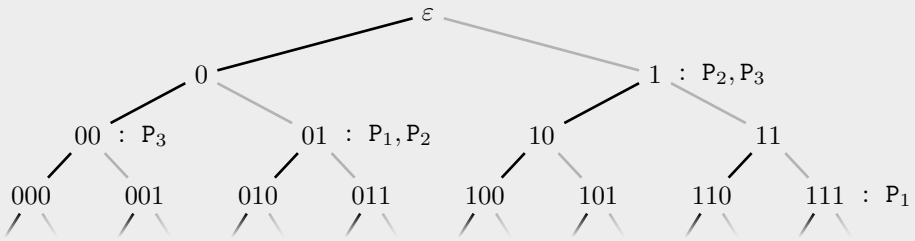
Definition 6.5 ((Adorned) Infinite Binary Tree)**.** The *infinite binary tree* is the instance

$$\mathcal{T}_{\text{bin}} = \left\{\texttt{Succ}_0(s, s0) \;\middle|\; s \in \{0,1\}^*\right\} \cup \left\{\texttt{Succ}_1(s, s1) \;\middle|\; s \in \{0,1\}^*\right\}$$

with two binary predicates $\texttt{Succ}_0$ and $\texttt{Succ}_1$. We use finite sequences composed of the symbols 0 and 1 to identify the infinitely many nulls of $\mathcal{T}_{\text{bin}}$. The *root* of $\mathcal{T}_{\text{bin}}$ is the null identified by the empty sequence, denoted ε.

An *adorned infinite binary tree* (short: adorned tree) is obtained by extending $\mathcal{T}_{\text{bin}}$ with arbitrarily many atoms of the form $\texttt{P}(s)$ where $\texttt{P}$ is a unary predicate from a finite signature and $s \in \{0,1\}^*$. Let $\mathbb{T}$ denote the set of all adorned trees. ◇

Example 6.6. The below drawing illustrates the concept of adorned infinite binary trees just defined. It shows (the initial part of) some tree $\mathcal{T} \in \mathbb{T}$ adorned with unary predicates $\texttt{P}_1$, $\texttt{P}_2$, and $\texttt{P}_3$. $\texttt{Succ}_0$ and $\texttt{Succ}_1$ are represented as black (left) and gray (right) downward edges, respectively.



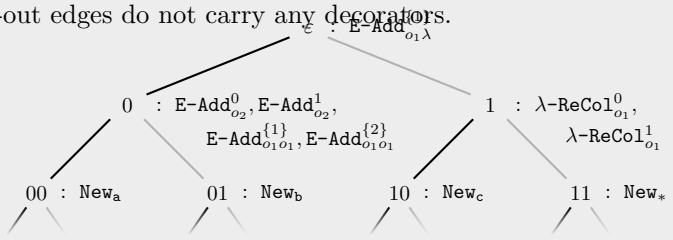
After specifying how adorned infinite binary trees are encoded as instances, the next step consists in providing the proper adornments for our purpose (referred to as *decorators*) and specify local and global constraints regulating where which decorators can be put inside the tree-instance. Intuitively, the purpose of these deliberations is to ensure that there is a clear one-to-one correspondence between properly adorned tree-instances (referred to as *well-decorated trees*) and assembly syntax trees as originally envisioned.

Definition 6.7 (Decorators and Decorated Trees)**.** Let Σ and $\mathbb{L}$ be as in Definition 6.3 and let $\mathrm{Cnst} \subseteq \mathbf{C}$ be a finite set of constants. $\mathbb{L}$ may contain special colors from the set $\{o_1, \ldots, o_{\mathsf{ar}(\Sigma)}\}$ with $\mathsf{ar}(o_i) = i$, i.e., o_i is a distinguished i-ary color for $1 \leq i \leq \mathsf{ar}(\Sigma)$. Then, the set of *decorators* $\mathrm{Dec}(\mathbb{L}, \mathrm{Cnst}, \Sigma)$ is $\{\mathtt{New}_*\} \cup \{\mathtt{New}_\mathsf{c} \mid \mathsf{c} \in \mathrm{Cnst}\} \cup \mathrm{Instr}(\mathbb{L}, \Sigma)$.

A $(\mathbb{L}, \mathrm{Cnst}, \Sigma)$-*decorated infinite binary tree* (or *decorated tree* for short) is an adorned tree where the unary predicates are from the set $\mathrm{Dec}(\mathbb{L}, \mathrm{Cnst}, \Sigma)$. A decorated tree $\mathcal{T}$ is called a *well-decorated tree* if

- for every null $s \in \{0,1\}^*$, if $\mathcal{T}$ contains a fact of the form $\mathtt{New}_\mathsf{c}(s)$ or $\mathtt{New}_*(s)$ (in which case s will be called a *pseudoleaf*), then $\mathcal{T}$ contains no other fact of the form $\mathtt{Dec}(s)$ with $\mathtt{Dec} \in \mathrm{Dec}(\mathbb{L}, \mathrm{Cnst}, \Sigma)$ (that is, every pseudoleaf carries exactly one "creation command");
- for any pseudoleaf s, any $s' \in \{0,1\}^+$, and any any $\mathtt{Dec} \in \mathrm{Dec}(\mathbb{L}, \mathrm{Cnst}, \Sigma)$, we have $\mathtt{Dec}(ss') \notin \mathcal{T}$ (that is, descendants of pseudoleafs are undecorated);
- for every $\mathsf{c} \in \mathrm{Const}$, $\mathcal{T}$ contains at most one fact of the form $\mathtt{New}_\mathsf{c}(s)$ (that is, constant-creation commands are globally unique throughout $\mathcal{T}$);
- for every null $s \in \{0,1\}^*$, if $\mathtt{Dec}(s), \mathtt{Dec}'(s) \in \mathcal{T}$ with $\mathtt{Dec}, \mathtt{Dec}' \in \mathrm{Instr}(\mathbb{L}, \Sigma)$ then $\mathtt{Dec}$ and $\mathtt{Dec}'$ are compatible (that is, no node is decorated by incompatible instructions). ◇

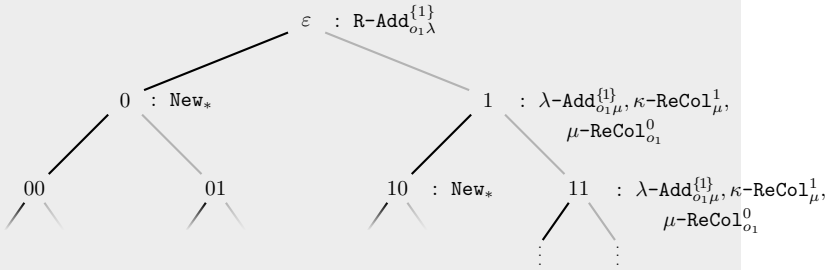
Example 6.8. Below we display (an initial part of) the well-decorated tree associated with Example 6.1. Note that it is an infinite structure but the nodes below the faded-out edges do not carry any decorators.



Example 6.9. The well-decorated tree associated with Example 6.2 is the instance

$$\mathcal{T}_{\mathrm{bin}} \cup \{\mathtt{R\text{-}Add}^{\{1\}}_{o_1\lambda}(\varepsilon)\} \cup \{\lambda\text{-}\mathtt{Add}^{\{1\}}_{o_1\mu}(1^n), \kappa\text{-}\mathtt{ReCol}^1_\mu(1^n), \mu\text{-}\mathtt{ReCol}^0_{o_1}(1^n), \mathtt{New}_*(1^{n-1}0) \mid n \in \mathbb{N}^+\}$$

The initial part of the tree is depicted below. The subtrees rooted in 1 and 11 are isomorphic.



After having precisely characterized the type of adorned tree-instances that serve as "instance construction blueprints", what remains to be done is to formally define the connection between the "blueprint instances" (that is, well-decorated trees) and the instances they represent. This is the purpose of the following definition. It states that, on an intuitive level, when presented with a well-decorated tree $\mathcal{T}$, each tree node s will be associated with a colored instance whose active domain, denoted $\mathsf{ent}^{\mathcal{T}}(s)$, are s's pseudoleaf descendants (renamed to constants, if applicable). The function $\mathsf{col}^{\mathcal{T}}_s$ gives a full account of the s-instance's coloring by associating every color of some arity k with the set of k-tuples from $\mathsf{ent}^{\mathcal{T}}(s)$ that carry that color. Finally, $\mathcal{I}^{\mathcal{T}}_s$ provides all information regarding the s-instance's atoms. For pseudoleafs, their associated colored instance is given explicitly, for the other nodes s, the associated colored instance is obtained as a combination of the instances associated to s's children nodes $s0$ and $s1$. The specifics of this combination are governed by the instruction-decorators attached to s, in line with the intuitions provided in the previous section.[8] This way, the well-decorated tree $\mathcal{T}$ can indeed be understood as a "bottom-up assembly workflow", where every node comes with a colored instance representing an intermediate state of the assembly process, which will be combined with other colored instances further up in the tree. The final result thereof is the instance $\mathcal{I}^{\mathcal{T}}_\varepsilon$ that is attached to $\mathcal{T}$'s root node ε.

Definition 6.10. Let $\mathcal{T}$ be a $(\mathbb{L}, \mathrm{Cnst}, \Sigma)$-well-decorated tree. We define the function $\mathsf{ent}^{\mathcal{T}} \colon \{0,1\}^* \to 2^{\mathrm{Cnst} \cup \{0,1\}^*}$ mapping each null $s \in \{0,1\}^*$ to its *entities* (a set of nulls and constants) as follows:

$$\mathsf{ent}^{\mathcal{T}}(s) = \{ss' \mid \mathtt{New}_*(ss') \in \mathcal{T}\} \cup \{\mathsf{c} \mid \mathtt{New}_{\mathsf{c}}(ss') \in \mathcal{T}, \mathsf{c} \in \mathrm{Cnst}\}.$$

Every tree node s of $\mathcal{T}$ is assigned a *color* map

$$\mathsf{col}^{\mathcal{T}}_s \colon \mathbb{L} \to 2^{(\mathsf{ent}^{\mathcal{T}}(s))^*},$$

indicating which tuples of s's entities are accordingly colored: The color maps are defined such that $\mathsf{col}^{\mathcal{T}}_s(\lambda)$ equals

- $\{(\mathsf{c}, \ldots, \mathsf{c})\}$ iff $\mathtt{New}_{\mathsf{c}}(s) \in \mathcal{T}$, $\mathsf{c} \in \mathrm{Cnst}$, and $\lambda = o_{|(\mathsf{c},\ldots,\mathsf{c})|}$,
- $\{(s, \ldots, s)\}$ iff $\mathtt{New}_*(s) \in \mathcal{T}$, $s \in \{0,1\}^*$, and $\lambda = o_{|(s,\ldots,s)|}$,
- and in all other cases

$$\bigcup_{\lambda\text{-}\mathtt{Add}^I_{\kappa\mu}(s) \in \mathcal{T}} \mathsf{col}^{\mathcal{T}}_{s0}(\kappa) +\!\!+_I \mathsf{col}^{\mathcal{T}}_{s1}(\mu) \;\cup\; \bigcup_{\lambda\text{-}\mathtt{ReCol}^i_\kappa(s) \in \mathcal{T}} \mathsf{col}^{\mathcal{T}}_{si}(\kappa) \;\cup\; \bigcup_{i \in \{0,1\}} \begin{cases} \emptyset & \text{if } \kappa\text{-}\mathtt{ReCol}^i_\lambda(s) \in \mathcal{T} \text{ for any } \kappa \in \mathbb{L} \\ \mathsf{col}^{\mathcal{T}}_{si}(\lambda) & \text{otherwise.} \end{cases}$$

Every node s is assigned a set of Σ-atoms over its entities as indicated by the sets Atoms_s. These sets are defined such that Atoms_s equals:

[8] By design, descendants of pseudoleafs are irrelevant for the described assembly process and can be entirely disregarded as spurious nodes. Technically, $\mathsf{ent}^{\mathcal{T}}$ assigns them an empty active domain and therefore, they are associated with empty colored instances.

- $\{\top(\mathsf{c})\}$ if $\mathtt{New}_{\mathsf{c}}(s) \in \mathcal{T}$ and $\mathsf{c} \in \mathrm{Const}$,
- $\{\top(s)\}$ if $\mathtt{New}_{*}(s) \in \mathcal{T}$,
- and otherwise

$$\bigcup_{\mathtt{R\text{-}Add}^{I}_{\kappa\mu}(s)\in\mathcal{T}} \big\{\mathtt{R}(\boldsymbol{e}) \mid \boldsymbol{e} \in \mathsf{col}^{\mathcal{T}}_{s0}(\kappa) +\!\!+_I \mathsf{col}^{\mathcal{T}}_{s1}(\mu)\big\} \;\cup\; \bigcup_{\mathtt{R\text{-}Add}^{i}_{\kappa}(s)\in\mathcal{T}} \big\{\mathtt{R}(\boldsymbol{e}) \mid \boldsymbol{e} \in \mathsf{col}^{\mathcal{T}}_{si}(\kappa)\big\}.$$

Let $\mathcal{I}^{\mathcal{T}}_s = \bigcup_{s'\in\{0,1\}^*} \mathrm{Atoms}_{ss'}$ denote the instance associated to the node s of $\mathcal{T}$. Finally we define the *instance* $\mathcal{I}^{\mathcal{T}}$ *represented by* $\mathcal{T}$ as $\mathcal{I}^{\mathcal{T}}_{\varepsilon}$ where ε is the root of $\mathcal{T}$. ◇

Remark 6.11. The well-versed reader might sense a certain oddity about the above definition. While $\mathsf{ent}^{\mathcal{T}}(s)$ is clearly specified for every s, the coloring $\mathsf{col}^{\mathcal{T}}_s$ and atom set $\mathcal{I}^{\mathcal{T}}_s$ associated to s are "defined" in terms of the colorings and atomsets of s's child nodes $s0$ and $s1$, which raises justified concerns about the well-foundedness of their definition, in view of the infinity of $\mathcal{T}$. To resolve the oddity, we will now briefly argue why the definition is not problematic. We observe that both $\mathsf{col}^{\mathcal{T}}_s$ and $\mathcal{I}^{\mathcal{T}}_s$ are fully determined by providing for every tuple $\mathbf{t}$ over $\mathsf{ent}^{\mathcal{T}}(s)$ the information which colors and predicates that tuple carries. Now for any such tuple $\mathbf{t}$, each of its entries t corresponds to one specific pseudoleaf of $\mathcal{T}$, where t is being "introduced". By the way the assignments of colorings and atom sets are specified, the colors and predicates carried by $\mathbf{t}$ are fully determined by any finite initial segment (aka prefix) of $\mathcal{T}$ that contains all the pseudoleafs introducing the entries of $\mathbf{t}$. Therefore, for every tuple $\mathbf{t}$ and node s, the colors and predicates that $\mathbf{t}$ carries in the colored s-instance are uniquely determined. Consequently, $\mathsf{col}^{\mathcal{T}}_s$ and $\mathcal{I}^{\mathcal{T}}_s$ as a whole are uniquely determined for every s. Summing up: while the tree $\mathcal{T}$ we operate on is infinite as a whole, any tuple-specific coloring and predicate assignment is uniquely determined through a (bottom-up) inductive definition on some finite prefix of $\mathcal{T}$.

Consequently, an equivalent alternative definition of $\mathcal{I}^{\mathcal{T}}$ is obtained by looking at the infinite sequence $\mathcal{T}_0,\ \mathcal{T}_1,\ \mathcal{T}_2, \ldots$ of finite binary decorated trees obtained from $\mathcal{T}$ by truncating it at depth $0,\ 1,\ 2, \ldots$. One can then use a very minor adaptation of Definition 6.10 to define the corresponding instances $\mathcal{I}^{\mathcal{T}_0},\ \mathcal{I}^{\mathcal{T}_1},\ \mathcal{I}^{\mathcal{T}_2}, \ldots$ in a well-founded bottom-up manner. Finally, one lets $\mathcal{I}^{\mathcal{T}} = \bigcup_{i\in\mathcal{N}} \mathcal{I}^{\mathcal{T}_i}$. In fact, this is the strategy Courcelle uses to define clique-width for countable graphs [Cou04].

Example 6.12. Let $\mathcal{T}$ be as given in Example 6.8. We then find the following values for the node-wise entities, colorings, and atom sets.

- $\mathsf{ent}^{\mathcal{T}} = \big\{\varepsilon \mapsto \{\mathsf{a},\mathsf{b},\mathsf{c},11\}, 0 \mapsto \{\mathsf{a},\mathsf{b}\}, 1 \mapsto \{\mathsf{c},11\}, 00 \mapsto \{\mathsf{a}\}, 01 \mapsto \{\mathsf{b}\}, 10 \mapsto \{\mathsf{c}\}, 11 \mapsto \{11\},$ $s \mapsto \emptyset$ whenever $|s| > 2\big\}$
- $\mathsf{col}^{\mathcal{T}}_{\varepsilon} = \big\{o_1 \mapsto \{\mathsf{a},\mathsf{b}\},\ \lambda \mapsto \{\mathsf{c},11\},\ o_2 \mapsto \{(\mathsf{a},\mathsf{a}),(\mathsf{b},\mathsf{b}),(\mathsf{c},\mathsf{c}),(11,11)\}\big\}$
- $\mathsf{col}^{\mathcal{T}}_{0} = \big\{o_1 \mapsto \{\mathsf{a},\mathsf{b}\},\ \lambda \mapsto \emptyset,\ o_2 \mapsto \{(\mathsf{a},\mathsf{a}),(\mathsf{b},\mathsf{b})\}\big\}$
- $\mathsf{col}^{\mathcal{T}}_{1} = \big\{o_1 \mapsto \emptyset,\ \lambda \mapsto \{\mathsf{c},11\},\ o_2 \mapsto \{(\mathsf{c},\mathsf{c}),(11,11)\}\big\}$
- $\mathsf{col}^{\mathcal{T}}_{00} = \big\{o_1 \mapsto \{\mathsf{a}\},\ \lambda \mapsto \emptyset,\ o_2 \mapsto \{(\mathsf{a},\mathsf{a})\}\big\}$
- $\mathsf{col}^{\mathcal{T}}_{01} = \big\{o_1 \mapsto \{\mathsf{b}\},\ \lambda \mapsto \emptyset,\ o_2 \mapsto \{(\mathsf{b},\mathsf{b})\}\big\}$
- $\mathsf{col}^{\mathcal{T}}_{10} = \big\{o_1 \mapsto \{\mathsf{c}\},\ \lambda \mapsto \emptyset,\ o_2 \mapsto \{(\mathsf{c},\mathsf{c})\}\big\}$
- $\mathsf{col}^{\mathcal{T}}_{11} = \big\{o_1 \mapsto \{11\},\ \lambda \mapsto \emptyset,\ o_2 \mapsto \{(11,11)\}\big\}$
- $\mathsf{col}^{\mathcal{T}}_{s} = \big\{o_1 \mapsto \emptyset,\ \lambda \mapsto \emptyset,\ o_2 \mapsto \emptyset\big\}$ for all s with $|s| > 2$,

- $\mathcal{I}_s^{\mathcal{T}} = \mathrm{Atoms}_s = \emptyset$ for all $s \neq \varepsilon$,
- $\mathcal{I}_0^{\mathcal{T}} = \mathrm{Atoms}_0 = \{\mathtt{E}(\mathtt{a},\mathtt{a}), \mathtt{E}(\mathtt{b},\mathtt{b}), \mathtt{E}(\mathtt{a},\mathtt{b}), \mathtt{E}(\mathtt{b},\mathtt{a})\}$,
- $\mathrm{Atoms}_\varepsilon = \{\mathtt{E}(\mathtt{a},\mathtt{c}), \mathtt{E}(\mathtt{a},11), \mathtt{E}(\mathtt{b},\mathtt{c}), \mathtt{E}(\mathtt{b},11)\}$,
- $\mathcal{I}^{\mathcal{T}} = \mathcal{I}_\varepsilon^{\mathcal{T}} = \{\mathtt{E}(\mathtt{a},\mathtt{a}), \mathtt{E}(\mathtt{b},\mathtt{b}), \mathtt{E}(\mathtt{a},\mathtt{b}), \mathtt{E}(\mathtt{b},\mathtt{a}), \mathtt{E}(\mathtt{a},\mathtt{c}), \mathtt{E}(\mathtt{a},11), \mathtt{E}(\mathtt{b},\mathtt{c}), \mathtt{E}(\mathtt{b},11)\}$

It is now easy to check that these values are in line with Definition 6.10. The isomorphism $\{\mathtt{a} \mapsto \mathtt{a}, \mathtt{b} \mapsto \mathtt{b}, \mathtt{c} \mapsto \mathtt{c}, 11 \mapsto x\}$ witnesses that $\mathcal{I}^{\mathcal{T}}$ is indeed isomorphic to $\mathcal{I}_{\mathrm{ex}}$.

Example 6.13. Let $\mathcal{T}$ be as given in Example 6.9. We then find the following values for the node-wise entities, colorings, and atom sets (letting i range over $0, 1, 2, \ldots$, while s ranges over $\{0,1\}^+$, and s' over $\{0,1\}^*$.

- $\mathsf{ent}^{\mathcal{T}}(\varepsilon) = \{0, 10, 110, 1110, \ldots\} = \{1^j0 \mid 0 \leq j\}$,
- $\mathsf{ent}^{\mathcal{T}}(s) = \{ss' \mid s' \in \{0,1\}^*\} \cap \mathsf{ent}^{\mathcal{T}}(\varepsilon)$,
- $\mathsf{col}_\varepsilon^{\mathcal{T}} = \{o_1 \mapsto \{0\}, \ \mu \mapsto \{10\}, \ \kappa \mapsto \{1^j0 \mid j \geq 2\}, \ \lambda \mapsto \{(1^j0, 1^{j+1}0) \mid j \geq 1\}\}$,
- $\mathsf{col}_{1^{i+1}}^{\mathcal{T}} = \{o_1 \mapsto \emptyset, \ \mu \mapsto \{1^{i+1}0\}, \ \kappa \mapsto \{1^j0 \mid j > i{+}1\}, \ \lambda \mapsto \{(1^j0, 1^{j+1}0) \mid j > i\}\}$,
- $\mathsf{col}_{1^i0}^{\mathcal{T}} = \{o_1 \mapsto \{1^i0\}, \mu \mapsto \emptyset, \ \kappa \mapsto \emptyset, \ \lambda \mapsto \emptyset\}$,
- $\mathsf{col}_{s'0s}^{\mathcal{T}} = \{o_1 \mapsto \emptyset, \ \mu \mapsto \emptyset, \ \kappa \mapsto \emptyset, \ \lambda \mapsto \emptyset\}$,
- $\mathcal{I}^{\mathcal{T}} = \mathcal{I}_\varepsilon^{\mathcal{T}} = \mathrm{Atoms}_\varepsilon = \{\mathtt{R}(0, 1^i0, 1^{i+1}0 \mid i \geq 1)\}$ and $\mathcal{I}_s^{\mathcal{T}} = \mathrm{Atoms}_s = \emptyset$ for all $s \neq \varepsilon$.

It can now be readily checked that these values are in line with Definition 6.10. Also, the isomorphism $\mathsf{iso} : \{0, 10, 110, 1110, \ldots\} \to \{-1, 0, 1, 2, \ldots\}$ with $1^i0 \mapsto i{-}1$ for all $i \geq 0$ witnesses that $\mathcal{I}^{\mathcal{T}}$ is indeed isomorphic to $\mathcal{I}_{\mathrm{tern}}$.

Having introduced the machinery for representing countable instances by means of well-decorated trees (which are themselves instances of a specific convenient shape), we are finally ready to formally define the notion of partitionwidth.

Definition 6.14. Given an instance $\mathcal{I}$ over a finite set Cnst of constants and a countable set of nulls as well as a finite signature Σ, the *partitionwidth* of $\mathcal{I}$, written $\mathsf{pw}(\mathcal{I})$, is defined to be the smallest natural number n such that $\mathcal{I}$ is isomorphic to an instance represented by some $(\mathbb{L}, \mathrm{Cnst}, \Sigma)$-well-decorated tree such that $|\mathbb{L}| = n$. If no $(\mathbb{L}, \mathrm{Cnst}, \Sigma)$-well-decorated tree with such a number exists, we let $\mathsf{pw}(\mathcal{I}) = \infty$. ◇

6.2. **Partitionwidth and Monadic Second-Order Logic.** Next, we recall known features of partitionwidth related to MSO and utilize them to show several properties which are useful for our purposes. A crucial role in this regard is played by the well-known notion of *MSO-interpretations* (also referred to as MSO-transductions or MSO-definable functions in the literature [ALS88, Cou91, Eng90, Cou94, CE12]).

Definition 6.15 (MSO-interpretations)**.** Let Σ and Σ' be signatures, and Cnst and Cnst' be finite sets of constants. An *MSO-interpretation* is a sequence $\mathfrak{I}$ of MSO-formulae over (Σ, Cnst), containing for every $\mathtt{R} \in \Sigma' \cup \{\top\}$ one formula $\varphi_{\mathtt{R}}$ with free variables $x_1, \ldots, x_{\mathsf{ar}(\mathtt{R})}$; for every $\mathtt{c} \in \mathrm{Cnst}'$ a formula $\varphi_{\mathtt{c}}$ with free variable x_1; and one formula $\varphi_=$ with free variables x_1, x_2. Given an instance $\mathcal{B}$ over (Σ, Cnst), let $D = \{t \in \mathsf{dom}(\mathcal{B}) \mid \mathcal{B} \models \varphi_\top(t)\}$ and let $\mathsf{f} : D \to \mathbf{N} \cup \mathrm{Cnst}'$ be an arbitrary but fixed

function satisfying $\mathsf{f}(t) = \mathsf{f}(t')$ *iff* $\mathcal{B} \models \varphi_{=}(t, t')$, and also $\mathsf{f}(t) = \mathsf{c}$ *iff* $\mathcal{B} \models \varphi_{\mathsf{c}}(t)$. If no such a function exists, $\mathfrak{I}(\mathcal{B})$ is undefined, otherwise let

$$\mathfrak{I}(\mathcal{B}) = \bigcup_{\mathtt{R} \in \Sigma' \cup \{\top\}} \{\mathtt{R}(\mathsf{f}(\boldsymbol{t})) \mid \mathcal{B} \models \varphi_{\mathtt{R}}(\boldsymbol{t}),\ \boldsymbol{t} \in D^{\mathsf{ar}(\mathtt{R})}\}.$$

An instance $\mathcal{A}$ is *MSO-interpretable* in an instance $\mathcal{B}$ (written $\mathcal{A} \leq_{\mathrm{MSO}} \mathcal{B}$) if there exists an MSO-interpretation $\mathfrak{I}$ such that $\mathcal{A} \cong \mathfrak{I}(\mathcal{B})$. It is *(MSO-)tree-interpretable* if $\mathcal{A} \leq_{\mathrm{MSO}} \mathcal{T}$ for some adorned tree $\mathcal{T}$. ◇

According to standard results about MSO-interpretations, for every interpretation $\mathfrak{I}$ from (Σ, Cnst) to $(\Sigma', \mathrm{Cnst}')$ and every MSO formula Ξ over $(\Sigma', \mathrm{Cnst}')$ one can effectively construct a formula $\Xi^{\mathfrak{I}}$ over (Σ, Cnst) such that for any two instances $\mathcal{A}, \mathcal{B}$ satisfying $\mathcal{A} \cong \mathfrak{I}(\mathcal{B})$, the following holds:

$$\mathcal{A} \models \Xi \iff \mathcal{B} \models \Xi^{\mathfrak{I}}.$$

Moreover, as MSO-interpretations can be composed, the MSO-interpretability relation ($\leq_{\mathrm{MSO}}$) is transitive: whenever $\mathcal{A} \leq_{\mathrm{MSO}} \mathcal{B}$ and $\mathcal{B} \leq_{\mathrm{MSO}} \mathcal{C}$ hold, then also $\mathcal{A} \leq_{\mathrm{MSO}} \mathcal{C}$. Blumensath showed that, for every set of decorators, the corresponding mapping $\mathcal{T} \mapsto \mathcal{I}^{\mathcal{T}}$ described in Definition 6.10 can be realized by an appropriate MSO-interpretation [Blu06, Prop. 17].

Lemma 6.16. *For every set of decorators* $\mathrm{Dec}(\mathbb{L}, \mathrm{Cnst}, \Sigma)$*, one can compute an MSO-interpretation* $\mathfrak{I}$ *such that for every* $(\mathbb{L}, \mathrm{Cnst}, \Sigma)$*-well-decorated tree* $\mathcal{T}$*, we get* $\mathcal{I}^{\mathcal{T}} \cong \mathfrak{I}(\mathcal{T})$*.*

In view of this result, establishing MSO-friendliness is then rather straightforward.

Theorem 6.17. *Partitionwidth is an MSO-friendly width measure.*

Proof. Let n be a fixed natural number. Take a MSO sentence Ξ over some signature Σ and a set of constants Cnst. We will show how to decide if there exists a model $\mathcal{I}$ of Ξ such that $\mathsf{pw}(\mathcal{I}) \leq n$. This implies the claimed result as we can enumerate all MSO sentences and check model existence.

Up to renaming of colors, there are only finitely many distinct color sets $\mathbb{L}$ with $|\mathbb{L}| \leq n$, where each color is assigned an arity between 1 and $\mathsf{ar}(\Sigma)$. By definition, Ξ has a model of partitionwidth $\leq n$ *iff* for one of these $\mathbb{L}$ there exists a $(\mathbb{L}, \mathrm{Cnst}, \Sigma)$-well-decorated tree $\mathcal{T}$ satisfying $\mathcal{I}^{\mathcal{T}} \models \Xi$. With the corresponding MSO-interpretation $\mathfrak{I}$ from Lemma 6.16, this holds *iff* $\mathcal{T} \models \Xi^{\mathfrak{I}}$. It is easy to see that there exists a MSO-sentence Θ which is satisfied by a given decorated tree *iff* it is well-decorated (cf. Definition 6.7). Consequently, Ξ is satisfiable in a model of partitionwidth $\leq n$ *iff* $\Xi^{\mathfrak{I}} \wedge \Theta$ is satisfiable in any $(\mathbb{L}, \mathrm{Cnst}, \Sigma)$-decorated tree (for an appropriate $\mathbb{L}$). Let $\Xi' = \exists \mathrm{Dec}(\mathbb{L}, \mathrm{Cnst}, \Sigma).(\Xi^{\mathfrak{I}} \wedge \Theta)$, i.e. it is the sentence obtained by reinterpreting all unary predicates of $\Xi^{\mathfrak{I}} \wedge \Theta$ as MSO set variables and quantifying over them existentially. Clearly, Ξ' is a MSO sentence over the signature $\{\mathtt{Succ}_0, \mathtt{Succ}_1\}$ which is valid in $\mathcal{T}_{\mathrm{bin}}$ *iff* some well-decoration of $\mathcal{T}_{\mathrm{bin}}$ makes $\Xi^{\mathfrak{I}}$ true. Thus, we have reduced our problem to checking the validity of finitely many MSO sentences (one for each $\mathbb{L}$) in $\mathcal{T}_{\mathrm{bin}}$, which is decidable by Rabin's Tree Theorem [Rab69, Thm 1.1]. □

By Lemma 6.16, any pw-finite $\mathcal{I}$ is tree-interpretable. Remarkably, Blumensath showed that the converse also holds [Blu06, Thm. 41].

Theorem 6.18. *For every instance* $\mathcal{I}$ *the following holds:* $\mathcal{I}$ *is* pw*-finite iff there is some* $\mathcal{T} \in \mathbb{T}$ *such that* $\mathcal{I} \leq_{MSO} \mathcal{T}$*.*

This coincidence of tree-interpretability and finiteness of partitionwidth shows that the latter is uniquely positioned among the width measures. We will exploit

it in various ways; most importantly, we can use it to show that finiteness of partitionwidth is preserved under MSO-interpretations.

Theorem 6.19. *If $\mathcal{A} \leq_{MSO} \mathcal{B}$ and $\mathcal{B}$ is* pw*-finite, then $\mathcal{A}$ is* pw*-finite.*

Proof. Given a pw-finite $\mathcal{B}$, Theorem 6.18 implies $\mathcal{B} \leq_{\text{MSO}} \mathcal{T}$ for some $\mathcal{T} \in \mathbb{T}$. Then, $\mathcal{A} \leq_{\text{MSO}} \mathcal{B}$ and transitivity of $\leq_{\text{MSO}}$ yield $\mathcal{A} \leq_{\text{MSO}} \mathcal{T}$ and thus, again by Theorem 6.18, it follows that $\mathcal{A}$ is pw-finite. □

6.3. **Robustness under Controlled Modifications.** We will now turn our attention to certain modifications of instances that preserve width finiteness properties. They all have in common that the instance's domain stays the same. We start with an easy, yet useful observation (the proof of which can be found in the appendix).

Definition 6.20. Let Σ be a signature, $\mathtt{U} \in \Sigma$ a unary predicate, $\mathcal{I}$ an instance over Σ, and $T \subseteq \mathsf{dom}(\mathcal{I})$ a set of domain elements. We define the *unary replacement of* $\mathcal{I}$ w.r.t. $\mathtt{U}$ and T as the instance $\mathcal{I}^{\mathtt{U}:=T} = \mathcal{I} \setminus \{\mathtt{U}(t) \in \mathcal{I}\} \cup \{\mathtt{U}(t) \mid t \in T\}$. ◇

Observation 6.21. *Finiteness of treewidth, cliquewidth, and partitionwidth are all preserved under unary replacements.*

Finite partitionwidth exhibits further convenient properties, which will turn out to be quite useful later on: it is preserved when extending an instance by defined relations, where definitions can be given through first-order formulas or regular path expressions. This is a rather straightforward consequence from the fact that all such definitorial extensions are particular cases of MSO-interpretations (see the appendix for the detailed argument).

Definition 6.22. Let Σ be a signature, and $\mathcal{I}$ be an instance over Σ. We define *two-way regular path expressions* (2RPEs) over Σ to be regular expressions over $\Sigma^{\leftrightarrow}_{\text{bin}}$, the set containing the symbols $\mathtt{P}^{\rightarrow}$ and $\mathtt{P}^{\leftarrow}$ for each binary $\mathtt{P} \in \Sigma$. We associate with any such 2RPE E the binary relation $E^{\mathcal{I}}$ over $\mathsf{dom}(\mathcal{I})$:

$$
\begin{aligned}
(\mathtt{R}^{\rightarrow})^{\mathcal{I}} &= \{(t_1, t_2) \mid \mathtt{R}(t_1, t_2) \in \mathcal{I}\} \\
(\mathtt{R}^{\leftarrow})^{\mathcal{I}} &= \{(t_2, t_1) \mid \mathtt{R}(t_1, t_2) \in \mathcal{I}\} \\
(E_1 E_2)^{\mathcal{I}} &= \{(t_1, t_3) \mid (t_1, t_2) \in (E_1)^{\mathcal{I}}, (t_2, t_3) \in (E_2)^{\mathcal{I}}\} \\
(E_1 \cup E_2)^{\mathcal{I}} &= (E_1)^{\mathcal{I}} \cup (E_2)^{\mathcal{I}} \\
(E^*)^{\mathcal{I}} &= \{(t, t) \mid t \in \mathsf{dom}(\mathcal{I})\} \cup (E)^{\mathcal{I}} \cup (EE)^{\mathcal{I}} \cup \dots
\end{aligned}
$$

Given a binary predicate $\mathtt{P} \in \Sigma$, we define the *2RPE-extension of* $\mathcal{I}$ w.r.t. $\mathtt{P}$ and E as the instance $\mathcal{I}^{\mathtt{P}:=\mathtt{P} \cup E} = \mathcal{I} \cup \{\mathtt{P}(t, t') \mid (t, t') \in E^{\mathcal{I}}\}$. Moreover, given a $|\boldsymbol{x}|$-ary predicate $\mathtt{R} \in \Sigma$ and a first-order formula $\varphi(\boldsymbol{x})$ with free variables $\boldsymbol{x}$, we define the *FO-extension of* $\mathcal{I}$ w.r.t. $\mathtt{R}$ and φ as the instance $\mathcal{I}^{\mathtt{R}:=\mathtt{R} \cup \varphi} = \mathcal{I} \cup \{\mathtt{R}(\boldsymbol{t}) \mid \mathcal{I} \models \varphi(\boldsymbol{t})\}$. ◇

Theorem 6.23. *If $\mathcal{I}$ has finite partitionwidth, then so does any 2RPE-extension and any FO-extension of $\mathcal{I}$.*

6.4. **Relationship to Other Width Notions.** In this section we are going to clarify how the notion of finite partitionwidth relates to those of finite treewidth and finite cliquewidth. Figure 1 summarizes these insights.

Without providing full formal details, we recall that the notion of *cliquewidth* in its version for countable instances [FLOR23] follows a very similar idea as partitionwidth, but only allows for unary colors and uses a different set of assembly operations: nullary operations c_κ and $*_\kappa$, creating singleton instances of κ-colored constants or nulls, respectively; a binary operator $\oplus$ creating the (plain) disjoint union; unary operators $\mathsf{Recolor}_{\kappa \to \mu}$ assigning all κ-colored terms the new color μ; and another type of unary operators $\mathsf{Add}_{\mathtt{R}, \kappa_1, \dots, \kappa_n}$, creating atoms $\mathtt{R}(t_1, \dots, t_n)$ whenever

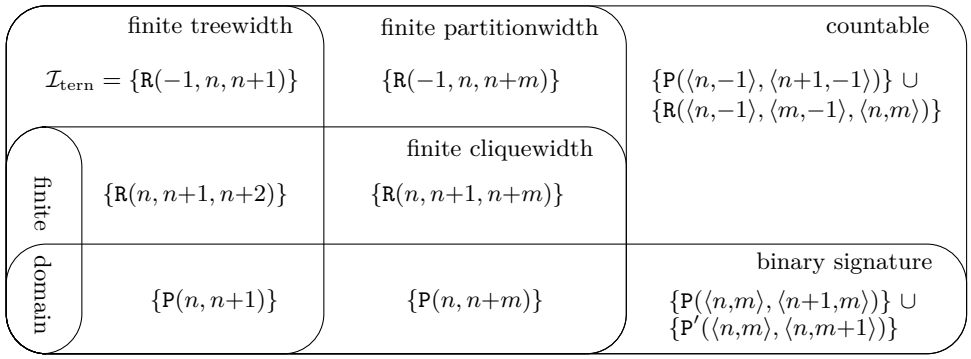


FIGURE 1. Set diagram displaying the inclusions between classes of countable instances satisfying diverse types of width-finiteness, conditioned on signature arity. Example instances witness the strictness of the inclusions (n and m range over $\mathbb{N}$).

each term t_i carries the corresponding color κ_i for all i. As an example, the instance from Example 6.2 can be created through the expression

$$\mathsf{Add}_{\mathtt{E},\kappa,\mu}\big(\mathsf{Add}_{\mathtt{E},\kappa,\kappa}(\mathsf{a}_\kappa \oplus \mathsf{b}_\kappa) \oplus \mathsf{Recolor}_{\kappa\to\mu}(\mathsf{c}_\kappa \oplus *_\kappa)\big).$$

Earlier investigations showed that, while finite treewidth implies finite cliquewidth for binary signatures, the two notions are incomparable for signatures of higher arity [FLOR23]: notably, the example $\mathcal{I}_{\text{tern}}$ from Example 6.2 exhibits a treewidth of 2 but infinite cliquewidth. As demonstrated by Example 6.2, however, $\mathcal{I}_{\text{tern}}$ does have finite partitionwidth. In fact, the class of instances with finite partitionwidth subsumes both the class of treewidth-finite and the class of cliquewidth-finite instances, independent of the signature's arity. With the help of Theorem 6.18, this result can be derived from the fact that these classes consist of tree-interpretable instances (the detailed argument can be found in the appendix).

Theorem 6.24. *Let $\mathcal{I}$ be a countable instance over a finite signature of arbitrary arity. Then the following hold:*

- *If $\mathcal{I}$ has finite treewidth, then $\mathcal{I}$ has finite partitionwidth.*
- *If $\mathcal{I}$ has finite cliquewidth, then $\mathcal{I}$ has finite partitionwidth.*

Both inclusions of Theorem 6.24 are strict; there even are instances of finite partitionwidth whose treewidth and cliquewidth are both infinite. The instance

$$\mathcal{I}_{\text{tern}*} = \{\mathtt{R}(-1, n, n{+}m) \mid n, m \in \mathbb{N}\}$$

has infinite treewidth since any two natural numbers co-occur in some atom, enforcing that any tree-decomposition must contain some node X with $\mathbb{N} \subseteq X$. It also has infinite cliquewidth, which can be argued along the same lines as for $\mathcal{I}_{\text{tern}}$ [FLOR23]. Yet, $\mathcal{I}_{\text{tern}*}$ has finite partitionwidth, as it is isomorphically represented by the decorated tree corresponding to the (infinite) assembly expression $* \oplus_{\{\mathtt{R\text{-}Add}^{\{1\}}_{o_1\lambda}\}} E$ where

$$E = \big(* \oplus_{\{\kappa\text{-}\mathtt{ReCol}^0_{o_1}, \lambda\text{-}\mathtt{ReCol}^0_{o_2}, \lambda\text{-}\mathtt{Add}^{\{1\}}_{o_1\kappa}\}} E\big).$$

It should, however be noted that, finite partitionwidth and finite cliquewidth coincide for binary signatures (see the appendix for the proof).

Proposition 6.25. *Any countable instance over a binary signature has finite cliquewidth iff it has finite partitionwidth.*

At the end of this section, let us outline how the just obtained insights regarding partitionwidth can be harnessed to show decidability of very expressive querying in some of the most expressive contemporary description logics.

Example 6.26. The description logics $\mathcal{SRIQ}b_s$, $\mathcal{SROI}b_s$, and $\mathcal{SROQ}b_s$ are extensions of the mentioned $\mathcal{ALCHIQ}b$, $\mathcal{ALCHOI}b$, and $\mathcal{ALCHOQ}b$, respectively. One additional feature of $\mathcal{SR}*$ knowledge bases is an *RBox*, which contains *role-chain axioms* like transitivity statements for binary relations such as $\texttt{Friend} \circ \texttt{Friend} \sqsubseteq \texttt{Friend}$ but also more complex relationships like $\texttt{Friend} \circ \texttt{Enemy} \sqsubseteq \texttt{Enemy}$. Such axioms prevent the existence of treewidth-finite (finitely) universal model sets, but not of partitionwidth-finite ones, as can be shown via known model-theoretic insights: Taking some model, it can be unravelled into a "pre-model" of finite treewidth (and thus partitionwidth), as one would do in the RBox-free case. This structure can then be extended to a model proper, coping with the still violated RBox axioms by applying RPE-extensions [Kaz08, CEO09], which – as we now know – preserve finiteness of partitionwidth. It is easy to see that the model thus obtained maps homomorphically into the original one. Hence, we obtain decidability of HUSOMSOQs for all these description logics. For query languages up to UC2RPQs, this had already been known (even with tight complexity bounds [CEO09, BR19]), but for more expressive query languages, decidability had hitherto been open.

7. Application to Existential Rules

In the remainder of the paper, we show how our decidability framework can be fruitfully applied to the very popular logical formalism of *existential rules* – also referred to as *tuple-generating dependencies* (TGDs) [AHV95], *conceptual graph rules* [SM96], Datalog$^{\pm}$ [Got09], and $\forall\exists$*-rules* [BLMS11] – which has become a de-facto standard of ontological querying over databases. As entailment of even the most simple queries is undecidable for unconstrained existential rules [CLM81], there has been ample research into fragments that restore decidable querying. This includes investigations based on width notions, although existing work almost entirely focuses on (single) universal models of finite treewidth.[9] Thus, the upcoming part of the paper will significantly advance this line of research. Before we introduce relevant notions of existential rules, the reader may want to have a look at the following example.

Example 7.1. We present the ruleset $\mathcal{R}_{\text{bio}}$ about biological relationships describes female/male individuals, related by mother-, father-, parent- and ancestorhood, as well as compatibility. A ternary relation (`CommonDescendant`) indicates two individuals' "earliest" joint descendant, and helps to describe

[9]Two exceptions are our prior work proposing *cliquewidth-finiteness* as alternative to treewidth-finiteness for universal models [FLOR23], and a work investigating *finitely* universal models of finite treewidth [BMR23].

hereditary patterns of some genetic risk. As usual, we omit universal quantifiers and use $\leftrightarrow$ as a shorthand for two rules.

(1) $\texttt{Individual}(x) \to \exists yz.\texttt{Mother}(x,y) \wedge \texttt{Father}(x,z)$

(2) $\texttt{Mother}(x,y) \leftrightarrow \texttt{Parent}(x,y) \wedge \texttt{Female}(y) \qquad \texttt{Father}(x,y) \leftrightarrow \texttt{Parent}(x,y) \wedge \texttt{Male}(y)$

(3) $\texttt{Female}(x) \to \texttt{Individual}(x) \qquad \texttt{Male}(x) \to \texttt{Individual}(x)$

(4) $\texttt{Compatible}(x,y) \leftrightarrow \texttt{Female}(x) \wedge \texttt{Male}(y)$

(5) $\texttt{Parent}(x,y) \to \texttt{Ancestor}(x,y) \qquad \texttt{Ancestor}(x,y) \wedge \texttt{Ancestor}(y,z) \to \texttt{Ancestor}(x,z)$

(6) $\texttt{Mother}(x,y) \wedge \texttt{Ancestor}(y,y') \wedge \texttt{Father}(x,z) \wedge \texttt{Ancestor}(z,z') \to \texttt{CommonDescendant}(x,y',z')$

(7) $\texttt{Risk}(x) \wedge \texttt{Mother}(x,y) \to \texttt{Risk}(y)$

(8) $\texttt{CommonDescendant}(x,y,z) \wedge \texttt{Risk}(y) \wedge \texttt{Risk}(z) \to \texttt{Risk}(x)$

The rest of the section is devoted to introducing the necessary preliminaries on existential rules, the new contributions commence in Section 8.

Existential rules and knowledge bases. An *(existential) rule* is a first-order sentence of the form $\rho = \forall \boldsymbol{x}\boldsymbol{y}.\big(\varphi(\boldsymbol{x},\boldsymbol{y}) \rightarrow \exists \boldsymbol{z}.\psi(\boldsymbol{y},\boldsymbol{z})\big)$, where $\boldsymbol{x}$, $\boldsymbol{y}$, and $\boldsymbol{z}$ are mutually disjoint tuples of variables, and both the *body* $\varphi(\boldsymbol{x},\boldsymbol{y})$ and the *head* $\psi(\boldsymbol{y},\boldsymbol{z})$ of ρ (denoted with $\mathsf{body}(\rho)$ and $\mathsf{head}(\rho)$, respectively) are conjunctions (possibly empty, sometimes construed as sets of their conjuncts) of atoms containing the indicated variables. The *frontier* $fr(\rho)$ of ρ is the set of variables $\boldsymbol{y}$ shared between body and head. For better readability, we often omit the universal quantifiers prefixing existential rules. We say that a rule ρ is *full iff* ρ does not contain an existential quantifier. We call a finite set of existential rules $\mathcal{R}$ a *ruleset*. Satisfaction of a rule ρ (a ruleset $\mathcal{R}$) by an instance $\mathcal{I}$ according to standard FO semantics is written $\mathcal{I} \models \rho$ ($\mathcal{I} \models \mathcal{R}$, respectively). Given a database $\mathcal{D}$ and a ruleset $\mathcal{R}$, we define the pair $(\mathcal{D},\mathcal{R})$ to be a *knowledge base*. Then, an instance $\mathcal{I}$ is a *model* of the knowledge base (i.e., of $\mathcal{D}$ and $\mathcal{R}$), written $\mathcal{I} \models (\mathcal{D},\mathcal{R})$, *iff* $\mathcal{D} \subseteq \mathcal{I}$ and $\mathcal{I} \models \mathcal{R}$.

Trigger, rule application, and Skolem chase. An existential rule $\rho = \varphi(\boldsymbol{x},\boldsymbol{y}) \rightarrow \exists \boldsymbol{z}.\psi(\boldsymbol{y},\boldsymbol{z})$ is *applicable* to an instance $\mathcal{I}$ *iff* there is a homomorphism h mapping $\varphi(\boldsymbol{x},\boldsymbol{y})$ to $\mathcal{I}$. We then call (ρ,h) a *trigger* of $\mathcal{I}$. The *application* of a trigger (ρ,h) in $\mathcal{I}$ yields the instance $\mathbf{Ch}(\mathcal{I},\rho,h) = \mathcal{I} \cup \bar{h}(\psi(\boldsymbol{y},\boldsymbol{z}))$, where $\bar{h}$ is the extension of h that maps every variable z from $\boldsymbol{z}$ to a null denoted $z_{\rho,h(\boldsymbol{y})}$. Note that $\bar{h}(\psi(\boldsymbol{y},\boldsymbol{z}))$ is then unique for a given rule and the mapping of its frontier variables, with the effect that applying ρ twice with homomorphisms that agree on the frontier variables does not produce more atoms, that is, $\mathbf{Ch}(\mathbf{Ch}(\mathcal{I},\rho,h),\rho,h') = \mathbf{Ch}(\mathcal{I},\rho,h)$ whenever $h(\boldsymbol{y}) = h'(\boldsymbol{y})$. Moreover, applications of different rules or the twofold application of the same rule with a different frontier-mapping to some instance $\mathcal{I}$ are independent from each other and the order of their application is irrelevant. This allows us to define the parallel one-step application of all applicable rules as

$$\mathbf{Ch}_1(\mathcal{I},\mathcal{R}) = \bigcup_{\rho\in\mathcal{R},\,(\rho,h)\,\text{trigger of}\,\mathcal{I}} \mathbf{Ch}(\mathcal{I},\rho,h).$$

To define the *(breadth-first) Skolem chase sequence* we let $\mathbf{Ch}_0(\mathcal{I},\mathcal{R}) = \mathcal{I}$ as well as $\mathbf{Ch}_{i+1}(\mathcal{I},\mathcal{R}) = \mathbf{Ch}_1(\mathbf{Ch}_i(\mathcal{I},\mathcal{R}),\mathcal{R})$, and obtain the *Skolem chase*

$$\mathbf{Ch}_\infty(\mathcal{I},\mathcal{R}) = \bigcup_{i\in\mathbb{N}} \mathbf{Ch}_i(\mathcal{I},\mathcal{R}),$$

which is a universal model of $(\mathcal{I},\mathcal{R})$ [FKMP05, Mar09]. We note that the Skolem chase of a countable instance is countable, as is the number of overall rule applications performed to obtain it. In accordance with standard FO-semantics, an instance $\mathcal{I}$ and ruleset $\mathcal{R}$ *entail* a BCQ $q = \exists \boldsymbol{x}.\varphi(\boldsymbol{x})$, written $(\mathcal{I},\mathcal{R}) \models q$ *iff* $\varphi(\boldsymbol{x})$ maps homomorphically into each model of $\mathcal{I}$ and $\mathcal{R}$. This coincides with the existence of a homomorphism from $\varphi(\boldsymbol{x})$ into any universal model of $\mathcal{I}$ and $\mathcal{R}$ (e.g., the Skolem chase $\mathbf{Ch}_\infty(\mathcal{I},\mathcal{R})$).

Rewritings and finite-unification sets. Given a ruleset $\mathcal{R}$ and a CQ $q(\boldsymbol{y})$, we say that a UCQ $\psi(\boldsymbol{y})$ is a *rewriting* of $q(\boldsymbol{y})$ under the ruleset $\mathcal{R}$ *iff* for any database $\mathcal{D}$ and any tuple of its elements $\mathbf{a}$ the following holds:

$$\mathbf{Ch}_\infty(\mathcal{D},\mathcal{R}) \models q(\mathbf{a}) \text{ iff } \mathcal{D} \models \psi(\mathbf{a}).$$

A ruleset $\mathcal{R}$ is a *finite-unification set* (**fus**) *iff* for every CQ, there exists a UCQ rewriting [BLMS11], in which case it is also always possible to compute it [KLMT13]. This property is also referred to as *first-order rewritability*. If a ruleset $\mathcal{R}$ is **fus**, then for any given CQ $q(\boldsymbol{y})$, we fix one of its rewritings under $\mathcal{R}$ and denote it with $\mathsf{rew}_\mathcal{R}(q(\boldsymbol{y}))$.

8. Abstracting from Databases: Finite w-Sets

When investigating existential rules, one is commonly interested in semantic properties of rulesets that hold independently of the associated database. This motivates the following definition.

Definition 8.1 (finite-w set)**.** A ruleset $\mathcal{R}$ is called a *finite-*w *set iff* for every database $\mathcal{D}$, there exists a w-finite universal model $\mathcal{I}^*$ of $(\mathcal{D}, \mathcal{R})$. ◇

Various previously defined classes of rulesets with decidable querying can be seen as instances of this framework: *finite-expansion sets* (**fes**) are obtained by using the expansion function returning an instance's domain size as a width measure; *finite-treewidth sets* (**fts**) are obtained when using treewidth;[10] our more recently proposed *finite-cliquewidth sets* (**fcs**) are the outcome of employing cliquewidth [FLOR23]. Due to the established relationships between the width measures, we immediately obtain the following result regarding the novel class of *finite-partitionwidth sets* (**fps**).

Corollary 8.2. **fps** *subsumes all of* **fes**, **fts**, *and* **fcs**.

By virtue of entirely subsuming **fts** (Theorem 6.24), **fps** contains all fragments of the guarded family, ranging from the most basic (guarded rules) to the most elaborate ones (such as glut-frontier-guarded rules) [BLMS11, CGK13, KR11], as well as the various **fes** rule classes based on acyclicity [GHK+13]. By virtue of subsuming **fcs**, **fps** inherits the corresponding decidable subclasses [FLOR23]; in particular, any **fus** ruleset consisting of single-headed rules over a binary signature is **fps**. While for all of these, decidability of querying was previously only established for query formalisms of significantly lower expressivity (though sometimes beyond CQs and with precise complexities [BBMT17]), Corollary 5.5 implies that decidability also holds for more expressive queries like NEMODEQs and MDDlog queries. Yet, the novel **fps** notion also encompasses a great variety of rulesets which aren't contained in any of the previously established classes.

Example 8.3. As an example, consider again the ruleset $\mathcal{R}_{\text{bio}}$ from Example 7.1. $\mathcal{R}_{\text{bio}}$ does not fall under any of the previously known decidable classes: even with the simple database $\mathcal{D}_{\mathtt{a}}$ just containing the fact $\mathtt{Individual}(\mathtt{a})$, any universal model of $(\mathcal{D}_{\mathtt{a}}, \mathcal{R}_{\text{bio}})$ will have an infinite domain (due to indefinite "value invention" through the rules (1)–(4)), infinite treewidth (due to the transitivity of the `Ancestor` relation along infinite `Parent` chains), and infinite cliquewidth (due to the ternary `CommonDescendent` relation, which cannot be created by a unary coloring scheme). $\mathcal{R}_{\text{bio}}$ is also not **fus**: the UCQ rewriting of the CQ $\mathtt{Ancestor}(x, y)$ under $\mathcal{R}_{\text{bio}}$ would have to contain an infinite number of disjuncts: $\exists z.\mathtt{Ancestor}(x, z) \wedge \mathtt{Ancestor}(z, y)$ as well as $\exists z, z'.\mathtt{Ancestor}(x, z) \wedge \mathtt{Ancestor}(z, z') \wedge \mathtt{Ancestor}(z', y)$ and so forth. Yet, the results obtained in the next sections will enable us to show that $\mathcal{R}_{\text{bio}}$ is indeed **fps** and thus allows for decidable expressive querying, as will be discussed in Example 9.5.

It should be noted that, similar to the general case discussed in the first part of the paper, no general complexity bounds exist for any of the width-based existential rules classes: already the special case of plain CQ entailment checking over **fes** rulesets is known to exhibit arbitrarily high data complexities [BCK+21]. Moreover,

[10] In the literature, finite-treewidth sets are also often referred to as "bounded treewidth sets" and denoted **bts**. Beyond being misleading in this case, the usage of the **bts** notion is also ambiguous; it may also refer to structural properties of a specific type of monotonic chase.

it is undecidable to determine if a given ruleset is in **fes**, **fts**, **fcs**, or **fps**, as can be shown via slight modifications of a known argument for **fts** [BLMS11] employing a reduction from CQ entailment for unrestricted existential rules.

9. Extending **fps** through Stratification

For analyzing rulesets toward showing closure properties and decidable CQ entailment, the idea of stratification based on rule dependencies has proved helpful. We use a notion of rule dependency going back to Baget [Bag04] and Baget et al. [BLMS11].

Definition 9.1 (rule dependencies, cuts)**.** Given rules ρ and ρ', we say that ρ' *depends* on ρ, written $\rho \rightsquigarrow \rho'$, *iff* there exists an instance $\mathcal{I}$ such that

- ρ is applicable to $\mathcal{I}$ via a homomorphism h,
- ρ' is applicable to $\mathbf{Ch}(\mathcal{I}, \rho, h)$ via a homomorphism h', and
- ρ' is not applicable to $\mathcal{I}$ via h'.

If a rule ρ' does not depend on a rule ρ, we write $\rho \not\rightsquigarrow \rho'$. We define an *n-cut* to be a partition $\{\mathcal{R}_1, \ldots, \mathcal{R}_n\}$ of a ruleset $\mathcal{R}$ such that for every rule $\rho \in \mathcal{R}_i$ and $\rho' \in \mathcal{R}_j$, $\rho' \not\rightsquigarrow \rho$, for $1 \leq i < j \leq n$. We use the notation $\mathcal{R} = \mathcal{R}_1 \triangleright \cdots \triangleright \mathcal{R}_n$ to denote that $\{\mathcal{R}_1, \ldots, \mathcal{R}_n\}$ is an n-cut of $\mathcal{R}$. For classes of rulesets **class** and **class**$'$ we let **class** $\triangleright$ **class**$'$ denote the class containing all rulesets representable as $\mathcal{R} \triangleright \mathcal{R}'$ where $\mathcal{R}$ is in **class** while $\mathcal{R}'$ is in **class**$'$. $\diamond$

Intuitively, $\rho \rightsquigarrow \rho'$ means that applying ρ to some instance may create a new trigger for ρ' and thus make ρ' applicable via a new homomorphism.

It was shown that **fes** $\triangleright$ **fes** = **fes**, **fes** $\triangleright$ **fts** = **fts**, **fus** $\triangleright$ **fus** = **fus**, and also that **fts** $\triangleright$ **fus** exhibits decidable CQ entailment [BLMS11]. The following easy lemma comes in handy when working with cuts (see the appendix for the proof).

Lemma 9.2. *For any ruleset $\mathcal{R} = \mathcal{R}_1 \triangleright \cdots \triangleright \mathcal{R}_n$ and any database $\mathcal{D}$, the following holds:*

$$\mathbf{Ch}_\infty(\mathcal{D}, \mathcal{R}) = \mathbf{Ch}_\infty(\ldots \mathbf{Ch}_\infty(\mathbf{Ch}_\infty(\mathcal{D}, \mathcal{R}_1), \mathcal{R}_2) \ldots, \mathcal{R}_n).$$

Toward a more principled way to establish **fps**ness for complex rulesets, we now define categories of rulesets, whose chase causes the instance modifications described in Section 6.3. Then, in view of Observation 6.21 and Theorem 6.23, it is immediate that the **fps** class can integrate "layers" of these introduced types of rulesets.

Definition 9.3. A ruleset $\mathcal{R}$ is called a

- *unary replacement set* (**urs**) iff for all instances $\mathcal{I}$, $\mathbf{Ch}_\infty(\mathcal{I}, \mathcal{R})$ can be obtained from $\mathcal{I}$ via unary replacements;
- *2RPE-definition set* (**reds**) iff for all instances $\mathcal{I}$, $\mathbf{Ch}_\infty(\mathcal{I}, \mathcal{R})$ can be obtained from $\mathcal{I}$ via 2RPE-extensions;
- *FO-definition set* (**fods**) iff for all instances $\mathcal{I}$, $\mathbf{Ch}_\infty(\mathcal{I}, \mathcal{R})$ can be obtained from $\mathcal{I}$ via FO-extensions. $\diamond$

Corollary 9.4. *The following relationships hold:* **fps** $\triangleright$ **urs** = **fps***;* **fps** $\triangleright$ **reds** = **fps***;* **fps** $\triangleright$ **fods** = **fps***.*

Any set of full rules where all rule heads contain only unary predicates is **urs**. The **reds** class includes transitivity rules of the form $\mathtt{P}(x, y) \wedge \mathtt{P}(y, z) \rightarrow \mathtt{P}(x, z)$, which have been known as notoriously difficult to include into decidable rule classes [BBMR15, GMV99, KR21, ST04]. More complex cases of **reds** can be derived from regular grammars and have in fact been described and used under the name *(regular) RBoxes* in expressive description logics [HKS06]. An easy example for **fods**

are concept products [BMP17, RKH08a] like $\mathtt{A}(x) \wedge \mathtt{B}(y) \to \mathtt{P}(x,y)$, but **fods** goes far beyond that: most notably, it contains any full ruleset that is **fus**.[11]

Example 9.5. At this point, let us return to our example ruleset $\mathcal{R}_{\text{bio}}$ from Example 7.1: Rules (1)–(4) are a union of frontier-one (1) and full **fus** (2)-(4) and thus **fps**, by the forthcoming Proposition 10.1. The two rules under (5) constitute a **reds** ruleset stratified on top of (1)-(4). Rule (6) adds another stratum which is **fods**, while rules (7) and (8) add a **urs** stratum. Thus, we obtain a **fps** ▷ **reds** ▷ **fods** ▷ **urs** ruleset which is **fps** by Corollary 9.4 and thus allows for decidable HUSOMSOQ entailment.

10. The Case of First-Order Rewritability

In this section, we will observe that the prospects of accommodating **fus** into the universal-model-based finite-width decidability framework are limited. In fact, already a simple concrete fragment of **fus** exhibits undecidable HUSOMSOQ entailment. Within these factual boundaries, there is still a lot to be achieved depending on one's preferences:

- As just discussed, many existential-quantifier-free rulesets – and in particular *any* that is **fus** and full – can be stratified on top of **fps** rulesets in a way that preserves **fps**ness (and thus decidability of HUSOMSOQ entailment). Sometimes even a tighter integration is possible.
- If one is willing to settle for decidability of CQ rather than full-on HUSOMSOQ entailment, one might resort to the very comprehensive class **fps** ▷ **fus** that properly subsumes both **fps** and **fus**.
- On a more speculative note, there is justified hope that all of **fus** can be captured by a width-based machinery that does not rely on universal models. For some popular (and otherwise defiant) concrete subclass of **fus** we can corroborate this hope.

10.1. **The Price of Expressive Querying.** In prior work [FLOR23], we identified a **fus** ruleset $\mathcal{R}_{\text{grid}}$ whose chase creates an infinite grid, which immediately implies undecidability of entailment of monadic datalog queries (which are subsumed by HUSOMSOQs). By contraposition, $\mathcal{R}_{\text{grid}}$ cannot be **fps**, nor can it be a finite-$\mathtt{w}$ set for *any* MSO-friendly width measure $\mathtt{w}$. As $\mathcal{R}_{\text{grid}}$ was constructed ad hoc and is not contained in any of the known "concrete" **fus** classes, we were hopeful that good width properties could be established for most of these. However, we found that the simple ruleset $\mathcal{R}'_{\text{grid}}$ consisting of the rules

$$\mathtt{A}(x) \to \exists y.\mathtt{E}(x,y) \qquad \mathtt{E}(x,y) \to \mathtt{A}(y) \qquad \mathtt{A}(x) \wedge \mathtt{A}(y) \to \exists z.\mathtt{R}(x,y,z)$$

falls in the rather basic **fus** class of *sticky rules* [CGP12], while it still shares the mentioned unpleasant properties with $\mathcal{R}_{\text{grid}}$. That is, it also creates a grid-like structure albeit in a slightly more covert way. Note that $\mathtt{R}$ can be conceived as a "pairing function": observing that $\mathbf{Ch}_\infty(\{\mathtt{A}(0)\}, \mathcal{R}'_{\text{grid}})$ is isomorphic to

$$\mathcal{I}'_{\text{grid}} = \{\mathtt{A}(n), \mathtt{E}(n, n{+}1), \mathtt{R}(n, m, \langle n,m\rangle) \mid n, m \in \mathbb{N}\},$$

$\mathtt{R}$'s third entries represent grid points; horizontal neighbours are accessible via $\varphi_{\mathtt{H}}(z,z') = \mathtt{R}(x,y,z) \wedge \mathtt{E}(x,x') \wedge \mathtt{R}(x',y,z')$, vertical ones via $\varphi_{\mathtt{V}}(z,z') = \mathtt{R}(x,y,z) \wedge \mathtt{E}(y,y') \wedge \mathtt{R}(x,y',z')$.

[11] In the literature, the class of full **fus** rulesets is also referred to as *bounded datalog*, but we will refrain from that naming to avoid confusion with datalog queries, which are second-order sentences.

We conclude that entailment of monadic datalog queries over sticky rules is undecidable and that sticky rules cannot be captured by any notion of finite-w set for any MSO-friendly w.

10.2. **A Novel fps Class.** We already saw that, as opposed to **fus** in general, the existential-quantifier-free fragment of **fus** shows better compatibility with **fps**. Even beyond stratification, it allows us to define a novel **fps** fragment, for which decidability of querying does not follow from any of the previously established approaches (not even when restricting to CQs). We recall that a rule ρ is *frontier-one* if $|fr(\rho)| = 1$.

Proposition 10.1. *Let $\mathcal{R} = \mathcal{R}_{\text{fr1}} \cup \mathcal{R}_{\text{ff}}$ be a ruleset where $\mathcal{R}_{\text{fr1}}$ is frontier-one and $\mathcal{R}_{\text{ff}}$ is full and* **fus**. *Then $\mathcal{R}$ is* **fps** *and thus allows for decidable HUSOMSOQ entailment.*

The detailed proof can be found in the appendix. Note that the interaction between $\mathcal{R}_{\text{fr1}}$ and $\mathcal{R}_{\text{ff}}$ is not restricted in any way; no stratification is required in this case. Proposition 10.1 ceases to hold, however, when considering arbitrary unions of a *guarded* and a full **fus** rulesets: note that replacing the third rule in $\mathcal{R}'_{\text{grid}}$ by the two rules

$$\mathtt{A}(x) \wedge \mathtt{A}(y) \to \mathtt{P}(x,y) \qquad\qquad \mathtt{P}(x,y) \to \exists z.\mathtt{R}(x,y,z)$$

yields a ruleset of the latter type for which HUSOMSOQ entailment is undecidable for the same reasons as for $\mathcal{R}'_{\text{grid}}$.

10.3. **A Broad Class for Decidable CQ Entailment.** We close this section by noting that the rule class **fps** $\triangleright$ **fus** exhibits decidable CQ entailment.

Proposition 10.2. *Let $\mathcal{R} = \mathcal{R}_{\text{fps}} \triangleright \mathcal{R}_{\text{fus}}$ where $\mathcal{R}_{\text{fps}}$ is* **fps** *and $\mathcal{R}_{\text{fus}}$ is* **fus**. *Then the query entailment problem $(\mathcal{D}, \mathcal{R}) \models q$ for databases $\mathcal{D}$, and (Boolean) CQs q is decidable.*

Proof. The claim follows directly from the insight that the stratification allows us to reduce $(\mathcal{D}, \mathcal{R}_{\text{fps}} \triangleright \mathcal{R}_{\text{fus}}) \models q$ to $(\mathcal{D}, \mathcal{R}_{\text{fps}}) \models \mathsf{rew}_{\mathcal{R}_{\text{fus}}}(q)$. □

Observe that **fps** $\triangleright$ **fus** fully subsumes both **fps** and **fus** without arity restrictions and hence contains the vast majority of the known rules classes for which decidability of CQ entailment has been hitherto established. Unlike for "pure" **fps**, however, it remains unclear if this "hybrid" class corresponds to a structural property of the underlying universal models.

10.4. **Beyond Universal Models.** As just discussed, the **fus** class generally does not exhibit universal models of finite width. This, however, does not rule out the possibility that **fus** is w-controllable for the class of (U)CQs for some appropriate w. In fact, it is a long-standing well-known conjecture that all **fus** rulesets are finitely controllable [GM13a, GM13b]. Should that turn out to be true, **fus** would be $\mathsf{expansion}$-controllable for UCQs and be captured by our decidability framework. What gives some hope in this respect is the fact that finite controllability was already shown to hold for the prominent concrete **fus** subclass of sticky rules [GM13a, GM17].

11. Conclusion

We are optimistic that the generic methods and results presented in this article can advance the understanding of general model-theoretic foundations of decidability of entailment problems. We deem it quite likely that many more useful width notions can be defined (or discovered in the existing literature), leading to novel avenues for establishing decidability of expressive entailment problems. Thus, we

plan to investigate other (existing or newly designed) width notions regarding their friendliness properties. Since less expressive $\mathfrak{L}$ may admit more general $\mathfrak{L}$-friendly w, there is a Pareto-optimality boundary to explore. For controllability $\mathrm{Cont}_{\mathsf{w}}(\mathfrak{F}, \mathfrak{Q})$, there is a three-way tradeoff between the generality of w, the expressivity of $\mathfrak{F}$, and the expressivity of $\mathfrak{Q}$.

One particularly interesting challenge in that respect would be to establish a very general FO-friendly width notion, ideally one that both

- generalizes all the width notions considered in this article and
- still reflects structural model-theoretic properties that are "cognitively graspable".

The second – admittedly rather vague – condition aims to rule out very technical width notions which can generally be obtained by "reverse-engineering" known decision procedures, but which would not advance our model-theoretic understanding of decidability.

We also are confident that our provided "toolbox" for establishing a fragment's partitionwidth-finiteness in a stratification-based, modular manner will be helpful in obtaining specific decidability results for a wide spectrum of logical formalisms in diverse areas related to computational logic. Indeed, several instances of Entailment that are known to be decidable still lack a width-based decidability argument. Notable examples include satisfiability of logics without the finite model property, such as GF+TG and TGF+TG (the guarded and triguarded fragment extended by transitive guards [ST04, KM20]) as well as $\mathcal{C}^2$, but of course also BCQ entailment from **fus** knowledge bases. Our goal is to accommodate these and other cases in our framework.

Finally, as we have already demonstrated in this paper, we are confident that the width-based framework can guide the search for novel, very expressive, syntactically defined combinations of specification and query languages with a decidable entailment problem. Of course, for cases thus identified, subsequent investigations would have to determine the precise corresponding complexities.

Acknowledgements

This article constitutes a central contribution to the European Research Council (ERC) Consolidator Grant 771779 *A Grand Unified Theory of Decidability in Logic-Based Knowledge Representation* (DeciGUT). The work presented herein was only possible thanks to the continuous funding of all of the authors over several years.

We are indebted to various colleagues for discussing our work, and for commenting on earlier drafts of this paper. First and foremost, our thanks go to Achim Blumensath, who patiently answered our numerous questions regarding his ingenious work on partitionwidth. Beyond that, we are grateful for feedback from Bartosz Bednarczyk, Michael Benedikt, David Carral, Bruno Courcelle, Lukas Gerlach, Erich Grädel, Carsten Lutz, Jerzy Marcinkowski, Vincent Peth, Andreas Pieris, Ian Pratt-Hartmann, and certainly many others that we forgot.

We finally thank the anonymous reviewers of this article and precursor works for their constructive feedback that led to numerous improvements.

Appendix A. Overview of Discussed Logical Fragments

A.1. **Logical Fragments in Standard Syntax.** We give a brief description of the logics we discuss in this paper. *First-order logic (FO)* formulae are built from atomic formulae (that is, atoms) using existential quantification $\exists$, universal quantification $\forall$, negation $\neg$, conjunction $\wedge$, and disjunction $\vee$ in the usual way. We use the symbol $\rightarrow$ for implication and $\leftrightarrow$ for equivalence as the usual shortcuts for better readability. In this article, we discuss the following FO fragments:

- *Prefix classes.* For these logics, we assume that the sentences are in prenex normal form and that the quantifier prefix is restricted by a regular expression (e.g., $\exists^*\forall^*$ for the Bernays-Schönfinkel class), following standard nomenclature as used by Börger, Grädel and Gurevich [BGG97].
- *The guarded fragment* (GF). All occurrences of $\forall$ have the form $\forall\mathbf{x}.(\mathtt{P}(\mathbf{y}) \rightarrow \varphi[\mathbf{y}])$, and occurrences of $\exists$ the form $\exists\mathbf{x}.(\mathtt{P}(\mathbf{y}) \wedge \varphi[\mathbf{y}])$ for $\mathbf{x} \subseteq \mathbf{y}$ and predicates $\mathtt{P}$. This shape is called *guarded quantification* and $\mathtt{P}(\mathbf{y})$ the respective *guard*.
- *The triguarded fragment* (TGF). Every quantification over formulae with three or more free variables must be guarded. Equality is disallowed.
- *The unary negation fragment* (UNF). The symbol $\forall$ is disallowed and every occurrence of $\neg$ must be as $\neg\varphi$ with φ containing at most one free variable.
- *The guarded negation fragment* (GNF). The symbol $\forall$ is disallowed and every occurrence of $\neg$ must be in the form of $\mathtt{P}(\mathbf{x}) \wedge \neg\varphi[\mathbf{x}]$.
- *The 2-variable fragment* (FO^2). All variables used must be from $\{x, y\}$.
- *The 2-variabe fragment of FO with counting quantifiers* ($\mathcal{C}^2$): Same as FO^2 but we allow quantifiers $\exists^{\leq n}$, $\exists^{=n}$, $\exists^{\geq n}$ for $n \in \mathbb{N}$, specifying the number of satisfying instantiations of the quantified variable. When measuring formula length, we assume binary encoding of n. $\mathcal{C}^2$ can be expressed in FO (however at the cost of using more variables and an exponential blowup in formula length).

All the considered logics are fragments of *second-order logic* (SO), which extends first-order logic by *second-order quantifiers*, of the form $\exists\mathtt{P}$ or $\forall\mathtt{P}$ for a predicate symbol $\mathtt{P}$. Of particular interest will be the following SO fragments:

- *Guarded second-order logic* (GSO). A second-order quantifier block followed by a GF formula [GHO00]. An alternative but equally expressive way of defining GSO is as arbitrary SO sentences with modified semantics: the second-order quantification only ranges over "guarded relations", that is, relations containing only tuples all of whose elements co-occur in one tuple of a predicate extension. Note that as a consequence, GSO subsumes FO.
- *Monadic second-order logic* (MSO). Second-order quantification is only allowed over unary predicate symbols. In terms of expressivity, MSO subsumes FO and is subsumed by GSO.
- *Universal second-order logic* ($\forall$SO). The syntactic fragment of SO where all second-order quantifiers are universal and prenex.
- *Disjunctive datalog* (DDlog) [EGM97]. A disjunctive datalog sentence over a signature Σ is a $\forall$SO sentence of the shape $\forall\Sigma'.\neg\bigwedge_i \forall\boldsymbol{x}_i.\big(\varphi_i \rightarrow \psi_i\big)$, where Σ' consists of fresh predicates, every φ_i is a (possibly empty) conjunction of atoms over $\Sigma \cup \Sigma'$, every ψ_i is a (possibly empty) disjunction of atoms over Σ', and every $\boldsymbol{x}_i$ comprises all the variables occurring in $\varphi_i \rightarrow \psi_i$.
- *Frontier-guarded disjunctive datalog* (FGDDlog). A disjunctive datalog (DDL) sentence where in every subformula $\forall\boldsymbol{x}_i.\big(\varphi_i \rightarrow \psi_i\big)$, the conjunction φ_i contains one atom over Σ which jointly carries all the variables occurring in ψ_i. Every FGDDlog sentence can be transformed into an equivalent GSO formula [GHO00].

- *Monadic disjunctive datalog* (MDDlog). The fragment of DDlog where all predicates in Σ' are unary. This makes it a syntactic fragment of MSO.
- *Datalog* (Dlog). The fragment of DDlog where every ψ_i consists of at most one disjunct.
- *Frontier-guarded datalog* (FGDlog). The syntactic intersection of FGDDlog and Dlog.
- *Monadic datalog* (MDlog). The syntactic intersection of MDDlog and Dlog.

A.2. **Description Logics.** Description logics (DLs) are a very prominent family of formalisms used in Knowledge Representation. While using a different syntax than FOL, all DLs discussed in this article can be understood as FOL fragments via a syntactic translation into standard FOL syntax.[12] For the sake of conciseness, we will omit certain "syntactic extras" from the presentation in case they are not relevant for the logical expressivity of the DLs. For a full treatise on DLs, we refer the interested reader to the standard literature [BHLS17, Rud11].

Like in other FOL fragments, DLs are built on sets of constants and of predicates from some signature. Constants from $\mathbf{C}$ are referred to as *individual names*, predicates from Σ are subdivided into unary predicates (Σ_1, referred to as *concept names*) and binary predicates (Σ_2, referred to as *role names*), whereas predicates of higher arity are disallowed. Σ_2 is subdivided into *simple role names* Σ_2^{s} and *non-simple role names* Σ_2^{ns}, the latter containing as a distinguished predicate the *universal role* $\mathtt{U}$ and being strictly ordered by some strict order $\prec$.[13] Then, the set $\mathscr{R}^{\mathrm{s}}$ of *simple role expressions* is defined by

$$R_1, R_2 ::= \mathtt{S} \mid \mathtt{S}^- \mid R_1 \cup R_2 \mid R_1 \cap R_2 \mid R_1 \setminus R_2,$$

with $\mathtt{S} \in \Sigma_2^{\mathrm{s}}$, while the set of (arbitrary) *role expressions* is $\mathscr{R} = \mathscr{R}^{\mathrm{s}} \cup \{\mathtt{R}, \mathtt{R}^- \mid \mathtt{R} \in \Sigma_2^{\mathrm{ns}}\}$. For convenience, we introduce the function inv that "inverts" roles, i.e. we set $\mathsf{inv}(\mathtt{R}) := \mathtt{R}^-$ and $\mathsf{inv}(\mathtt{R}^-) := \mathtt{R}$ in order to simplify notation. In the sequel, we will use the symbols R, S, possibly with subscripts, to denote role expressions.

Having these sets at hand, we can now turn to the three building blocks of $\mathcal{SROIQ}b_s$ knowledge bases: RBox, TBox and ABox.

RBox. A *role inclusion axiom* (RIA, sometimes also referred to as *role chain axiom*) is a statement of the form $R_1 \circ \ldots \circ R_n \sqsubseteq R$ where $R_1, \ldots, R_n, R$ are roles. As a special case thereof (for $n = 1$), we obtain *direct role inclusions* $R \sqsubseteq S$. A finite set of such RIAs is called a *role hierarchy*. A role hierarchy is *regular* if every RIA is of one of the forms

$$R \circ R \sqsubseteq R, \quad \mathsf{inv}(R) \sqsubseteq R, \quad S_1 \circ \ldots \circ S_n \sqsubseteq R, \quad R \circ S_1 \circ \ldots \circ S_n \sqsubseteq R, \quad S_1 \circ \ldots \circ S_n \circ R \sqsubseteq R,$$

such that $R \in \Sigma_2$ is a (non-inverse) non-simple role name $\mathtt{R} \in \Sigma_2^{\mathrm{ns}}$, and $S_i \prec R$ for $i = 1, \ldots, n$ whenever S_i is non-simple. A $\mathcal{SROIQ}b_s$ *RBox* (usually denoted by $\boldsymbol{\mathcal{R}}$) is a regular role hierarchy.

TBox. Given a $\mathcal{SROIQ}b_s$ RBox $\boldsymbol{\mathcal{R}}$, we now inductively define *concept expressions* (also simply called *concepts*) as follows:

- the symbols $\top$ and $\bot$ as well as all concept names $\mathtt{A} \in \Sigma_1$ are concept expressions,
- $\{\mathtt{a}_1, \ldots, \mathtt{a}_n\}$ is a concept expression for any finite set $\{\mathtt{a}_1, \ldots, \mathtt{a}_n\} \subseteq \mathbf{C}$ of individual names,

[12] Alternatively, DLs can be seen as logics in their own right, with a special model-theoretic semantics, or as extended multi-modal logics. All these formulations are equivalent and for our purpose it is most convienient to view DLs as FOL fragments employing a non-standard syntax.

[13] In the original definition of $\mathcal{SROIQ}b_s$, simplicity of roles and $\prec$ are not given a priori, but meant to be implicitly determined by the set of axioms. Our choice to fix them explicitly upfront simplifies the presentation without restricting expressivity.

- if C and D are concept expressions then so are $\neg C$ as well as $C \sqcap D$ and $C \sqcup D$,
- if R is a role and C is a concept expression, then $\exists R.C$ and $\forall R.C$ are concept expressions,
- if R is a simple role, n is a natural number and C is a concept expression, then $\geqslant nR.C$ and $\leqslant nR.C$ as well as $\exists R.\mathsf{Self}$ are also concept expressions.

We will denote the set of all concept expressions thus defined by $\mathscr{C}$. Throughout this section, the symbols C, D will be used to denote concept expressions. A *general concept inclusion axiom* (short: GCI) has the form $C \sqsubseteq D$ where C and D are concepts. Finally, a $\mathcal{SROIQ}b_s$ *TBox* (usually denoted by $\boldsymbol{\mathcal{T}}$) is a finite set of GCIs.

ABox. An *individual assertion* can have any of the following forms:

$$C(\mathtt{a}), \qquad R(\mathtt{a},\mathtt{b}), \qquad \neg R(\mathtt{a},\mathtt{b}), \qquad \mathtt{a} \approx \mathtt{b}, \qquad \mathtt{a} \not\approx \mathtt{b},$$

with $\mathtt{a},\mathtt{b} \in \mathbf{C}$ individual names, $C \in \mathscr{C}$ a concept expression, and $R \in \mathscr{R}$ a role. A $\mathcal{SROIQ}b_s$ *ABox* (usually denoted by $\boldsymbol{\mathcal{A}}$) is a finite set of individual assertions.

Finally, a $\mathcal{SROIQ}b_s$ *knowledge base* $\boldsymbol{\mathcal{K}}$ is the union of an RBox $\boldsymbol{\mathcal{R}}$ and a TBox $\boldsymbol{\mathcal{T}}$ as well as an ABox $\boldsymbol{\mathcal{A}}$ for $\boldsymbol{\mathcal{R}}$. The elements of $\boldsymbol{\mathcal{K}}$ are referred to as *axioms*.

In order to establish the connection between DL syntax and FOL, we now define an easy syntactic translation fol. Every $\mathcal{SROIQ}b_s$ knowledge base $\boldsymbol{\mathcal{K}}$ thus translates via fol to a FOL sentence $\mathsf{fol}(\boldsymbol{\mathcal{K}})$. We define $\mathsf{fol}(\boldsymbol{\mathcal{K}}) = \bigwedge_{\alpha\in\boldsymbol{\mathcal{K}}} \mathsf{fol}(\alpha)$, i.e., we build the conjunction over the separately translated axioms of the knowledge base. How exactly $\mathsf{fol}(\alpha)$ is defined depends on the type of the axiom α:

$$\begin{aligned}
\mathsf{fol}(R_1 \circ \ldots \circ R_n \sqsubseteq R) &= \forall x_0 \ldots x_n (\textstyle\bigwedge_{1\leq i\leq n} \mathsf{fol}_{\mathscr{R}}(R_i, x_{i-1}, x_i)) \rightarrow \mathsf{fol}_{\mathscr{R}}(R, x_0, x_n)\\
\mathsf{fol}(C \sqsubseteq D) &= \forall x_0 (\mathsf{fol}_{\mathscr{C}}(C, x_0) \rightarrow \mathsf{fol}_{\mathscr{C}}(D, x_0))\\
\mathsf{fol}(C(\mathtt{a})) &= \mathsf{fol}_{\mathscr{C}}(C, x_0)[x_0/\mathtt{a}]\\
\mathsf{fol}(R(\mathtt{a},\mathtt{b})) &= \mathsf{fol}_{\mathscr{R}}(R, x_0, x_1)[x_0/\mathtt{a}][x_1/\mathtt{b}]\\
\mathsf{fol}(\neg R(\mathtt{a},\mathtt{b})) &= \neg \mathsf{fol}(R(\mathtt{a},\mathtt{b}))\\
\mathsf{fol}(\mathtt{a} \approx \mathtt{b}) &= \mathtt{a} = \mathtt{b}\\
\mathsf{fol}(\mathtt{a} \not\approx \mathtt{b}) &= \neg(\mathtt{a} = \mathtt{b})
\end{aligned}$$

These definitions make use of auxiliary translation functions $\mathsf{fol}_{\mathscr{R}} : \mathscr{R} \times \mathbf{V} \times \mathbf{V} \rightarrow \mathrm{FOL}$ for roles and $\mathsf{fol}_{\mathscr{C}} : \mathscr{C} \times \mathbf{V} \rightarrow \mathrm{FOL}$ for concepts (where we assume $\mathbf{V} = \{x_0, x_1, \ldots\}$ for the set of variables):

$$\begin{aligned}
\mathsf{fol}_{\mathscr{R}}(\mathtt{U}, x_i, x_j) &= \text{true}\\
\mathsf{fol}_{\mathscr{R}}(\mathtt{S}, x_i, x_j) &= \mathtt{S}(x_i, x_j)\\
\mathsf{fol}_{\mathscr{R}}(\mathtt{S}^-, x_i, x_j) &= \mathtt{S}(x_j, x_i)\\
\mathsf{fol}_{\mathscr{R}}(R_1 \cup R_2, x_i, x_j) &= \mathsf{fol}_{\mathscr{R}}(R_1, x_i, x_j) \vee \mathsf{fol}_{\mathscr{R}}(R_2, x_i, x_j)\\
\mathsf{fol}_{\mathscr{R}}(R_1 \cap R_2, x_i, x_j) &= \mathsf{fol}_{\mathscr{R}}(R_1, x_i, x_j) \wedge \mathsf{fol}_{\mathscr{R}}(R_2, x_i, x_j)\\
\mathsf{fol}_{\mathscr{R}}(R_1 \setminus R_2, x_i, x_j) &= \mathsf{fol}_{\mathscr{R}}(R_1, x_i, x_j) \wedge \neg \mathsf{fol}_{\mathscr{R}}(R_2, x_i, x_j)
\end{aligned}$$

$$
\begin{aligned}
\mathsf{fol}_{\mathscr{C}}(\mathtt{A}, x_i) &= \mathtt{A}(x_i)\\
\mathsf{fol}_{\mathscr{C}}(\top, x_i) &= \text{true}\\
\mathsf{fol}_{\mathscr{C}}(\bot, x_i) &= \text{false}\\
\mathsf{fol}_{\mathscr{C}}(\{\mathtt{a}_1, \ldots, \mathtt{a}_n\}, x_i) &= \textstyle\bigvee_{1 \le j \le n} x_i = \mathtt{a}_j\\
\mathsf{fol}_{\mathscr{C}}(\neg C, x_i) &= \neg \mathsf{fol}_{\mathscr{C}}(C, x_i)\\
\mathsf{fol}_{\mathscr{C}}(C \sqcap D, x_i) &= \mathsf{fol}_{\mathscr{C}}(C, x_i) \wedge \mathsf{fol}_{\mathscr{C}}(D, x_i)\\
\mathsf{fol}_{\mathscr{C}}(C \sqcup D, x_i) &= \mathsf{fol}_{\mathscr{C}}(C, x_i) \vee \mathsf{fol}_{\mathscr{C}}(D, x_i)\\
\mathsf{fol}_{\mathscr{C}}(\exists R.C, x_i) &= \exists x_{i+1}.\big(\mathsf{fol}_{\mathscr{R}}(R, x_i, x_{i+1}) \wedge \mathsf{fol}_{\mathscr{C}}(C, x_{i+1})\big)\\
\mathsf{fol}_{\mathscr{C}}(\forall R.C, x_i) &= \forall x_{i+1}.\big(\mathsf{fol}_{\mathscr{R}}(R, x_i, x_{i+1}) \rightarrow \mathsf{fol}_{\mathscr{C}}(C, x_{i+1})\big)\\
\mathsf{fol}_{\mathscr{C}}(\exists R.\mathsf{Self}, x_i) &= \mathsf{fol}_{\mathscr{R}}(R, x_i, x_i)\\
\mathsf{fol}_{\mathscr{C}}(\geqslant n R.C, x_i) &= \exists x_{i+1} \ldots x_{i+n}.\big(\textstyle\bigwedge_{i+1 \le j < k \le i+n}(x_j \neq x_k)\\
&\qquad \wedge \textstyle\bigwedge_{i+1 \le j \le i+n}(\mathsf{fol}_{\mathscr{R}}(R, x_i, x_j) \wedge \mathsf{fol}_{\mathscr{C}}(C, x_j))\big)\\
\mathsf{fol}_{\mathscr{C}}(\leqslant n R.C, x_i) &= \neg \mathsf{fol}_{\mathscr{C}}(\geqslant (n+1) R.C, x_i)
\end{aligned}
$$

Obviously, $\mathsf{fol}_{\mathscr{R}}$ assigns to a role a FOL formula with (at most) two free variables whereas $\mathsf{fol}_{\mathscr{C}}$ assigns to a concept a FOL formula with (at most) one free variable.

After defining syntax and FOL-translation of $\mathcal{SROIQ}b_s$ we now obtain all the DLs discussed in this article as fragments of $\mathcal{SROIQ}b_s$ by disallowing certain expressive features.

- $\mathcal{SRIQ}b_s$ is obtained from $\mathcal{SROIQ}b_s$ by disallowing concept expressions of the form $\{\mathtt{a}_1, \ldots, \mathtt{a}_n\}$.
- $\mathcal{SROI}b_s$ is obtained from $\mathcal{SROIQ}b_s$ by disallowing concept expressions of the form $\geqslant nR.C$ and $\leqslant nR.C$.
- $\mathcal{SROQ}b_s$ is obtained from $\mathcal{SROIQ}b_s$ by disallowing inverse roles, i.e. the constructor $(\cdot)^-$ inside role expressions.

Finally, the description logics $\mathcal{ALCHOIQ}b$, $\mathcal{ALCHIQ}b$, $\mathcal{ALCHOI}b$, and $\mathcal{ALCHOQ}b$ are obtained from $\mathcal{SRIOQ}b_s$, $\mathcal{SRIQ}b_s$, $\mathcal{SROI}b_s$, and $\mathcal{SROQ}b_s$, respectively, by disallowing the usage of non-simple role names and concept expressions of the form $\exists R.\mathsf{Self}$.

A structural analysis unveils that $\mathcal{ALCHOI}b$ is a fragment of GF. It is also well-known that $\mathcal{ALCHOIQ}b$ and all its fragments can be expressed in $\mathcal{C}^2$ [RKH08b].

Appendix B. Proofs for Section 5

Lemma B.1. *Let Ψ be a homomorphism-closed $\forall$SO sentence. Then for any instance $\mathcal{I}$ holds: $\mathcal{I} \models \Psi$ iff there is some finite $\mathcal{I}' \subseteq \mathcal{I}$ satisfying $\mathcal{I}' \models \Psi$.*

Proof. Let $\Psi = \forall \Sigma'.\Psi^*$. Note that for any $\mathcal{I}' \subseteq \mathcal{I}$, the identity function is a homomorphism from $\mathcal{I}'$ to $\mathcal{I}$.

Then, the "if" part follows immediately from the fact that Ψ is homomorphism-closed. For the "only if" part, assume $\mathcal{I} \models \Psi$. Let now $\mathcal{I}^c$ denote the (very likely infinite) set of ground facts obtained from $\mathcal{I}$ by uniformly replacing all nulls by fresh constant names. Note that $\mathcal{I}$ has a homomorphism into every model of the sentence set $\mathcal{I}^c$. Thus, thanks to Ψ being homomorphism-closed by assumption, we can conclude that $\mathcal{I}^c$ logically entails Ψ. This means that $\mathcal{I}^c \cup \{\neg\Psi\}$ is unsatisfiable, therefore also $\mathcal{I}^c \cup \{\neg\Psi^*\}$ must be an unsatisfiable set of FO sentences. By compactness of FOL, this implies that there must be a finite $\mathcal{I}^c_{\text{fin}} \subseteq \mathcal{I}^c$ for which $\mathcal{I}^c_{\text{fin}} \cup \{\neg\Psi^*\}$ is already unsatisfiable, implying unsatisfiability of $\mathcal{I}^c_{\text{fin}} \cup \{\neg\Psi\}$. Thus we obtain that $\mathcal{I}^c_{\text{fin}}$ logically entails Ψ. Replacing the introduced constant names back by nulls, we obtain a finite instance $\mathcal{I}_{\text{fin}} \subseteq \mathcal{I}$, as desired. □

Theorem 5.2. *Let Φ be an FO sentence having some finitely universal model set $\mathfrak{I}$ and let Ψ be a homomorphism-closed $\forall$SO sentence. Then $\Phi \not\models \Psi$ iff there exists a model $\mathcal{I} \in \mathfrak{I}$ with $\mathcal{I} \not\models \Psi$.*

Proof. The "if" part is an easy consequence of the fact the $\mathcal{I}$ serves as a countermodel.

For the "only if" part, assume $\Phi \not\models \Psi$, that is, there is a countermodel $\mathcal{J} \models \Phi$ with $\mathcal{J} \not\models \Psi$. Then, because of $\mathcal{J} \models \Phi$, there must be some $\mathcal{I} \in \mathfrak{I}$ of which every finite subset maps homomorphically into $\mathcal{J}$. We conclude our argument by showing that $\mathcal{I} \not\models \Psi$. Toward a contradiction, suppose $\mathcal{I}$ were a model of Ψ. Then Lemma B.1 would imply the existence of some finite $\mathcal{I}' \subseteq \mathcal{I}$ with $\mathcal{I}' \models \Psi$. Yet then, noting that $\mathcal{I}'$ maps homomorphically to $\mathcal{J}$ and Ψ is homomorphism-closed, we would conclude $\mathcal{J} \models \Psi$ contradicting the initial assumption. □

Lemma 5.3. *Let Φ be a FO sentence and Ψ a homomorphism-closed $\forall$SO sentence. Then $\Phi \models \Psi$ iff there exists a union of BCQs $Q = \bigvee_{i=1}^{n} q_i$ satisfying $\Phi \models Q$ and $Q \models \Psi$.*

Proof. The "if" direction is immediate. For the "only if" direction, let $\mathfrak{C}$ denote the set of BCQs q satisfying $q \models \Psi$. Now assume $\Phi \models \Psi$. By Lemma B.1, this means that every model of Φ must be a model of some $q \in \mathfrak{C}$. Yet, this means that $\{\Phi\} \cup \{\neg q \mid q \in \mathfrak{C}\}$ is an unsatisfiable set of FO sentences. Again, by compactness of FO, we can conclude that $\{\Phi\} \cup \{\neg q \mid q \in \mathfrak{C}'\}$ must be unsatisfiable for some finite $\mathfrak{C}' \subseteq \mathfrak{C}$. Yet, this means that Φ logically entails $\bigvee_{q \in \mathfrak{C}'} q$ which is a union of BCQs, so we have identified the claimed Q. □

Appendix C. Detailed Material on Section 6

C.1. **MSO-Definition of Well-Decoratedness (Theorem 6.17).** Recall that a decorated tree $\mathcal{T}$ is called a *well-decorated tree iff*

(1) for every null $s \in \{0,1\}^*$, if $\mathcal{T}$ contains a fact of the form $\mathtt{New}_{\mathtt{c}}(s)$ or $\mathtt{New}_{*}(s)$ (in which case s will be called a *pseudoleaf*), then $\mathcal{T}$ contains no other fact of the form $\mathtt{Dec}(s)$ with $\mathtt{Dec} \in \mathrm{Dec}(\mathbb{L}, \mathrm{Cnst}, \Sigma)$,
(2) for every pseudoleaf s and $s' \in \{0,1\}^+$, $\mathtt{Dec}(ss') \notin \mathcal{T}$ for any $\mathtt{Dec} \in \mathrm{Dec}(\mathbb{L}, \mathrm{Cnst}, \Sigma)$ (i.e., descendants of pseudoleafs are undecorated),
(3) for every $\mathtt{c} \in \mathrm{Const}$, $\mathcal{T}$ contains at most one fact of the form $\mathtt{New}_{\mathtt{c}}(s)$ (i.e., constants are unique),
(4) for every null $s \in \{0,1\}^*$, if $\mathtt{Dec}(s), \mathtt{Dec}'(s) \in \mathcal{T}$ with $\mathtt{Dec}, \mathtt{Dec}' \in \mathrm{Instr}(\mathbb{L}, \Sigma)$ then $\mathtt{Dec}$ and $\mathtt{Dec}'$ are compatible (i.e., no node is decorated by incompatible instructions).

Here, fixing $\mathbb{L}, \Sigma$, and Cnst, we will define a MSO-sentence Θ such that for any $(\mathbb{L}, \mathrm{Cnst}, \Sigma)$-decorated tree $\mathcal{T}$, $\mathcal{T}$ is well-decorated *iff* $\mathcal{T} \vDash \Theta$.
Let us denote $\mathrm{Dec}(\mathbb{L}, \mathrm{Cnst}, \Sigma)$ with Γ. We write $\varphi_{\mathrm{ancestor}}(x, y)$ to abbreviate

$$\neg(x = y) \wedge \forall X. \big(x \in X \wedge \forall z, z'. \big((z \in X \wedge (\mathtt{Succ}_0(z, z') \vee \mathtt{Succ}_1(z, z'))) \rightarrow z' \in X\big) \rightarrow y \in X\big).$$

We first define the following four sentences, where $\mathtt{c} \in \mathrm{Cnst}, \mathtt{d} \in \mathrm{Cnst} \cup \{*\}$, and then use these to define Θ below:

$$\Theta^{(1)} = \bigwedge_{\mathtt{d} \in \mathrm{Cnst} \cup \{*\}} \forall x. \big(\mathtt{New}_{\mathtt{d}}(x) \to \bigwedge_{\mathtt{Dec} \in \Gamma \setminus \{\mathtt{New}_{\mathtt{d}}\}} \neg \mathtt{Dec}(x)\big)$$

$$\Theta^{(2)} = \bigwedge_{\mathtt{d} \in \mathrm{Cnst} \cup \{*\}} \Big(\forall xy. \big(\mathtt{New}_{\mathtt{d}}(x) \wedge \varphi_{\mathrm{ancestor}}(x, y)\big) \to \bigwedge_{\mathtt{Dec} \in \Gamma} \neg \mathtt{Dec}(y)\Big)$$

$$\Theta^{(3)} = \bigwedge_{\mathtt{c} \in \mathrm{Cnst}} \Big(\forall xy. \big(\mathtt{New}_{\mathtt{c}}(x) \wedge \mathtt{New}_{\mathtt{c}}(y)\big) \to x = y\Big)$$

$$\Theta^{(4)} = \forall x. \bigwedge_{\lambda\text{-}\mathtt{ReCol}^i_\kappa, \lambda'\text{-}\mathtt{ReCol}^i_\kappa \in \Gamma, \lambda \neq \lambda'} \neg\big(\lambda\text{-}\mathtt{ReCol}^i_\kappa(x) \wedge \lambda'\text{-}\mathtt{ReCol}^i_\kappa(x)\big) \wedge \bigwedge_{\lambda\text{-}\mathtt{Add}^I_{\kappa\mu}, \lambda'\text{-}\mathtt{Add}^I_{\kappa\mu} \in \Gamma, \lambda \neq \lambda'} \neg\big(\lambda\text{-}\mathtt{Add}^I_{\kappa\mu}(x) \wedge \lambda'\text{-}\mathtt{Add}^I_{\kappa\mu}(x)\big)$$

Finally, we define

$$\Theta = \Theta^{(1)} \wedge \Theta^{(2)} \wedge \Theta^{(3)} \wedge \Theta^{(4)}.$$

C.2. **A Normal Form for Well-Decorated Trees.** In order to simplify the proofs of Section 6.3 and Section 6.4 we now expose a normal form for well-decorated trees.

Definition C.1. Let $\mathcal{T}$ be a $(\mathbb{L}, \mathrm{Cnst})$-well-decorated tree. For $\mathtt{c} \in \mathrm{Cnst}$, let $\mathtt{c}^{\mathcal{T}}$ denote the unique $s \in \{0,1\}^*$ with $\mathtt{c}(s) \in \mathcal{T}$. Moreover, we define the injection $\mathsf{orig} : \mathsf{dom}(\mathcal{I}^{\mathcal{T}}) \to \{0,1\}^*$ by letting $\mathsf{orig}(e) = e^{\mathcal{T}}$ if $e \in \mathrm{Cnst}$, and $\mathsf{orig}(e) = e$ otherwise. ◇

Let $u_1, \ldots, u_n$ be nodes in a well-decorated tree $\mathcal{T}$. We denote by $\mathsf{cca}(u_1, \ldots, u_n)$ the *closest common ancestor of* $u_1, \ldots, u_n$, which is the unique node such that (i) $\mathsf{cca}(u_1, \ldots, u_n)$ is an ancestor of $u_1, \ldots, u_n$ and (ii) every other common ancestor of $u_1, \ldots, u_n$ is a strict ancestor of $\mathsf{cca}(u_1, \ldots, u_n)$.

Definition C.2. Let Σ be a finite signature, Const a finite set of constants and $\mathbb{L}$ a finite set of colors. Let $\mathcal{T}$ be a $(\mathbb{L}, \mathrm{Cnst}, \Sigma)$-well-decorated tree. We call $\mathcal{T}$ to be in *prenormal form*, if for every fact $\mathtt{R}(\boldsymbol{e}) \in \mathcal{I}^{\mathcal{T}}$ the following holds:

(1) If $\boldsymbol{e} = (e, \ldots, e)$ for one term e from $\mathcal{I}^{\mathcal{T}}$, then for $s = \mathsf{orig}(e)$ there are $i \in \{0,1\}$, $s' \in \{0,1\}^*$ with $s = s'i$ such that $\mathtt{R}\text{-}\mathtt{Add}^i_{o_{\mathsf{ar}(\mathtt{R})}}(s') \in \mathcal{T}$.
(2) Otherwise, there are $I \subsetneq \{1, \ldots, |\boldsymbol{e}|\}$ and $\kappa_1, \kappa_2 \in \mathbb{L}$ with $\mathsf{ar}(\kappa_1) = |I|$, $\mathsf{ar}(\kappa_2) = |\boldsymbol{e}| - |I|$, such that $\mathtt{R}\text{-}\mathtt{Add}^I_{\kappa_1\kappa_2}(s) \in \mathcal{T}$ for $s = \mathsf{cca}(\mathsf{orig}(\boldsymbol{e}))$, and $\boldsymbol{e} \in \mathsf{col}^{\mathcal{T}}_{s0}(\kappa_1) +\!\!+_I \mathsf{col}^{\mathcal{T}}_{s1}(\kappa_2)$.

A $(\mathbb{L}, \mathrm{Cnst}, \Sigma)$-well-decorated tree $\mathcal{T}$ in prenormal form is in *normal form* if the following property holds: If $\mathtt{R}\text{-}\mathtt{Add}^i_\lambda(s) \in \mathcal{T}$ for some $\mathtt{R} \in \Sigma$, $i \in \{0,1\}$, and $s \in \{0,1\}^*$, then si is a pseudoleaf and $\lambda = o_{\mathsf{ar}(\mathtt{R})}$. ◇

Lemma C.3. *Let Σ be a finite signature,* Const *a finite set of constants and $\mathbb{L}$ a finite set of colors. Let $\mathcal{T}$ be a $(\mathbb{L}, \mathrm{Cnst}, \Sigma)$-well-decorated tree. Then there is a $(\mathbb{L}, \mathrm{Cnst}, \Sigma)$-well-decorated tree $\mathcal{T}' \supseteq \mathcal{T}$ that is in prenormal form and for which $\mathcal{I}^{\mathcal{T}'} = \mathcal{I}^{\mathcal{T}}$ holds.*

Proof. We will show how $\mathcal{T}'$ can be obtained by enriching $\mathcal{T}$ with additional facts. Every $\mathtt{R}(\boldsymbol{e}) \in \mathcal{I}^{\mathcal{T}}$ falls into one of three cases: Either

(1) $\boldsymbol{e} = (e, \ldots, e)$ for some term e,
(2) the above does not hold and there is some $\lambda^* \in \mathbb{L}$ with $\mathsf{ar}(\lambda^*) = |\boldsymbol{e}|$ and $\boldsymbol{e} \in \mathsf{col}^{\mathcal{T}}_{\varepsilon}(\lambda^*)$, or
(3) neither of the two above – in particular, there is no $\lambda^* \in \mathbb{L}$ as described previously.

In Case 1, let $s_e = \mathsf{orig}(e)$, let s'_e be the unique parent node of s_e and let $i_e \in \{0, 1\}$ be such that $s_e = s'_e i_e$. Then we add for every fact $\mathtt{R}(e, \dots, e) \in \mathcal{I}^{\mathcal{T}}$ the atom $\mathtt{R\text{-}Add}^{i_e}_{o_{\mathsf{ar}(\mathtt{R})}}(s'_e)$ to $\mathcal{T}$ to obtain $\mathcal{T}_1$.
In Case 2, $\boldsymbol{e}$ must have been colored for the first time at $s_{\boldsymbol{e}} = \mathsf{cca}(\mathsf{orig}(\boldsymbol{e}))$. In particular, $\mathcal{T}$ must contain an atom $\lambda\text{-}\mathtt{Add}^{I}_{\kappa_1\kappa_2}(s_{\boldsymbol{e}})$ such that $\boldsymbol{e} \in \mathsf{col}^{\mathcal{T}}_{s_{\boldsymbol{e}}}(\lambda)$ for some $\kappa_1, \kappa_2 \in \mathbb{L}$ with $\mathsf{ar}(\kappa_1) + \mathsf{ar}(\kappa_2) = \mathsf{ar}(\mathtt{R})$, $I \subsetneq \{1, \dots, \mathsf{ar}(\mathtt{R})\}$, and $\boldsymbol{e} \in \mathsf{col}^{\mathcal{T}}_{s0}(\kappa_1) +\!\!+_I \mathsf{col}^{\mathcal{T}}_{s1}(\kappa_2)$. Adding for every fact $\mathtt{R}(\boldsymbol{e}) \in \mathcal{I}^{\mathcal{T}}$ falling under this case the corresponding fact $\mathtt{R\text{-}Add}^{I}_{\kappa_1\kappa_2}(s_{\boldsymbol{e}})$ to $\mathcal{T}_1$ we obtain $\mathcal{T}'$.
In Case 3, $\mathtt{R\text{-}Add}^{I}_{\kappa_1\kappa_2}(s_{\boldsymbol{e}}) \in \mathcal{T}$ must already hold for $s_{\boldsymbol{e}} = \mathsf{cca}(\mathsf{orig}(\boldsymbol{e}))$, some $\kappa_1, \kappa_2 \in \mathbb{L}$ with $\mathsf{ar}(\kappa_1) + \mathsf{ar}(\kappa_2) = \mathsf{ar}(\mathtt{R})$, $I \subsetneq \{1, \dots, \mathsf{ar}(\mathtt{R})\}$, and $\boldsymbol{e} \in \mathsf{col}^{\mathcal{T}}_{s0}(\kappa_1) +\!\!+_I \mathsf{col}^{\mathcal{T}}_{s1}(\kappa_2)$. Hence there is nothing to do here.

It is straightforward to see that $\mathcal{T}'$ is still a $(\mathbb{L}, \mathrm{Cnst}, \Sigma)$-well-decorated tree and that it is in prenormal form. $\mathcal{I}^{\mathcal{T}'} = \mathcal{I}^{\mathcal{T}}$ clearly holds, as throughout the described construction we only introduced relations in $\mathcal{T}'$ closer to the introduction of the involved entities than or at the node as in $\mathcal{T}'$. □

Lemma C.4. *Let Σ be a finite signature,* Const *a finite set of constants and $\mathbb{L}$ a finite set of colors. Let $\mathcal{T}$ be a $(\mathbb{L}, \mathrm{Cnst}, \Sigma)$-well-decorated tree in prenormalform. Then there is a $(\mathbb{L}, \mathrm{Cnst}, \Sigma)$-well-decorated tree $\mathcal{T}' \subseteq \mathcal{T}$ that is in normal form. Furthermore, $\mathcal{I}^{\mathcal{T}'} = \mathcal{I}^{\mathcal{T}}$.*

Proof. For every $\mathtt{R\text{-}Add}^{i}_{\lambda}(s) \in \mathcal{T}$ do the following: if $\lambda \neq o_{\mathsf{ar}(\mathtt{R})}$ or si is no pseudoleaf of $\mathcal{T}$, remove $\mathtt{R\text{-}Add}^{i}_{\lambda}(s)$ from $\mathcal{T}$. After having done this, we obtain $\mathcal{T}'$ which is well-decorated and in normal form. $\mathcal{I}^{\mathcal{T}'} = \mathcal{I}^{\mathcal{T}}$ holds since by $\mathcal{T}$ being in prenormal form, the removed facts of the type $\mathtt{R\text{-}Add}^{i}_{\lambda}(s)$ are redundant. □

Corollary C.5. *Let Σ be a finite signature,* Const *a finite set of constants and $\mathbb{L}$ a finite set of colors. Let $\mathcal{T}$ be a $(\mathbb{L}, \mathrm{Cnst}, \Sigma)$-well-decorated tree. Then there is a $(\mathbb{L}, \mathrm{Cnst}, \Sigma)$-well-decorated tree $\mathcal{T}'$ that is in normal form. Furthermore, $\mathcal{I}^{\mathcal{T}'} = \mathcal{I}^{\mathcal{T}}$.*

Proof. Apply Lemma C.3 and Lemma C.4 successively. □

C.3. **Proofs for Section 6.3.**

Observation 6.21. *Finiteness of treewidth, cliquewidth, and partitionwidth are all preserved under unary replacements.*

We reformulate the above result into the following (more explicit) technical lemma, which we prove afterwards.

Lemma C.6. *Let w be any of the following width measures: treewidth, cliquewidth, partitionwidth. Let Σ be a signature and $\mathtt{U} \in \Sigma$ a unary predicate. Let $\mathcal{I}$ be an instance over Σ such that $\mathsf{w}(\mathcal{I})$ is finite, and let $A \subseteq \mathsf{dom}(\mathcal{I})$ be an arbitrary set of domain elements. Then $\mathsf{w}(\mathcal{I}^{\mathtt{U}:=A})$ is finite.*

Proof. We consider the case where w is treewidth. Let $T = (V, E)$ be a tree decomposition of $\mathcal{I}$ and k be the width of T. Note that, by virtue of being a tree decomposition of $\mathcal{I}$, T satisfies the following:

(1) each node $X \in V$ is a set of terms of $\mathcal{I}$ with $\bigcup_{X \in V} X = \mathsf{dom}(\mathcal{I})$,
(2) for each $\mathtt{R}(t_1, \dots, t_n) \in \mathcal{I}$, there is an $X \in V$ with $\{t_1, \dots, t_n\} \subseteq X$,
(3) for each term t in $\mathcal{I}$, the subgraph of T consisting of the nodes X with $t \in X$ is connected.

We now argue that T also is a tree decomposition of $\mathcal{I}^{\mathtt{U}:=A}$, without requiring any modification. Observe $\mathsf{dom}(\mathcal{I}) = \mathsf{dom}(\mathcal{I}^{\mathtt{U}:=A})$, thus Item (1) is satisfied. Note that Item (2) is also satisfied, as the only atoms that were changed are unary. Finally, Item (3) is trivially satisfied as well.

For the case of w being cliquewidth, let $\mathcal{T}$ be a $(\mathbb{L}, \mathrm{Cnst}, \Sigma)$-well-decorated tree witnessing that $\mathcal{I}$ has finite cliquewidth. We define $\mathcal{T}^{\mathtt{U}}$ as a result of the following transformation of $\mathcal{T}$.

- Let f be the isomorphism from a colored instance represented by $\mathcal{T}$ to $\mathcal{I}$.
- For every $\kappa \in \mathbb{L}$ and any node s decorated with $\mathtt{Add}_{\mathtt{U},\kappa}$ substitute the subtree of $\mathcal{T}$ that is rooted in s with the subtree rooted in $s0$.
- For every node s of $\mathcal{T}$ decorated with $*_\kappa$ (for some $\kappa \in \mathbb{L}$) for which $\mathtt{U}(\mathsf{f}(s)) \in \mathcal{I}^{\mathtt{U}:=A}$ holds, replace the decorator of s with $\mathtt{Add}_{\mathtt{U},\kappa}$ and replace the $\mathtt{Void}$ decorator of $s0$ with $*_\kappa$.
- For every node s of $\mathcal{T}$ decorated with c_κ (for some $\mathsf{c} \in \mathrm{Cnst}$ and $\kappa \in \mathbb{L}$) for which $\mathtt{U}(\mathsf{c}) \in \mathcal{I}^{\mathtt{U}:=A}$ holds, replace the decorator of s with $\mathtt{Add}_{\mathtt{U},\kappa}$ and replace the $\mathtt{Void}$ decorator of $s0$ with c_κ.

Then $\mathcal{T}^{\mathtt{U}}$ represents a colored instance isomorphic to $\mathcal{I}^{\mathtt{U}:=A}$.

In case w is partitionwidth, let $\mathcal{T}$ be a $(\mathbb{L}, \mathrm{Cnst}, \Sigma)$-well-decorated tree witnessing that $\mathcal{I}$ has finite partitionwidth. By Corollary C.5 we assume w.l.o.g. that $\mathcal{T}$ is in normal form. The well-decorated tree $\mathcal{T}^{\mathtt{U}}$ obtained from the following transformation satisfies $\mathcal{I}^{\mathcal{T}^{\mathtt{U}}} \cong \mathcal{I}^{\mathtt{U}:=A}$. Let $\mathsf{iso}\colon \mathcal{I}^{\mathcal{T}} \to \mathcal{I}$ be an isomorphism. We define the following set:

$$N_{\mathrm{add}} = \{(s', i) \in (\{0,1\}^* \times \{0,1\}) \mid e \in A, s = \mathsf{orig}(\mathsf{iso}^{-1}(e)), i \in \{0,1\}, s = s'i\}.$$

We then let

$$\mathcal{T}^{\mathtt{U}} = \mathcal{T} \setminus \{\mathtt{U\text{-}Add}^i_\kappa(s) \mid s \in \{0,1\}^*, i \in \{0,1\}, \kappa \in \mathbb{L}\} \cup \{\mathtt{U\text{-}Add}^i_{o_1}(s') \mid (s', i) \in N_{\mathrm{add}}\}.$$

By construction and $\mathcal{T}$ being in normal form, we get $\mathcal{I}^{\mathcal{T}^{\mathtt{U}}} \cong \mathcal{I}^{\mathtt{U}:=A}$ witnessed by iso. □

C.4. **Proofs for Section 6.4.**

Theorem 6.24. *Let $\mathcal{I}$ be a countable instance over a finite signature of arbitrary arity. Then the following hold:*

- *If $\mathcal{I}$ has finite treewidth, then $\mathcal{I}$ has finite partitionwidth.*
- *If $\mathcal{I}$ has finite cliquewidth, then $\mathcal{I}$ has finite partitionwidth.*

We split the proof of the theorem into two propositions.

Proposition C.7. *Let $\mathcal{I}$ be a countable instance over a signature Σ. If $\mathcal{I}$ has finite treewidth, then $\mathcal{I}$ has finite partitionwidth.*

Proof. Courcelle showed [Cou89, Theorem 8.1] that, given an MSO-sentence Ψ talking about hypergraphs of treewidth n, the set of terms representing hypergraphs that satsify Ψ, is MSO-definable. In the course of this proof, he exhibits an MSO-definition that can be used in the follwing way for our proof: Given an instance $\mathcal{I}$ of finite treewidth, first rewrite it as a hypergraph $\mathcal{G}_{\mathcal{I}}$. Then there is a (term-)tree $\mathcal{T}$ representing $\mathcal{G}_{\mathcal{I}}$, meaning that the (possibly infinite) term evaluates to $\mathcal{G}_{\mathcal{I}}$, $\mathsf{val}(\mathcal{T}) = \mathcal{G}_{\mathcal{I}}$. We can then produce an MSO-definition of $\mathsf{val}(\mathcal{T})$ in $\mathcal{T}$. Since $\mathcal{T}$ is a tree, we obtain that $\mathcal{G}_{\mathcal{I}}$ is MSO-tree-definable, hence MSO-tree-interpretable. By invoking Theorem 6.18 and translating our previous result to $\mathcal{I}$, we obtain that $\mathcal{I}$ has finite partitionwidth, concluding the proof. □

In order to disambiguate between well-decorated trees in the partitionwidth sense (as in Definition 6.7) and those in the cliquewidth sense [FLOR23, Definition 6], let for a set of unary colors $\mathbb{L}$, $\mathrm{Dec}_{\mathrm{cw}}(\mathbb{L}, \mathrm{Cnst}, \Sigma)$ denote the set of decorators in the latter sense.

Proposition C.8. *Let $\mathcal{I}$ be a countable instance over a signature Σ. If $\mathcal{I}$ has finite cliquewidth, then $\mathcal{I}$ has finite partitionwidth.*

Proof. Let $\mathcal{I}$ be an instance of finite cliquewidth over signature Σ and set of constants Const. Then there is a finite set of (unary) colors $\mathbb{L}$ and a $(\mathbb{L}, \mathrm{Cnst}, \Sigma)$-well-decorated tree $\mathcal{T}$ in the cliquewidth sense [FLOR23, Definition 6] such that $\mathcal{I} = \mathcal{I}^{\mathcal{T}}$ in the cliquewidth sense [FLOR23, Definition 7]. We recall that for the notion of cliquewidth [FLOR23, Definition 8] an MSO-interpretation of instances in well-decorated trees (in the cliquewidth sense [FLOR23, Definition 6]) has been exhibited. Using the results obtained there, let $\varphi_\top(x_1) = \bigvee \big(\mathtt{d}_k(x_1) \mid \mathtt{d} \in \mathrm{Const} \cup \{*\}, k \in \mathbb{L}\big)$; moreover, let $\varphi_{\mathtt{c}}(x_1) = \mathtt{c}_k(x_1)$ for all $\mathtt{c} \in \mathrm{Cnst}, k \in \mathbb{L}$; and let $\varphi_=(x_1, x_2) = (x_1 = x_2)$. For every $\mathtt{R} \in \Sigma$ we define a formula $\varphi_{\mathtt{R}}(x_1, \ldots, x_{\mathsf{ar}(\mathtt{R})})$ as follows: Let

$$\varphi_{\mathsf{up}}(x, y) = \mathtt{Succ}_0(y, x) \vee \mathtt{Succ}_1(y, x)$$

and

$$\varphi_k^{\mathrm{colIn}}(x, y) = \varphi_\top(x) \wedge \forall_{\ell \in \mathbb{L}} X_\ell . \Big(\big(\bigwedge_{\psi \in M_{xy}} \psi \big) \to X_k(y) \Big),$$

where X_ℓ are MSO set variables and M_{xy} contains the following formulae (for all respective elements of $\mathrm{Dec}_{\mathrm{cw}}(\mathbb{L}, \mathrm{Cnst}, \Sigma)$):

$$\begin{aligned}
\mathtt{c}_k(x) &\to X_k(x) \\
\forall x'y' . X_k(x') \wedge \varphi_{\mathsf{up}}(x', y') \wedge \mathtt{Add}_{\mathtt{R},k',k''}(y') &\to X_k(y') \\
\forall x'y' . X_k(x') \wedge \varphi_{\mathsf{up}}(x', y') \wedge \mathtt{Add}_{\mathtt{R},k'}(y') &\to X_k(y') \\
\forall x'y' . X_k(x') \wedge \varphi_{\mathsf{up}}(x', y') \wedge \mathtt{Recolor}_{k \to k'}(y') &\to X_{k'}(y') \\
\forall x'y' . X_k(x') \wedge \varphi_{\mathsf{up}}(x', y') \wedge \mathtt{Recolor}_{k' \to k''}(y') &\to X_k(y') \quad \text{for } k \neq k' \\
\forall x'y' . X_k(x') \wedge \varphi_{\mathsf{up}}(x', y') \wedge \oplus(y') &\to X_k(y')
\end{aligned}$$

For each predicate $\mathtt{R} \in \Sigma$ of arity n, we define

$$\varphi_{\mathtt{R}}(x_1, \ldots, x_n) = \exists z . \bigvee_{(k_1, \ldots, k_n) \in \mathbb{L}^n} \Big(\mathtt{Add}_{\mathtt{R},k_1,\ldots,k_n}(z) \wedge \bigwedge_{1 \leq i \leq n} \varphi_{k_i}^{\mathrm{colIn}}(x_i, z) \Big)$$

Then $\mathfrak{I}$ is the sequence of the MSO-formulae: $\varphi_{\mathtt{R}}(x_1, \ldots, x_{\mathsf{ar}(\mathtt{R})})$ for every $\mathtt{R} \in \Sigma \cup \{\top\}$; $\varphi_{\mathtt{c}}(x_1)$ for every $\mathtt{c} \in \mathrm{Const}$; and $\varphi_=(x_1, x_2)$ as described above. Hence $\mathcal{I} \leq_{\mathrm{MSO}} \mathcal{T}$. Invoking Theorem 6.18 we obtain that $\mathcal{I}$ has finite partitionwidth. □

Proposition 6.25. *Let $\mathcal{I}$ be a countable instance over a binary signature Σ. $\mathcal{I}$ has finite cliquewidth iff $\mathcal{I}$ has finite partitionwidth.*

Before proving Proposition 6.25, we introduce some technical notions.

Definition C.9. Let $\mathcal{T}$ be a $(\mathbb{L}, \mathrm{Cnst}, \Sigma)$-well-decorated tree and $i \in \{0, 1\}$. Let s be a node of $\mathcal{T}$ that is an ancestor of a pseudoleaf. We call s *recolor-cyclical for* i, if there is a $n \geq 2$ and a sequence of colors $\kappa_1, \ldots, \kappa_n \in \mathbb{L}$ of the same arity, such that $\kappa_2\text{-}\mathtt{ReCol}^i_{\kappa_1}(s), \ldots, \kappa_n\text{-}\mathtt{ReCol}^i_{\kappa_{n-1}}(s), \kappa_1\text{-}\mathtt{ReCol}^i_{\kappa_n}(s) \in \mathcal{T}$. In this case we call $(i, \kappa_1, \ldots, \kappa_n)$ a *recolor cycle at s for i*. We call s *recolor-cyclical* if it is recolor-cyclical for some $i \in \{0, 1\}$. If s is not recolor-cyclical, we also call s *non-recolor-cyclical*. We say an instruction $\lambda\text{-}\mathtt{ReCol}^i_\kappa$ *participates in a recolor-cycle at s*, if there is some recolor-cycle $(i, \kappa_1, \ldots, \kappa_m)$ at s and either some $1 \leq j \leq m-1$ such that $\kappa = \kappa_j$ and $\lambda = \kappa_{j+1}$ or $\kappa = \kappa_m$ and $\lambda = \kappa_1$. ◇

Definition C.10. Let Σ be a binary signature. Given a $(\mathbb{L}, \mathrm{Cnst}, \Sigma)$-well-decorated tree $\mathcal{T}$ (in the partitionwidth-sense, Definition 6.7) in normal form, we associate to it a $(\mathbb{L}_{\mathrm{cw}}, \mathrm{Cnst}, \Sigma)$-well-decorated tree (in the cliquewidth sense [FLOR23, Definition 6]) inductively as detailed in the following.

First define the set of (unary) colors $\mathbb{L}_{\mathrm{cw}} = \mathbb{L}_1 \uplus (\mathbb{L}_1 \times \{0,1\}) \uplus \{\mu_{\mathrm{cb}}\}$, where $\mathbb{L}_1 \subseteq \mathbb{L}$ is the subset of all unary colors of $\mathbb{L}$ and μ_{cb} is a still unused color. The following auxiliary maps will be defined in an interleaving top-down way along the construction (where P denotes the pseudoleafs of $\mathcal{T}$ and A_P all proper ancestors of pseudoleafs): $\mathsf{r}^{\mathcal{T}} \colon P \uplus A_P \to \{0,1\}^*$, $\mathsf{suc}_0^{\mathcal{T}} \colon A_P \to \{0,1\}^*$, $\mathsf{suc}_1^{\mathcal{T}} \colon A_P \to \{0,1\}^*$, and a map associating to every $s \in P \uplus A_P$ an instance $\mathcal{T}^s_{\mathrm{cw}}$. Let $s \in P \uplus A_P$ and $\mathsf{r}^{\mathcal{T}}, \mathsf{suc}_0^{\mathcal{T}}, \mathsf{suc}_1^{\mathcal{T}}$ be defined for every proper ancestor s' of s but not for s. If $s = \varepsilon$, initialize $\mathsf{r}^{\mathcal{T}}(\varepsilon) = \varepsilon$. Otherwise define $\mathsf{r}^{\mathcal{T}}(s) = \mathsf{suc}_i^{\mathcal{T}}(s')$ where $s'i = s$, i.e. s' is the unique predecessor of s. We distinguish between two cases:

- *s is a pseudoleaf.* Then set $\mathcal{T}^s_{\mathrm{cw}} = \{\mathtt{d}_{o_1}(\mathsf{r}^{\mathcal{T}}(s)) \mid \mathtt{New}_{\mathtt{d}}(s) \in \mathcal{T}, \mathtt{d} \in \mathrm{Cnst} \cup \{*\}\}$.
- *s is not a pseudoleaf.* Then define with $i \in \{0,1\}$ the sets

$$\mathsf{PRec}^i_s = \{\lambda\text{-}\mathtt{ReCol}^i_\kappa \mid \kappa \in \mathbb{L}_1, \lambda\text{-}\mathtt{ReCol}^i_\kappa(s) \in \mathcal{T}\}, \qquad \mathsf{URel}^i_s = \{\mathtt{R}\text{-}\mathtt{Add}^i_\lambda \mid \mathtt{R} \in \Sigma\}, \text{ and}$$
$$\mathsf{Rel}_s = \{\mathtt{R}\text{-}\mathtt{Add}^I_{\kappa_1\kappa_2} \mid \mathtt{R} \in \Sigma, \mathsf{ar}(\mathtt{R}) = 2, \mathtt{R}\text{-}\mathtt{Add}^I_{\kappa_1\kappa_2}(s) \in \mathcal{T}\}.$$

Define C^i_s to be the set of all recolor cycles at s for $i \in \{0,1\}$. Set $\mathsf{Rec}^i_s = \mathsf{PRec}^i_s \cup \mathsf{C}^i_s$. Let $k_s = |\mathsf{Rel}_s|$, $k^i_s = |\mathsf{URel}^i_s|$, $\ell^i_s = |\mathsf{Rec}^i_s|$, and let $\mathsf{f}_s \colon \mathsf{Rel}_s \to \{0, \dots, k_s - 1\}$ and $\mathsf{f}^i_s \colon \mathsf{URel}^i_s \to \{0, \dots, k^i_s - 1\}$ be bijections. Additionally let $\mathsf{g}^i_s \colon \mathsf{Rec}^i_s \to \{0, \dots, \ell^i_s - 1\}$ be bijections with the following properties: For every $\mathrm{cyc} \in \mathsf{C}^i_s$ where $\mathrm{cyc} = (i, \kappa_1, \dots, \kappa_m)$:

- $\mathsf{g}^i_s(\mathrm{cyc}) = \mathsf{g}^i_s(\kappa_1\text{-}\mathtt{ReCol}_{\kappa_m}) + 1$ and $\mathsf{g}^i_s(\kappa_1\text{-}\mathtt{ReCol}_{\kappa_m}) = \mathsf{g}^i_s(\kappa_m\text{-}\mathtt{ReCol}_{\kappa_{m-1}}) + 1$,
- for every $1 \leq k \leq m-2$ we have $\mathsf{g}^i_s(\kappa_{k+2}\text{-}\mathtt{ReCol}_{\kappa_{k+1}}) = \mathsf{g}^i_s(\kappa_{k+1}\text{-}\mathtt{ReCol}_{\kappa_k}) + 1$.

Then we define $\mathsf{suc}_0^{\mathcal{T}}(s)$ and $\mathsf{suc}_1^{\mathcal{T}}(s)$ by letting $\mathsf{suc}_0^{\mathcal{T}}(s) = \mathsf{r}^{\mathcal{T}}(s)0^{2|\mathbb{L}_1| + k_s + k^0_s + k^1_s + \ell^0_s + \ell^1_s + 1 + |\mathbb{L}_1|}$ and $\mathsf{suc}_1^{\mathcal{T}}(s) = \mathsf{r}^{\mathcal{T}}(s)0^{2|\mathbb{L}_1| + k_s + k^0_s + k^1_s + \ell^0_s + \ell^1_s}10^{|\mathbb{L}_1|}$. Then $\mathcal{T}^s_{\mathrm{cw}}$ is the union of the following instances over $\mathrm{Dec}_{\mathrm{cw}}(\mathbb{L}^*, \mathrm{Cnst}, \Sigma)$:

$$\mathcal{T}^s_{\mathrm{pre},0} = \{\mathtt{Recolor}_{\kappa\to(\kappa,0)}(\mathsf{r}^{\mathcal{T}}(s)0^{2|\mathbb{L}_1| + k_s + k^0_s + k^1_s + \ell^0_s + \ell^1_s + 1 + j}) \mid \kappa \in \mathbb{L}_1, \mathsf{coen}(\kappa) = j\}$$
$$\mathcal{T}^s_{\mathrm{pre},1} = \{\mathtt{Recolor}_{\kappa\to(\kappa,1)}(\mathsf{r}^{\mathcal{T}}(s)0^{2|\mathbb{L}_1| + k_s + k^0_s + k^1_s + \ell^0_s + \ell^1_s}10^j) \mid \kappa \in \mathbb{L}_1, \mathsf{coen}(\kappa) = j\}$$
$$\mathcal{T}^s_{\mathrm{un}} = \{\oplus(\mathsf{r}^{\mathcal{T}}(s)0^{2|\mathbb{L}_1| + k_s + k^0_s + k^1_s + \ell^0_s + \ell^1_s})\}$$
$$\mathcal{T}^s_{\mathtt{Add}} = \{\mathtt{Add}_{\mathtt{R},(\kappa_1,i_1),(\kappa_2,i_2)}(\mathsf{r}^{\mathcal{T}}(s)0^{2|\mathbb{L}_1| + k^0_s + k^1_s + \ell^0_s + \ell^1_s + j}) \mid \mathtt{R}\text{-}\mathtt{Add}^I_{\kappa_1\kappa_2} \in \mathsf{Rel}_s, I = \{i_1 + 1\}, i_1 + i_2 = 1, \mathsf{f}_s(\mathtt{R}\text{-}\mathtt{Add}^I_{\kappa_1\kappa_2}) = j\}$$
$$\mathcal{T}^s_{\mathtt{UUAdd},0} = \{\mathtt{Add}_{\mathtt{R},(\kappa,0)}(\mathsf{r}^{\mathcal{T}}(s)0^{2|\mathbb{L}_1| + k^1_s + \ell^0_s + \ell^1_s + j}) \mid \mathsf{ar}(\mathtt{R}) = 1, \mathtt{R}\text{-}\mathtt{Add}^0_\kappa \in \mathsf{URel}^0_s, \mathsf{f}^0_s(\mathtt{R}\text{-}\mathtt{Add}^0_\kappa) = j\}$$
$$\mathcal{T}^s_{\mathtt{UBAdd},0} = \{\mathtt{Add}_{\mathtt{R},(\kappa,0),(\kappa,0)}(\mathsf{r}^{\mathcal{T}}(s)0^{2|\mathbb{L}_1| + k^1_s + \ell^0_s + \ell^1_s + j}) \mid \mathsf{ar}(\mathtt{R}) = 2, \mathtt{R}\text{-}\mathtt{Add}^0_\kappa \in \mathsf{URel}^0_s, \mathsf{f}^0_s(\mathtt{R}\text{-}\mathtt{Add}^0_\kappa) = j\}$$
$$\mathcal{T}^s_{\mathtt{UUAdd},1} = \{\mathtt{Add}_{\mathtt{R},(\kappa,1)}(\mathsf{r}^{\mathcal{T}}(s)0^{2|\mathbb{L}_1| + \ell^0_s + \ell^1_s + j}) \mid \mathsf{ar}(\mathtt{R}) = 1, \mathtt{R}\text{-}\mathtt{Add}^1_\kappa \in \mathsf{URel}^1_s, \mathsf{f}^1_s(\mathtt{R}\text{-}\mathtt{Add}^1_\kappa) = j\}$$
$$\mathcal{T}^s_{\mathtt{UBAdd},1} = \{\mathtt{Add}_{\mathtt{R},(\kappa,1),(\kappa,1)}(\mathsf{r}^{\mathcal{T}}(s)0^{2|\mathbb{L}_1| + \ell^0_s + \ell^1_s + j}) \mid \mathsf{ar}(\mathtt{R}) = 2, \mathtt{R}\text{-}\mathtt{Add}^1_\kappa \in \mathsf{URel}^1_s, \mathsf{f}^1_s(\mathtt{R}\text{-}\mathtt{Add}^1_\kappa) = j\}$$

$$\begin{aligned}
\mathcal{T}^s_{\texttt{Rec},0} &= \{\texttt{Recolor}_{(\kappa,0)\to(\lambda,0)}(\mathsf{r}^{\mathcal{T}}(s)0^{2|\mathbb{L}_1|+\ell^1_s+j}) && \mid \lambda\text{-}\texttt{ReCol}^0_\kappa \in \mathsf{Rec}^0_s, \mathsf{g}^0_s(\lambda\text{-}\texttt{ReCol}^0_\kappa) = j \\
&&& \quad \forall \mathrm{cyc} \in \mathsf{Rec}^0_s.\mathrm{cyc} = (0,\kappa_1,\dots,\kappa_m) \to \kappa \neq \kappa_1\} \\
\mathcal{T}^s_{\texttt{Rec},1} &= \{\texttt{Recolor}_{(\kappa,1)\to(\lambda,1)}(\mathsf{r}^{\mathcal{T}}(s)0^{2|\mathbb{L}_1|+j}) && \mid \lambda\text{-}\texttt{ReCol}^1_\kappa \in \mathsf{Rec}^1_s, \mathsf{g}^1_s(\lambda\text{-}\texttt{ReCol}^1_\kappa) = j, \\
&&& \quad \forall \mathrm{cyc} \in \mathsf{Rec}^1_s.\mathrm{cyc} = (1,\kappa_1,\dots,\kappa_m) \to \kappa \neq \kappa_1\} \\
\mathcal{T}^s_{\texttt{CRecOp},0} &= \{\texttt{Recolor}_{(\kappa_1,0)\to\mu_{\mathrm{cb}}}(\mathsf{r}^{\mathcal{T}}(s)0^{2|\mathbb{L}_1|+\ell^1_s+j}) && \mid \mathrm{cyc} \in \mathsf{Rec}^0_s, \mathsf{g}^0_s(\mathrm{cyc}) = j, \mathrm{cyc} = (0,\kappa_1,\dots,\kappa_m)\} \\
\mathcal{T}^s_{\texttt{CRecClo},0} &= \{\texttt{Recolor}_{\mu_{\mathrm{cb}}\to(\kappa_2,0)}(\mathsf{r}^{\mathcal{T}}(s)0^{2|\mathbb{L}_1|+\ell^1_s+j}) && \mid (0,\kappa_1,\dots,\kappa_m) \in \mathsf{Rec}^0_s, \mathsf{g}^0_s(\kappa_2\text{-}\texttt{ReCol}^0_{\kappa_1}) = j\} \\
\mathcal{T}^s_{\texttt{CRecOp},1} &= \{\texttt{Recolor}_{(\kappa_1,1)\to\mu_{\mathrm{cb}}}(\mathsf{r}^{\mathcal{T}}(s)0^{2|\mathbb{L}_1|+j}) && \mid \mathrm{cyc} \in \mathsf{Rec}^1_s, \mathsf{g}^1_s(\mathrm{cyc}) = j, \mathrm{cyc} = (1,\kappa_1,\dots,\kappa_m)\} \\
\mathcal{T}^s_{\texttt{CRecClo},1} &= \{\texttt{Recolor}_{\mu_{\mathrm{cb}}\to(\kappa_2,1)}(\mathsf{r}^{\mathcal{T}}(s)0^{2|\mathbb{L}_1|+j}) && \mid (1,\kappa_1,\dots,\kappa_m) \in \mathsf{Rec}^1_s, \mathsf{g}^1_s(\kappa_2\text{-}\texttt{ReCol}^1_{\kappa_1}) = j\} \\
\mathcal{T}^s_{\mathrm{pos},0} &= \{\texttt{Recolor}_{(\kappa,0)\to\kappa}(\mathsf{r}^{\mathcal{T}}(s)0^{|\mathbb{L}_1|+j}) && \mid \kappa \in \mathbb{L}_1, \mathsf{coen}(\kappa) = j\} \\
\mathcal{T}^s_{\mathrm{pos},1} &= \{\texttt{Recolor}_{(\kappa,1)\to\kappa}(\mathsf{r}^{\mathcal{T}}(s)0^{j}) && \mid \kappa \in \mathbb{L}_1, \mathsf{coen}(\kappa) = j\}.
\end{aligned}$$

Thus we obtain a map that maps every $s \in P \oplus A_P$ to an instance $\mathcal{T}^s_{\mathrm{cw}}$. We define $\mathcal{T}^{\mathrm{aux}}_{\mathrm{cw}} = \bigcup_{s \in P \uplus A_P} \mathcal{T}^s_{\mathrm{cw}}$. Finally,

$$\mathcal{T}_{\mathrm{cw}} = \mathcal{T}_{\mathrm{bin}} \cup \mathcal{T}^{\mathrm{aux}}_{\mathrm{cw}} \cup \{\texttt{Void}(s) \mid \forall \texttt{Dec} \in \mathrm{Dec}_{\mathrm{cw}}(\mathbb{L}^*, \mathrm{Cnst}, \Sigma).\texttt{Dec}(s) \notin \mathcal{T}^{\mathrm{aux}}_{\mathrm{cw}}\}.$$

Additionally, we define a map $\mathsf{h}^{\mathcal{T}} \colon \mathsf{dom}(\mathcal{I}^{\mathcal{T}}) \to \mathsf{dom}(\mathcal{I}^{\mathcal{T}_{\mathrm{cw}}})$ as follows:

$$\mathsf{h}^{\mathcal{T}}(e) = \begin{cases} \mathsf{c}, & \text{if } e = \mathsf{c} \text{ for some } \mathsf{c} \in \mathrm{Const}, \\ \mathsf{r}^{\mathcal{T}}(e), & \text{otherwise} \end{cases}$$

where $\mathcal{I}^{\mathcal{T}_{\mathrm{cw}}}$ is interpreted in the cliquewidth sense [FLOR23, Definition 7]. ◇

Observation C.11. *We observe the following:*

(1) $\mathcal{T}_{\mathrm{cw}}$ *is a* $(\mathbb{L}_{\mathrm{cw}}, \mathrm{Const}, \Sigma)$*-well-decorated tree (in the cliquewidth sense* [FLOR23, Definition 6]*).*

(2) By construction, $\mathsf{h}^{\mathcal{T}}$ *is a bijection between the terms of* $\mathcal{I}^{\mathcal{T}}$ *and* $\mathcal{I}^{\mathcal{T}_{\mathrm{cw}}}$*.*

Lemma C.12. *Let* Σ *be a binary signature and* $\mathcal{T}$ *a* $(\mathbb{L}, \mathrm{Cnst}, \Sigma)$*-well-decorated tree. Then for every node* s *of* $\mathcal{T}$ *that is a pseudoleaf or an ancestor of a pseudoleaf, every* $e \in \mathsf{ent}^{\mathcal{T}}(s)$*, and every* $\kappa \in \mathbb{L}_1$ *the following are equivalent:*

(1) $e \in \mathsf{col}^{\mathcal{T}}_s(\kappa)$

(2) $\mathsf{col}^{\mathcal{T}_{\mathrm{cw}}}_{\mathsf{r}^{\mathcal{T}}(s)}(\mathsf{h}^{\mathcal{T}}(e)) = \kappa$ *(in the cliquewidth sense* [FLOR23, Definition 7]*).*

Proof. Simple induction over the distance of s from $s_e = \mathsf{orig}(e)$.

- $s = s_e$. Then $e \in \mathsf{col}^{\mathcal{T}}_s(\kappa)$ *iff* $\kappa = o_1$. This is equivalent to $\mathsf{c}(s) \in \mathcal{T}$ holding for some $\mathsf{c} \in \mathrm{Cnst} \cup \{*\}$. This is equivalent by construction to $\mathsf{c}_{o_1}(\mathsf{r}^{\mathcal{T}}(s)) \in \mathcal{T}_{\mathrm{cw}}$. Which is equivalent to $\mathsf{col}^{\mathcal{T}_{\mathrm{cw}}}_{\mathsf{r}^{\mathcal{T}}(s)}(\mathsf{h}^{\mathcal{T}}(e)) = \kappa$.
- s is a proper ancestor of s_e and the assertion holds for every node s^* being an ancestor of s_e such that s' is a proper ancestor of s^*. Consequently also for the unique s' such that $s' = si$ and s' is an ancestor of s_e. Assume $e \in \mathsf{col}^{\mathcal{T}}_s(\kappa)$.We have two cases to consider:
  - $e \in \mathsf{col}^{\mathcal{T}}_{si}(\kappa)$. Then $\lambda\text{-}\texttt{ReCol}^i_\kappa(s) \notin \mathcal{T}$ for all $\lambda \in \mathbb{L}_1$, $\lambda \neq \kappa$. Hence for all nodes v of $\mathcal{T}_{cw}$ that are proper ancestors of $\mathsf{r}^{\mathcal{T}}(si)$ and for whom $\mathsf{r}^{\mathcal{T}}(s)$ is an ancestor, $\texttt{Recolor}_{(\kappa,i)\to(\lambda,i)}(v) \notin \mathcal{T}_{\mathrm{cw}}$ for all $\lambda \neq \kappa$. As $\mathsf{col}^{\mathcal{T}_{\mathrm{cw}}}_{\mathsf{r}^{\mathcal{T}}(si)}(\mathsf{h}^{\mathcal{T}}(e)) = \kappa$ by assumption, we conclude $\mathsf{col}^{\mathcal{T}_{\mathrm{cw}}}_{\mathsf{r}^{\mathcal{T}}(s)}(\mathsf{h}^{\mathcal{T}}(e)) = \kappa$. The other direction is completely analogous.
  - $e \in \mathsf{col}^{\mathcal{T}}_{si}(\lambda)$ for $\lambda \in \mathbb{L}_1$, $\lambda \neq \kappa$. Then $\kappa\text{-}\texttt{ReCol}^i_\lambda(s) \in \mathcal{T}$. By construction there is a node v of $\mathcal{T}_{cw}$ that is a proper ancestors of $\mathsf{r}^{\mathcal{T}}(si)$ and for whom $\mathsf{r}^{\mathcal{T}}(s)$ is an ancestor, such that $\texttt{Recolor}_{(\lambda,i)\to(\kappa,i)}(v) \in \mathcal{T}_{\mathrm{cw}}$. As $\mathsf{col}^{\mathcal{T}_{\mathrm{cw}}}_{\mathsf{r}^{\mathcal{T}}(si)}(\mathsf{h}^{\mathcal{T}}(e)) = \lambda$ by assumption, we conclude $\mathsf{col}^{\mathcal{T}_{\mathrm{cw}}}_{\mathsf{r}^{\mathcal{T}}(s)}(\mathsf{h}^{\mathcal{T}}(e)) = \kappa$. The other direction is completely analogous. □

Proposition 6.25. *Let $\mathcal{I}$ be a countable instance over a binary signature Σ. $\mathcal{I}$ has finite cliquewidth iff $\mathcal{I}$ has finite partitionwidth.*

Proof. By Proposition C.8 it suffices to prove that, given an instance $\mathcal{I}$ over a binary signature Σ with finite partitionwidth, $\mathcal{I}$ also has finite cliquewidth. So let Σ be a finite binary signature, $\mathbb{L}$ finite set of colors (of arity ≤ 2) and Cnst a finite set of constants. Assume $\mathcal{T}$ to be a $(\mathbb{L}, \text{Cnst}, \Sigma)$-well-decorated tree (in the partitionwidth sense, Definition 6.7) in normal form such that $\mathcal{I} = \mathcal{I}^{\mathcal{T}}$. This can be achieved by renaming (if necessary) of the nulls in $\mathcal{I}$. Let $\mathbb{L}^* = \mathbb{L}_1 \uplus (\mathbb{L}_1 \times \{0,1\}) \uplus \{\mu_{\text{cb}}\}$ and $\mathcal{T}_{\text{cw}}$ as in Definition C.10. If we show $\mathsf{h}^{\mathcal{T}} \colon \mathcal{I}^{\mathcal{T}} \to \mathcal{I}^{\mathcal{T}_{\text{cw}}}$ to be an isomorphism we are done. Since by Observation C.11, Item 1 we know $\mathsf{h}^{\mathcal{T}}$ to be bijective, we only need to prove it to be structure preserving in both ways. In the following, we use all the defined auxiliary maps, sets and instances as denoted in the construction in Definition C.10.

Let $\mathtt{R}(\boldsymbol{e}) \in \mathcal{I}$. Note that this automatically implies, that ε is not a pseudoleaf (in $\mathcal{T}$). We consider two cases:

- $\boldsymbol{e} = (e, e)$ or $\boldsymbol{e} = e$ for some entity e. Then let $s_e = \mathsf{orig}(e)$, $s'_e \in \{0,1\}^*$, and $i_e \in \{0,1\}$ such that $s_e = s'_e i_e$. Since $\mathcal{T}$ is in normal form, we conclude that $\mathtt{R\text{-}Add}^{i_e}_{o_{\mathsf{ar}(\mathtt{R})}}(s'_e) \in \mathcal{T}$. As s_e is a pseudoleaf by definition, there is a unique $\mathsf{c} \in \text{Cnst} \uplus \{*\}$ such that $\mathsf{c}(s) \in \mathcal{T}$. By construction, $\mathsf{c}_{o_1}(\mathsf{r}^{\mathcal{T}}(s)) \in \mathcal{T}_{\text{cw}}$. Additionally, $\mathtt{Recolor}_{o_1 \to (o_1, i_e)}(v) \in \mathcal{T}_{\text{cw}}^{s'_e}$ for some node $v \in \{0,1\}^*$ of $\mathcal{T}_{\text{cw}}$ such that
  - $\mathsf{r}^{\mathcal{T}}(s'_e)$ is a proper ancestor of v, and
  - v is a proper ancestor of $\mathsf{r}^{\mathcal{T}}(s)$.

  Finally, since $\mathtt{R\text{-}Add}^{i_e}_{o_{\mathsf{ar}(\mathtt{R})}}(s'_e) \in \mathcal{T}$ there is another node v' satisfying both
  - $\mathsf{r}^{\mathcal{T}}(s'_e)$ is a proper ancestor of v', and
  - v' is a proper ancestor of v,

  such that either $\mathtt{Add}_{\mathtt{R},(o_1,i_e),(o_1,i_e)}(v') \in \mathcal{T}_{\text{cw}}$ or $\mathtt{Add}_{\mathtt{R},(o_1,i_e)}(v') \in \mathcal{T}_{\text{cw}}$ depending on whether $\mathsf{ar}(\mathtt{R}) = 2$ or $\mathsf{ar}(\mathtt{R}) = 1$. By construction of $\mathcal{T}_{\text{cw}}$ and $\text{Atoms}_{v'}$ [FLOR23, Definition 7], we obtain $\mathtt{R}(\mathsf{h}^{\mathcal{T}}(\boldsymbol{e})) \in \text{Atoms}_{v'}$ and hence $\mathtt{R}(\mathsf{h}^{\mathcal{T}}(\boldsymbol{e})) \in \mathcal{I}^{\mathcal{T}_{\text{cw}}}$.
- $\boldsymbol{e} \neq (e, \ldots, e)$ for every entity e. This implies immediately that $\mathsf{ar}(\mathtt{R}) = 2$. Hence $\boldsymbol{e} = (e_1, e_2)$ with $e_1 \neq e_2$. Furthermore, as $\mathcal{T}$ is in normal form, for $s_{\boldsymbol{e}} = \mathsf{cca}(\mathsf{orig}(\boldsymbol{e}))$ there are some $\kappa_1, \kappa_2 \in \mathbb{L}$ with $\mathsf{ar}(\kappa_1) = \mathsf{ar}(\kappa_2) = 1$ and $I = i^*$ with $i^* \in \{1,2\}$ such that
  - $\mathtt{R\text{-}Add}^{\{i^*\}}_{\kappa_1 \kappa_2}(s_{\boldsymbol{e}}) \in \mathcal{T}$,
  - $e_{i^*} \in \mathsf{col}^{\mathcal{T}}_{s_{\boldsymbol{e}}0}(\kappa_1)$, and
  - $e_j \in \mathsf{col}^{\mathcal{T}}_{s_{\boldsymbol{e}}1}(\kappa_2)$ such that $\{i^*, j\} = \{1,2\}$.

  By construction, there are nodes $v, v_0, v_1 \in \{0,1\}^*$ from $\mathcal{T}_{\text{cw}}$ such that
  - v is a proper ancestor to v_0 and v_1,
  - $\mathsf{r}^{\mathcal{T}}(s)$ is a proper ancestor of v,
  - v_0 is a proper ancestor $\mathsf{r}^{\mathcal{T}}(s0)$, v_1 is a proper ancestor of $\mathsf{r}^{\mathcal{T}}(s1)$,
  - if $i^* = 1$, $\mathtt{Add}_{\mathtt{R},(\kappa_1,0),(\kappa_2,1)}(v) \in \mathcal{T}_{\text{cw}}$, if $i^* = 2$, $\mathtt{Add}_{\mathtt{R},(\kappa_2,1),(\kappa_1,0)}(v) \in \mathcal{T}_{\text{cw}}$, and
  - $\mathtt{Recolor}_{\kappa_1 \to (\kappa_1,0)}(v_0), \mathtt{Recolor}_{\kappa_2 \to (\kappa_2,1)}(v_1)$.

  Applying Lemma C.12 we obtain $\mathsf{col}^{\mathcal{T}_{\text{cw}}}_{\mathsf{r}^{\mathcal{T}}(s_{\boldsymbol{e}}0)}(e_{i^*}) = \kappa_1$ and $\mathsf{col}^{\mathcal{T}_{\text{cw}}}_{\mathsf{r}^{\mathcal{T}}(s_{\boldsymbol{e}}1)}(e_j) = \kappa_2$. Hence by construction $\mathsf{col}^{\mathcal{T}_{\text{cw}}}_{v0}(\mathsf{h}^{\mathcal{T}}(e_{i^*})) = (\kappa_1, 0)$ and $\mathsf{col}^{\mathcal{T}_{\text{cw}}}_{v0}(h(e_j)) = (\kappa_2, 1)$. Consequently $\mathtt{R}(\mathsf{h}^{\mathcal{T}}\boldsymbol{e}) \in \text{Atoms}_v$ and thus $\mathtt{R}(\mathsf{h}^{\mathcal{T}}(\boldsymbol{e})) \in \mathcal{I}^{\mathcal{T}_{\text{cw}}}$.

Now assume $\mathtt{R}(\mathsf{h}^{\mathcal{T}}(\boldsymbol{e})) \in \mathcal{I}^{\mathcal{T}_{\text{cw}}}$ for $\boldsymbol{e}$ from $\mathsf{dom}(\mathcal{I}^{\mathcal{T}})$. Note that this captures all the atoms in $\mathcal{I}^{\mathcal{T}_{\text{cw}}}$ as $\mathsf{h}^{\mathcal{T}}$ is a bijection on the sets of entities corresponding to the instances $\mathcal{I} = \mathcal{I}^{\mathcal{T}}$ and $\mathcal{I}^{\mathcal{T}_{\text{cw}}}$. We again consider two cases:

- $\boldsymbol{e} = (e, e)$ or $\boldsymbol{e} = e$ for some entity e of $\mathcal{I}$. Let $s = \mathsf{orig}(e)$ be the node of $\mathcal{T}$ introducing entity e. By construction there is a node v in $\mathcal{T}_{\text{cw}}$ such that $\mathsf{r}^{\mathcal{T}}(s) = v$ and there is a $\mathsf{c} \in \text{Cnst} \uplus \{*\}$ such that $\mathsf{c}(s) \in \mathcal{T}$ and $\mathsf{c}_{o_1}(v) \in \mathcal{T}_{\text{cw}}$. Let s' be the parent of s in $\mathcal{T}$ and $i \in \{0, 1\}$ such that $s = s'i$. Again by construction, there are nodes v_i, v' of $\mathcal{T}_{\text{cw}}$ such that
  - $\mathsf{r}^{\mathcal{T}}(s')$ is a proper ancestor of v', v_i, v,
  - v' is a proper ancestor of v_i and v,
  - v_i is a proper ancestor of v,
  - $\mathtt{Recolor}_{o_1 \to (o_1,i)}(v_i) \in \mathcal{T}_{\text{cw}}$,
  - $\mathtt{Add}_{\mathtt{R},(o_1,i),(o_1,i)}$ or $\mathtt{Add}_{\mathtt{R},(o_1,i)}$ depending on $\mathsf{ar}(\mathtt{R}) = 2$ or $\mathsf{ar}(\mathtt{R}) = 1$.

  By construction we may obtain $\mathtt{R\text{-}Add}^i_{o_2}(s') \in \mathcal{T}$ or $\mathtt{R\text{-}Add}^i_{o_1}(s') \in \mathcal{T}$ depending on $\mathsf{ar}(\mathtt{R}) = 2$ or $\mathsf{ar}(\mathtt{R}) = 1$. Consequently, $\mathtt{R}(\boldsymbol{e}) \in \mathcal{I}^{\mathcal{T}} = \mathcal{I}$.
- $\boldsymbol{e} \neq (e, \ldots, e)$ for every entity e. This implies immediately that $\mathsf{ar}(\mathtt{R}) = 2$. Hence $\boldsymbol{e} = (e_1, e_2)$ with $e_1 \neq e_2$. Let $s_{\boldsymbol{e}} = \mathsf{cca}(\mathsf{orig}(\boldsymbol{e}))$. Since $\mathtt{R}(\mathsf{h}^{\mathcal{T}}(\boldsymbol{e})) \in \mathcal{I}^{\mathcal{T}_{\text{cw}}}$ there is by construction a node v in $\mathcal{T}_{\text{cw}}$ such that $\mathtt{Add}_{\mathtt{R},(\kappa_1,i_1),(\kappa_2,i_2)}(v) \in \mathcal{T}_{\text{cw}}$ with $\{i_1, i_2\} = \{1, 2\}$, $\mathsf{col}^{\mathcal{T}_{\text{cw}}}_v(\mathsf{h}^{\mathcal{T}}(e_1)) = (\kappa_1, i_1)$ and $\mathsf{col}^{\mathcal{T}_{\text{cw}}}_v(\mathsf{h}^{\mathcal{T}}(e_2)) = (\kappa_2, i_2)$. Furthermore, v satisfies the following property: v is a proper ancestor to both $\mathsf{r}^{\mathcal{T}}(s0)$ and $\mathsf{r}^{\mathcal{T}}(s1)$ and $\mathsf{r}^{\mathcal{T}}(s)$ is a proper ancestor to v. Also, by construction we immediately obtain $\mathsf{col}^{\mathcal{T}_{\text{cw}}}_{\mathsf{r}^{\mathcal{T}}(s(i_1-1))}(\mathsf{h}^{\mathcal{T}}(e_1)) = \kappa_1$ and $\mathsf{col}^{\mathcal{T}_{\text{cw}}}_{\mathsf{r}^{\mathcal{T}}(s(i_2-1))}(\mathsf{h}^{\mathcal{T}}(e_2)) = \kappa_2$. Again by construction we now that $\mathtt{R\text{-}Add}^{\{i_1\}}_{\kappa_1\kappa_2}(s) \in \mathcal{T}$. Finally, applying Lemma C.12 we obtain $e_1 \in \mathsf{col}^{\mathcal{T}}_{s(i_1-1)}(\kappa_1)$ and $e_2 \in \mathsf{col}^{\mathcal{T}}_{s(i_2-1)}(\kappa_2)$. Together this implies $\mathtt{R}(e_1, e_2) \in \text{Atoms}_s$ and hence $\mathtt{R}(\boldsymbol{e}) \in \mathcal{I}^{\mathcal{T}} = \mathcal{I}$.

This proves that $\mathsf{h}^{\mathcal{T}}$ is an isomorphism between $\mathcal{I} = \mathcal{I}^{\mathcal{T}}$ and $\mathcal{I}^{\mathcal{T}_{\text{cw}}}$. As $|\mathbb{L}_{\text{cw}}| = 3 \cdot |\mathbb{L}_1| + 1 < 3 \cdot |\mathbb{L}| + 1$ we conclude that since $\mathcal{I}$ has finite partitionwidth, it has also finite cliquewidth. □

Appendix D. Proof for Section 9

Lemma 9.2. *Let $\mathcal{R}$ be a ruleset. If $\mathcal{R} = \mathcal{R}_1 \triangleright \cdots \triangleright \mathcal{R}_n$, then for any database $\mathcal{D}$, the following holds:*

$$\mathbf{Ch}_\infty(\mathcal{D}, \mathcal{R}) = \mathbf{Ch}_\infty(\ldots \mathbf{Ch}_\infty(\mathbf{Ch}_\infty(\mathcal{D}, \mathcal{R}_1), \mathcal{R}_2) \ldots, \mathcal{R}_n).$$

Proof. For $n = 1$ the statement is trivial. For all other n, we prove the result by induction.

Base case. Let $\mathcal{R} = \mathcal{R}_1 \triangleright \mathcal{R}_2$. We argue two cases: we first argue (i) that $\mathbf{Ch}_\infty(\mathcal{D}, \mathcal{R}) \subseteq \mathbf{Ch}_\infty(\mathbf{Ch}_\infty(\mathcal{D}, \mathcal{R}_1), \mathcal{R}_2)$, and then show (ii) that $\mathbf{Ch}_\infty(\mathbf{Ch}_\infty(\mathcal{D}, \mathcal{R}_1), \mathcal{R}_2) \subseteq \mathbf{Ch}_\infty(\mathcal{D}, \mathcal{R})$.

For case (i), let us suppose that $\mathtt{R}(\boldsymbol{t}) \in \mathbf{Ch}_\infty(\mathcal{D}, \mathcal{R})$. If $\mathtt{R}(\boldsymbol{t}) \in \mathcal{D}$, then $\mathtt{R}(\boldsymbol{t}) \in \mathbf{Ch}_\infty(\mathbf{Ch}_\infty(\mathcal{D}, \mathcal{R}_1), \mathcal{R}_2)$, so let us assume that there exists an i such that $\mathtt{R}(\boldsymbol{t}) \in \mathbf{Ch}_i(\mathcal{D}, \mathcal{R})$, but $\mathtt{R}(\boldsymbol{t}) \notin \mathbf{Ch}_j(\mathcal{D}, \mathcal{R})$ for $j < i$. It follows that there exists a trigger (ρ, h) of $\mathbf{Ch}_{i-1}(\mathcal{D}, \mathcal{R})$ such that $\mathtt{R}(\boldsymbol{t}) \in \overline{h}(\mathsf{head}(\rho))$. Either $\rho \in \mathcal{R}_1$ or $\rho \in \mathcal{R}_2$. If $\rho \in \mathcal{R}_1$, then since no rule of $\mathcal{R}_1$ depends on a rule in $\mathcal{R}_2$, it follows that $\mathtt{R}(\boldsymbol{t}) \in \mathbf{Ch}_\infty(\mathcal{D}, \mathcal{R}_1)$, meaning $\mathtt{R}(\boldsymbol{t}) \in \mathbf{Ch}_\infty(\mathbf{Ch}_\infty(\mathcal{D}, \mathcal{R}_1), \mathcal{R}_2)$. If $\rho \in \mathcal{R}_2$, then $\mathtt{R}(\boldsymbol{t}) \in \mathbf{Ch}_\infty(\mathbf{Ch}_\infty(\mathcal{D}, \mathcal{R}_1), \mathcal{R}_2)$, which completes the proof of case (i).

For case (ii), let us suppose that $\mathtt{R}(\boldsymbol{t}) \in \mathbf{Ch}_\infty(\mathbf{Ch}_\infty(\mathcal{D}, \mathcal{R}_1), \mathcal{R}_2)$. If $\mathtt{R}(\boldsymbol{t}) \in \mathcal{D}$, then $\mathtt{R}(\boldsymbol{t}) \in \mathbf{Ch}_\infty(\mathcal{D}, \mathcal{R})$, so let us suppose that $\mathtt{R}(\boldsymbol{t}) \notin \mathcal{D}$. Either $\mathtt{R}(\boldsymbol{t}) \in \mathbf{Ch}_\infty(\mathcal{D}, \mathcal{R}_1)$ or $\mathtt{R}(\boldsymbol{t}) \in \mathbf{Ch}_\infty(\mathbf{Ch}_\infty(\mathcal{D}, \mathcal{R}_1), \mathcal{R}_2)$. Regardless of the case, one can see that $\mathtt{R}(\boldsymbol{t}) \in \mathbf{Ch}_\infty(\mathcal{D}, \mathcal{R})$.

Inductive step. Suppose that $\mathcal{R} = \mathcal{R}_1 \triangleright \cdots \triangleright \mathcal{R}_n$, and let $\mathcal{R}' = \bigcup_{i \in \{1,\ldots n-1\}} \mathcal{R}_i$, from which it follows that $\mathcal{R}' = \mathcal{R}_1 \triangleright \cdots \triangleright \mathcal{R}_{n-1}$. By the base case, we know that

$\mathbf{Ch}_\infty(\mathcal{D},\mathcal{R}) = \mathbf{Ch}_\infty(\mathbf{Ch}_\infty(\mathcal{D},\mathcal{R}'),\mathcal{R}_n)$, and by IH, we know that

$$\mathbf{Ch}_\infty(\mathcal{D},\mathcal{R}') = \mathbf{Ch}_\infty(\ldots\mathbf{Ch}_\infty(\mathbf{Ch}_\infty(\mathcal{D},\mathcal{R}_1),\mathcal{R}_2)\ldots,\mathcal{R}_{n-1}).$$

Therefore, $\mathbf{Ch}_\infty(\mathcal{D},\mathcal{R}) = \mathbf{Ch}_\infty(\ldots\mathbf{Ch}_\infty(\mathbf{Ch}_\infty(\mathcal{D},\mathcal{R}_1),\mathcal{R}_2)\ldots,\mathcal{R}_n)$. □

Appendix E. Proofs for Section 10

Proposition 10.1. *Let $\mathcal{R} = \mathcal{R}_{\text{fr1}} \cup \mathcal{R}_{\text{ff}}$ be a ruleset where $\mathcal{R}_{\text{fr1}}$ is frontier-one and $\mathcal{R}_{\text{ff}}$ is full and* **fus**. *Then $\mathcal{R}$ is* **fps** *and thus allows for decidable HUSOMSOQ entailment.*

Proof. We define an atom $\mathtt{R}(\boldsymbol{t}) \in \mathbf{Ch}_\infty(\mathcal{D},\mathcal{R}_{\text{fr1}} \cup \mathcal{R}_{\text{ff}})$ to be an *ff-atom iff* $\mathtt{R}(\boldsymbol{t})$ was introduced at some step of the chase by a rule in $\mathcal{R}_{\text{ff}}$. We define

$$\mathbf{Ch}_{\text{fr1}}(\mathcal{D},\mathcal{R}_{\text{fr1}} \cup \mathcal{R}_{\text{ff}}) = \mathbf{Ch}_\infty(\mathcal{D},\mathcal{R}_{\text{fr1}} \cup \mathcal{R}_{\text{ff}}) \setminus \{\mathtt{R}(\boldsymbol{t}) \mid \mathtt{R}(\boldsymbol{t}) \text{ is an ff-atom}\}.$$

It is straightforward to confirm that $\mathbf{Ch}_{\text{fr1}}(\mathcal{D},\mathcal{R}_{\text{fr1}} \cup \mathcal{R}_{\text{ff}})$ has finite treewidth, and thus, the instance has finite partitionwidth by Theorem 6.24.

Let us now take the ruleset $\mathcal{R}_{\text{ff}}$ and observe that every rule in $\mathcal{R}_{\text{ff}}$ is a full rule of the form $\forall \boldsymbol{xy}(\varphi(\boldsymbol{x},\boldsymbol{y}) \to \mathtt{R}_1(\boldsymbol{z}_1)\wedge\cdots\wedge\mathtt{R}_n(\boldsymbol{z}_n))$ with $\boldsymbol{z}_1,\ldots,\boldsymbol{z}_n$ all variables in $\boldsymbol{y}$. We can therefore rewrite each such rule as the following, equivalent set of single-headed rules:

$$\{\forall \boldsymbol{xy}(\varphi(\boldsymbol{x},\boldsymbol{y}) \to \mathtt{R}_1(\boldsymbol{z}_1)),\ldots,\forall \boldsymbol{xy}(\varphi(\boldsymbol{x},\boldsymbol{y}) \to \mathtt{R}_n(\boldsymbol{z}_n))\}.$$

Based on this fact, we may assume w.l.o.g. that every rule in $\mathcal{R}_{\text{ff}}$ is a single-headed rule. Let us now rewrite $\mathcal{R}_{\text{ff}}$ relative to itself. To do this, we first define the rewriting of each rule in $\mathcal{R}_{\text{ff}}$. For each rule $\rho = \forall \boldsymbol{xy}(\varphi(\boldsymbol{x},\boldsymbol{y}) \to \mathtt{R}(\boldsymbol{y})) \in \mathcal{R}_{\text{ff}}$, we define the rewriting of ρ accordingly:

$$\mathsf{rew}(\rho) := \{\psi(\boldsymbol{x},\boldsymbol{y}) \to \mathtt{R}(\boldsymbol{y}) \mid \exists \psi(\boldsymbol{x},\boldsymbol{y}) \in \mathsf{rew}_{\mathcal{R}_{\text{ff}}}(\mathtt{R}(\boldsymbol{y}))\}.$$

We then define the rewriting of $\mathcal{R}_{\text{ff}}$ as follows: $\mathcal{R}'_{\text{ff}} := \bigcup_{\rho\in\mathcal{R}_{\text{ff}}} \mathsf{rew}(\rho)$. Since $\mathcal{R}_{\text{ff}}$ is **fus**, it follows that $\mathcal{R}'_{\text{ff}}$ is finite. Additionally, $\mathcal{R}'_{\text{ff}}$ is a full ruleset consisting solely of single-headed rules by definition, and so, we assume that the ruleset has the following form:

$$\mathcal{R}'_{\text{ff}} = \{\psi_1(\boldsymbol{x},\boldsymbol{y}) \to \mathtt{R}_1(\boldsymbol{y}),\ldots,\psi_n(\boldsymbol{x},\boldsymbol{y}) \to \mathtt{R}_n(\boldsymbol{y})\}.$$

We now show the following: For any $\mathtt{R}(\boldsymbol{t}) \in \mathbf{Ch}_\infty(\mathcal{D},\mathcal{R}_{\text{fr1}} \cup \mathcal{R}_{\text{ff}}) \setminus \mathbf{Ch}_{\text{fr1}}(\mathcal{D},\mathcal{R}_{\text{fr1}} \cup \mathcal{R}_{\text{ff}})$, there exists a trigger (ρ,h) in $\mathbf{Ch}_{\text{fr1}}(\mathcal{D},\mathcal{R}_{\text{fr1}} \cup \mathcal{R}_{\text{ff}})$ with $\rho \in \mathcal{R}'_{\text{ff}}$ such that applying (ρ,h) adds $\mathtt{R}(\boldsymbol{t})$ to $\mathbf{Ch}_{\text{fr1}}(\mathcal{D},\mathcal{R}_{\text{fr1}} \cup \mathcal{R}_{\text{ff}})$. To see this, observe that since $\mathcal{R}_{\text{ff}}$ is **fus**, we know by definition that for any predicate $\mathtt{R}$ occurring in any rule head of $\mathcal{R}_{\text{ff}}$ and any tuple $\boldsymbol{t}$ of terms in $\mathbf{Ch}_{\text{fr1}}(\mathcal{D},\mathcal{R}_{\text{fr1}} \cup \mathcal{R}_{\text{ff}})$, it holds that $\mathbf{Ch}_\infty(\mathbf{Ch}_{\text{fr1}}(\mathcal{D},\mathcal{R}_{\text{fr1}} \cup \mathcal{R}_{\text{ff}}),\mathcal{R}_{\text{ff}}) \models \mathtt{R}(\boldsymbol{t})$ *iff* there exists a CQ $\psi(\boldsymbol{x},\boldsymbol{y})$ in $\mathsf{rew}_{\mathcal{R}}(\mathtt{R}(\boldsymbol{y}))$ such that $\mathbf{Ch}_{\text{fr1}}(\mathcal{D},\mathcal{R}_{\text{fr1}} \cup \mathcal{R}_{\text{ff}}) \models \exists \boldsymbol{x}\psi(\boldsymbol{x},\boldsymbol{t})$. It therefore follows that a trigger exists for $\rho \in \mathcal{R}'_{\text{ff}}$ in $\mathbf{Ch}_{\text{fr1}}(\mathcal{D},\mathcal{R}_{\text{fr1}} \cup \mathcal{R}_{\text{ff}})$.

Based on the above fact, the following equation holds:

$$\mathbf{Ch}_\infty(\mathcal{D},\mathcal{R}_{\text{fr1}} \cup \mathcal{R}_{\text{ff}}) = \mathbf{Ch}_1(\mathbf{Ch}_{\text{fr1}}(\mathcal{D},\mathcal{R}_{\text{fr1}} \cup \mathcal{R}_{\text{ff}}),\mathcal{R}'_{\text{ff}}).$$

Recall that $\mathcal{R}'_{\text{ff}}$ is of the form $\{\psi_i(\boldsymbol{x},\boldsymbol{y}) \to \mathtt{R}_i(\boldsymbol{y}) \mid 1 \le i \le n\}$. The right hand side of the above equation can be rewritten as a finite sequence of FO-extensions (see Section 6.3):

$$\mathbf{Ch}_1(\mathbf{Ch}_{\text{fr1}}(\mathcal{D},\mathcal{R}_{\text{fr1}} \cup \mathcal{R}_{\text{ff}}),\mathcal{R}'_{\text{ff}}) = (\ldots(\mathbf{Ch}_{\text{fr1}}(\mathcal{D},\mathcal{R}_{\text{fr1}} \cup \mathcal{R}_{\text{ff}}))^{\mathtt{R}_1:=\mathtt{R}_1\cup\psi_1}\ldots)^{\mathtt{R}_n:=\mathtt{R}_n\cup\psi_n}.$$

By what was argued above, $\mathbf{Ch}_{\text{fr1}}(\mathcal{D},\mathcal{R}_{\text{fr1}} \cup \mathcal{R}_{\text{ff}})$ has finite partitionwidth, and since this property is preserved under FO-extensions (by Theorem 6.23), it follows that

$$\mathbf{Ch}_1(\mathbf{Ch}_{\text{fr1}}(\mathcal{D},\mathcal{R}_{\text{fr1}} \cup \mathcal{R}_{\text{ff}}),\mathcal{R}'_{\text{ff}})$$

has finite partitionwidth and hence, $\mathbf{Ch}_\infty(\mathcal{D}, \mathcal{R}_{\text{fr1}} \cup \mathcal{R}_{\text{ff}})$ has finite partitionwidth. Therefore, by Theorem 4.5, $\mathcal{R}_{\text{fr1}} \cup \mathcal{R}_{\text{ff}}$ allows for decidable HUSOMSOQ entailment. □

Proposition E.1. *Every full* **fus** *ruleset is* **fods**.

Proof. Consider a full **fus** ruleset $\mathcal{R}$ and observe that every rule in $\mathcal{R}$ is a full rule of the form $\forall \boldsymbol{xy}(\varphi(\boldsymbol{x}, \boldsymbol{y}) \rightarrow \mathtt{R}_1(\boldsymbol{z}_1) \wedge \cdots \wedge \mathtt{R}_n(\boldsymbol{z}_n))$ with $\boldsymbol{z}_1, \ldots, \boldsymbol{z}_n$ all variables in $\boldsymbol{y}$. We can therefore rewrite each such rule as the following, equivalent set of single-headed rules:

$$\{\forall \boldsymbol{xy}(\varphi(\boldsymbol{x}, \boldsymbol{y}) \rightarrow \mathtt{R}_1(\boldsymbol{z}_1)), \ldots, \forall \boldsymbol{xy}(\varphi(\boldsymbol{x}, \boldsymbol{y}) \rightarrow \mathtt{R}_n(\boldsymbol{z}_n))\}.$$

Based on this fact, we may assume w.l.o.g. that every rule in $\mathcal{R}$ is a single-headed rule.

We will next rewrite $\mathcal{R}$ relative to itself. To do this, we first define the rewriting of each rule in $\mathcal{R}$. For each rule $\rho = \forall \boldsymbol{xy}(\varphi(\boldsymbol{x}, \boldsymbol{y}) \rightarrow \mathtt{R}(\boldsymbol{y})) \in \mathcal{R}$, we define the rewriting of ρ accordingly:

$$\mathsf{rew}(\rho) := \{\psi(\boldsymbol{x}, \boldsymbol{y}) \rightarrow \mathtt{R}(\boldsymbol{y}) \mid \psi(\boldsymbol{x}, \boldsymbol{y}) \in \mathsf{rew}_{\mathcal{R}}(\mathtt{R}(\boldsymbol{y}))\}.$$

We then define the rewriting of $\mathcal{R}$ as follows: $\mathcal{R}' := \bigcup_{\rho \in \mathcal{R}} \mathsf{rew}(\rho)$. Since $\mathcal{R}$ is **fus** by assumption, it follows that $\mathcal{R}'$ is finite. Additionally, $\mathcal{R}'$ is a full ruleset consisting solely of single-headed rules by definition, and so, we assume that the ruleset has the following form:

$$\mathcal{R}' = \{\psi_1(\boldsymbol{x}, \boldsymbol{y}) \rightarrow \mathtt{R}_1(\boldsymbol{y}), \ldots, \psi_n(\boldsymbol{x}, \boldsymbol{y}) \rightarrow \mathtt{R}_n(\boldsymbol{y})\}.$$

We now show the following: For any $\mathcal{I}$ and any $\mathtt{R}(\boldsymbol{t}) \in \mathbf{Ch}_\infty(\mathcal{I}, \mathcal{R}) \setminus \mathcal{I}$, there exists a trigger (ρ, h) in $\mathcal{I}$ with $\rho \in \mathcal{R}'$ such that applying (ρ, h) adds $\mathtt{R}(\boldsymbol{t})$ to $\mathcal{I}$. To see this, observe that since $\mathcal{R}$ is **fus**, we know by definition that for any predicate $\mathtt{R}$ occurring in any rule head of $\mathcal{R}$ and any tuple $\boldsymbol{t}$ of terms in $\mathcal{I}$, it holds that $\mathbf{Ch}_\infty(\mathcal{I}, \mathcal{R}) \models \mathtt{R}(\boldsymbol{t})$ *iff* there exists a CQ $\exists \boldsymbol{x}.\psi(\boldsymbol{x}, \boldsymbol{y})$ in $\mathsf{rew}_{\mathcal{R}}(\mathtt{R}(\boldsymbol{y}))$ such that $\mathcal{I} \models \exists \boldsymbol{x} \psi(\boldsymbol{x}, \boldsymbol{t})$. It therefore follows that a trigger exists for the corresponding $\rho \in \mathcal{R}'_{\text{ff}}$ in $\mathcal{I}$.

Based on the above fact, we obtain $\mathbf{Ch}_\infty(\mathcal{I}, \mathcal{R}) = \mathbf{Ch}_1(\mathcal{I}, \mathcal{R}')$. Recall that $\mathcal{R}'_{\text{ff}}$ is of the form $\{\psi_i(\boldsymbol{x}, \boldsymbol{y}) \rightarrow \mathtt{R}_i(\boldsymbol{y}) \mid 1 \leq i \leq n\}$. Yet, the right hand side of the above equation can be rewritten as a finite sequence of FO-extensions as shown below:

$$\mathbf{Ch}_1(\mathcal{I}, \mathcal{R}') = (\ldots(\mathcal{I})^{\mathtt{R}_1 := \mathtt{R}_1 \cup \psi_1})\ldots)^{\mathtt{R}_n := \mathtt{R}_n \cup \psi_n},$$

which finishes our proof. □

Theorem 10.2. *Let $\mathcal{R} = \mathcal{R}_1 \triangleright \mathcal{R}_2$. If $\mathcal{R}_1$ is* **fps** *and $\mathcal{R}_2$ is* **fus**, *then the query entailment problem $(\mathcal{D}, \mathcal{R}) \models q$ for databases $\mathcal{D}$, and CQs q is decidable.*

Proof. Let us suppose that $\mathcal{R} = \mathcal{R}_1 \triangleright \mathcal{R}_2$, $\mathcal{R}_1$ is **fps**, and $\mathcal{R}_2$ is **fus**. Moreover, let $\mathcal{D}$ be an arbitrary database. Since $\mathcal{R} = \mathcal{R}_1 \triangleright \mathcal{R}_2$, we have $\mathbf{Ch}_\infty(\mathcal{D}, \mathcal{R}) = \mathbf{Ch}_\infty(\mathbf{Ch}_\infty(\mathcal{D}, \mathcal{R}_1), \mathcal{R}_2)$. As $\mathcal{R}_2$ is **fus**, we know that for any tuple $\mathtt{a}$ of constants from $\mathcal{D}$, $\mathbf{Ch}_\infty(\mathbf{Ch}_\infty(\mathcal{D}, \mathcal{R}_1), \mathcal{R}_2) \models q(\mathtt{a})$ *iff* $\mathbf{Ch}_\infty(\mathcal{D}, \mathcal{R}_1) \models \mathsf{rew}_{\mathcal{R}_2}(q(\mathtt{a}))$, where $\mathsf{rew}_{\mathcal{R}_2}(q)$ is a rewriting of q. By Theorem 4.5, we can decide if $\mathbf{Ch}_\infty(\mathcal{D}, \mathcal{R}_1) \models \mathsf{rew}_{\mathcal{R}_2}(q)$ as $\mathsf{rew}_{\mathcal{R}_2}(q)$ is a UCQ, which is a type of HUSOMSOQ. Hence, the query entailment problem $(\mathcal{D}, \mathcal{R}) \models q$ is decidable. □

References

[AHV95] Serge Abiteboul, Richard Hull, and Victor Vianu. *Foundations of Databases*. Addison-Wesley, 1995. URL: `http://webdam.inria.fr/Alice/`.

[ALS88] Stefan Arnborg, Jens Lagergren, and Detlef Seese. Problems easy for tree-decomposable graphs (extended abstract). In Timo Lepistö and Arto Salomaa, editors, *Proceedings of the 15th International Colloquium on Automata, Languages and Programming (ICALP'88)*, volume 317 of *LNCS*, pages 38–51. Springer, 1988. `doi:10.1007/3-540-19488-6_105`.

[ANvB98] Hajnal Andréka, István Németi, and Johan van Benthem. Modal languages and bounded fragments of predicate logic. *Journal of Philosophical Logic*, 27(3):217–274, 1998. doi:10.1023/A:1004275029985.

[AriCE] Aristotle. *Prior Analytics*. Organon. 350 BCE. Original title Αναλυτικα Προτερα. Translation into English by A. J. Jenkinson available via http://classics.mit.edu/Aristotle/prior.html.

[Bag04] Jean-François Baget. Improving the forward chaining algorithm for conceptual graphs rules. In *Proceedings of the 9th International Conference on Principles of Knowledge Representation and Reasoning (KR)'04*, page 407–414. AAAI Press, 2004. URL: http://www.aaai.org/Library/KR/2004/kr04-043.php.

[BBMR15] Jean-François Baget, Meghyn Bienvenu, Marie-Laure Mugnier, and Swan Rocher. Combining existential rules and transitivity: Next steps. In Qiang Yang and Michael J. Wooldridge, editors, *Proceedings of the 24th International Joint Conference on Artificial Intelligence (IJCAI'15)*, pages 2720–2726. AAAI Press, 2015. URL: http://ijcai.org/Abstract/15/385.

[BBMT17] Jean-François Baget, Meghyn Bienvenu, Marie-Laure Mugnier, and Michael Thomazo. Answering conjunctive regular path queries over guarded existential rules. In *Proceedings of the 26th International Joint Conference on Artificial Intelligence (IJCAI'17)*, pages 793–799, 2017. doi:10.24963/ijcai.2017/110.

[BBV16] Michael Benedikt, Pierre Bourhis, and Michael Vanden Boom. A step up in expressiveness of decidable fixpoint logics. In Martin Grohe, Eric Koskinen, and Natarajan Shankar, editors, *Proceedings of the 31st Annual ACM/IEEE Symposium on Logic in Computer Science (LICS'16)*, pages 817–826. ACM, 2016. doi:10.1145/2933575.2933592.

[BCK+21] Camille Bourgaux, David Carral, Markus Krötzsch, Sebastian Rudolph, and Michaël Thomazo. Capturing homomorphism-closed decidable queries with existential rules. In Meghyn Bienvenu, Gerhard Lakemeyer, and Esra Erdem, editors, *Proceedings of the 18th International Conference on Principles of Knowledge Representation and Reasoning (KR'21)*, pages 141–150, 2021. doi:10.24963/kr.2021/14.

[BCS15] Vince Bárány, Balder Ten Cate, and Luc Segoufin. Guarded negation. *Journal of the ACM*, 62(3), 2015. doi:10.1145/2701414.

[BFKR21] Manuel Bodirsky, Thomas Feller, Simon Knäuer, and Sebastian Rudolph. On logics and homomorphism closure. In *Proceedings of the 36th Annual ACM/IEEE Symposium on Logic in Computer Science (LICS'21)*, pages 1–13. IEEE, 2021. doi:10.1109/LICS52264.2021.9470511.

[BGG97] Egon Börger, Erich Grädel, and Yuri Gurevich. *The Classical Decision Problem*. Perspectives in Mathematical Logic. Springer, 1997.

[BGO14] Vince Bárány, Georg Gottlob, and Martin Otto. Querying the guarded fragment. *Logical Methods in Computer Science*, 10(2), 2014. doi:10.2168/LMCS-10(2:3)2014.

[BHLS17] Franz Baader, Ian Horrocks, Carsten Lutz, and Uli Sattler. *An Introduction to Description Logic*. Cambridge University Press, 2017. doi:10.1017/9781139025355.

[BKR14] Pierre Bourhis, Markus Krötzsch, and Sebastian Rudolph. How to best nest regular path queries. In Meghyn Bienvenu, Magdalena Ortiz, Riccardo Rosati, and Mantas Simkus, editors, *Informal Proceedings of the 27th International Workshop on Description Logics (DL'14)*, volume 1193 of *CEUR Workshop Proceedings*, pages 404–415. CEUR-WS.org, 2014. URL: http://ceur-ws.org/Vol-1193/paper_80.pdf.

[BKR15] Pierre Bourhis, Markus Krötzsch, and Sebastian Rudolph. Reasonable highly expressive query languages. In Qiang Yang and Michael J. Wooldridge, editors, *Proceedings of the 24th International Joint Conference on Artificial Intelligence (IJCAI'15)*, pages 2826–2832. AAAI Press, 2015. URL: http://ijcai.org/Abstract/15/400.

[BKR21] Manuel Bodirsky, Simon Knäuer, and Sebastian Rudolph. Datalog-expressibility for monadic and guarded second-order logic. In Nikhil Bansal, Emanuela Merelli, and James Worrell, editors, *Proceedings of the 48th International Colloquium on Automata, Languages, and Programming, (ICALP'21)*, volume 198 of *LIPIcs*, pages 120:1–120:17. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2021. doi:10.4230/LIPICS.ICALP.2021.120.

[BLMS11] Jean-François Baget, Michel Leclère, Marie-Laure Mugnier, and Eric Salvat. On rules with existential variables: Walking the decidability line. *Artificial Intelligence*, 175(9):1620–1654, 2011. doi:10.1016/j.artint.2011.03.002.

[Blu03] Achim Blumensath. *Structures of bounded partition width*. PhD thesis, RWTH Aachen University, 2003. URL: http://sylvester.bth.rwth-aachen.de/dissertationen/2003/256/index.htm.

[Blu06] Achim Blumensath. A model-theoretic characterisation of clique width. *Annals of Pure and Applied Logic*, 142(1-3):321–350, 2006. doi:10.1016/j.apal.2006.02.004.

[BMMP16] Pierre Bourhis, Marco Manna, Michael Morak, and Andreas Pieris. Guarded-based disjunctive tuple-generating dependencies. *ACM Transactions on Database Systems*, 41(4):27:1–27:45, 2016. doi:10.1145/2976736.

[BMP17] Pierre Bourhis, Michael Morak, and Andreas Pieris. Making cross products and guarded ontology languages compatible. In Carles Sierra, editor, *Proceedings of the 26th International Joint Conference on Artificial Intelligence (IJCAI'17)*, pages 880–886. ijcai.org, 2017. doi:10.24963/ijcai.2017/122.

[BMR23] Jean-François Baget, Marie-Laure Mugnier, and Sebastian Rudolph. Bounded tree-width and the infinite core chase: Complications and workarounds toward decidable querying. In Floris Geerts, Hung Q. Ngo, and Stavros Sintos, editors, *Proceedings of the 42nd ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems (PODS'23)*, pages 291–302. ACM, 2023. doi:10.1145/3584372.3588659.

[Boo54] George Boole. *An Investigation of the Laws of Thought on Which are Founded the Mathematical Theories of Logic and Probabilities*. Walton and Maberly, London, 1854.

[BR19] Bartosz Bednarczyk and Sebastian Rudolph. Worst-case optimal querying of very expressive description logics with path expressions and succinct counting. In Sarit Kraus, editor, *Proceedings of the 28th International Joint Conference on Artificial Intelligence (IJCAI'19)*, pages 1530–1536. ijcai.org, 2019. doi:10.24963/ijcai.2019/212.

[BR23] Bartosz Bednarczyk and Sebastian Rudolph. How to tell easy from hard: Complexities of conjunctive query entailment in extensions of ALC. *Journal of Artificial Intelligence Research*, 78, 2023. doi:10.1613/JAIR.1.14482.

[BtCLW14] Meghyn Bienvenu, Balder ten Cate, Carsten Lutz, and Frank Wolter. Ontology-based data access: A study through disjunctive datalog, csp, and MMSNP. *ACM Transactions on Database Systems*, 39(4):33:1–33:44, 2014. doi:10.1145/2661643.

[CE12] Bruno Courcelle and Joost Engelfriet. *Graph Structure and Monadic Second-Order Logic – A Language-Theoretic Approach*, volume 138 of *Encyclopedia of mathematics and its applications*. Cambridge University Press, 2012. URL: http://www.cambridge.org/fr/knowledge/isbn/item5758776/?site_locale=fr_FR.

[CEO09] Diego Calvanese, Thomas Eiter, and Magdalena Ortiz. Regular path queries in expressive description logics with nominals. In Craig Boutilier, editor, *Proceedings of the 21st International Joint Conference on Artificial Intelligence (IJCAI'09)*, pages 714–720, 2009. URL: http://ijcai.org/Proceedings/09/Papers/124.pdf.

[CGK13] Andrea Calì, Georg Gottlob, and Michael Kifer. Taming the infinite chase: Query answering under expressive relational constraints. *Journal of Artificial Intelligence Research*, 48:115–174, 2013. doi:10.1613/jair.3873.

[CGKV88] Stavros S. Cosmadakis, Haim Gaifman, Paris C. Kanellakis, and Moshe Y. Vardi. Decidable optimization problems for database logic programs (preliminary report). In Janos Simon, editor, *Proceedings of the 20th Annual ACM Symposium on Theory of Computing (STOC'88)*, pages 477–490. ACM, 1988. doi:10.1145/62212.62259.

[CGL98] Diego Calvanese, Giuseppe De Giacomo, and Maurizio Lenzerini. On the decidability of query containment under constraints. In Alberto O. Mendelzon and Jan Paredaens, editors, *Proceedings of the 17th ACM SIGACT-SIGMOD-SIGART Symposium on Principles of Database Systems (PODS'98)*, pages 149–158. ACM Press, 1998. doi:10.1145/275487.275504.

[CGP12] Andrea Calì, Georg Gottlob, and Andreas Pieris. Towards more expressive ontology languages: The query answering problem. *Artificial Intelligence*, 193:87–128, 2012. doi:10.1016/j.artint.2012.08.002.

[CLM81] Ashok K. Chandra, Harry R. Lewis, and Johann A. Makowsky. Embedded implicational dependencies and their inference problem. In *Proceedings of the 13th Annual ACM Symposium on Theory of Computing (STOC'81)*, pages 342–354. ACM, 1981. doi:10.1145/800076.802488.

[Cou89] Bruno Courcelle. The monadic second-order logic of graphs, II: infinite graphs of bounded width. *Mathematical Systems Theory*, 21(4):187–221, 1989. doi:10.1007/BF02088013.

[Cou91] Bruno Courcelle. The monadic second-order logic of graphs V: on closing the gap between definability and recognizability. *Theoretical Computer Science*, 80(2):153–202, 1991. doi:10.1016/0304-3975(91)90387-H.

[Cou94] Bruno Courcelle. Monadic second-order definable graph transductions: A survey. *Theoretical Computer Science*, 126(1):53–75, 1994. doi:10.1016/0304-3975(94)90268-2.

[Cou04] Bruno Courcelle. Clique-width of countable graphs: A compactness property. *Discrete Mathematics*, 276(1-3):127–148, 2004. doi:10.1016/S0012-365X(03)00303-0.

[EGM97] Thomas Eiter, Georg Gottlob, and Heikki Mannila. Disjunctive datalog. *ACM Transactions on Database Systems*, 22(3):364–418, 1997. doi:10.1145/261124.261126.

[Eng90] Joost Engelfriet. A characterization of context-free NCE graph languages by monadic second-order logic on trees. In Hartmut Ehrig, Hans-Jörg Kreowski, and Grzegorz Rozenberg, editors, *Proceedings of the 4th International Workshop on Graph-Grammars and Their Application to Computer Science*, volume 532 of *LNCS*, pages 311–327. Springer, 1990. doi:10.1007/BFb0017397.

[FKL19] Cristina Feier, Antti Kuusisto, and Carsten Lutz. Rewritability in monadic disjunctive datalog, mmsnp, and expressive description logics. *Logical Methods in Computer Science*, 15(2), 2019. doi:10.23638/LMCS-15(2:15)2019.

[FKMP05] R. Fagin, P. G. Kolaitis, R. J. Miller, and L. Popa. Data exchange: Semantics and query answering. *Theoretical Computer Science*, 336(1):89–124, 2005. doi:10.1016/J.TCS.2004.10.033.

[FLOR23] Thomas Feller, Tim S. Lyon, Piotr Ostropolski-Nalewaja, and Sebastian Rudolph. Finite-cliquewidth sets of existential rules: Toward a general criterion for decidable yet highly expressive querying. In Floris Geerts and Brecht Vandevoort, editors, *Proceedings of the 26th International Conference on Database Theory (ICDT'23)*, volume 255 of *LIPIcs*, pages 18:1–18:18. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2023. doi:10.4230/LIPIcs.ICDT.2023.18.

[FLS98] Daniela Florescu, Alon Y. Levy, and Dan Suciu. Query containment for conjunctive queries with regular expressions. In Alberto O. Mendelzon and Jan Paredaens, editors, *Proceedings of the 17th ACM SIGACT-SIGMOD-SIGART Symposium on Principles of Database Systems (PODS'98)*, pages 139–148. ACM, 1998. doi:10.1145/275487.275503.

[Fre79] Gottlob Frege. *Begriffsschrift, eine der arithmetischen nachgebildete Formelsprache des reinen Denkens*. Verlag von Louis Nebert, Halle, 1879. URL: http://resolver.sub.uni-goettingen.de/purl?PPN538957069.

[GHK+13] Bernardo Cuenca Grau, Ian Horrocks, Markus Krötzsch, Clemens Kupke, Despoina Magka, Boris Motik, and Zhe Wang. Acyclicity notions for existential rules and their application to query answering in ontologies. *Journal of Artificial Intelligence Research*, 47:741–808, 2013. doi:10.1613/jair.3949.

[GHO00] Erich Grädel, Colin Hirsch, and Martin Otto. Back and forth between guarded and modal logics. In *Proceedings of the 15th Annual IEEE Symposium on Logic in Computer Science (LICS'00)*, pages 217–228. IEEE, 2000. doi:10.1109/LICS.2000.855771.

[GHO02] Erich Grädel, Colin Hirsch, and Martin Otto. Back and forth between guarded and modal logics. *ACM Transactions on Computational Logic*, 3(3):418–463, 2002. doi:10.1145/507382.507388.

[GKV97] Erich Grädel, Phokion G. Kolaitis, and Moshe Y. Vardi. On the decision problem for two-variable first-order logic. *Bulletin of Symbolic Logic*, 3(1):53–69, 1997. doi:10.2307/421196.

[GM13a] Tomasz Gogacz and Jerzy Marcinkowski. Converging to the chase - A tool for finite controllability. In *Proceedings of the 28th Annual ACM/IEEE Symposium on Logic in Computer Science (LICS'13)*, pages 540–549. IEEE Computer Society, 2013. doi:10.1109/LICS.2013.61.

[GM13b] Tomasz Gogacz and Jerzy Marcinkowski. On the BDD/FC conjecture. In Richard Hull and Wenfei Fan, editors, *Proceedings of the 32nd ACM SIGMOD-SIGACT-SIGART Symposium on Principles of Database Systems (PODS'13)*, pages 127–138. ACM, 2013. doi:10.1145/2463664.2463668.

[GM17] Tomasz Gogacz and Jerzy Marcinkowski. Converging to the chase - A tool for finite controllability. *Journal of Computer and System Sciences*, 83(1):180–206, 2017. doi:10.1016/j.jcss.2016.08.001.

[GMV99] Harald Ganzinger, Christoph Meyer, and Margus Veanes. The two-variable guarded fragment with transitive relations. In *Proceedings of the 14th Annual IEEE Symposium on Logic in Computer Science (LICS'99)*, pages 24–34. IEEE Computer Society, 1999. doi:10.1109/LICS.1999.782582.

[Göd29] Kurt Gödel. *Über die Vollständigkeit des Logikkalküls*. PhD thesis, Universität Wien, 1929.

[GOR97] Erich Grädel, Martin Otto, and Eric Rosen. Two-variable logic with counting is decidable. In *Proceedings of the 12th Annual IEEE Symposium on Logic in Computer*

Science (LICS'97), pages 306–317. IEEE Computer Society, 1997. doi:10.1109/LICS.1997.614957.

[Got09] Georg Gottlob. Datalog+/-: A unified approach to ontologies and integrity constraints. In Valeria De Antonellis, Silvana Castano, Barbara Catania, and Giovanna Guerrini, editors, *Proceedings of the 17th Italian Symposium on Advanced Database Systems, (SEBD'09)*, pages 5–6. Edizioni Seneca, 2009.

[Grä99] Erich Grädel. On the restraining power of guards. *Journal of Symbolic Logic*, 64(4):1719–1742, 1999. doi:10.2307/2586808.

[Grä01] Erich Grädel. Why are modal logics so robustly decidable? In Gheorghe Paun, Grzegorz Rozenberg, and Arto Salomaa, editors, *Current Trends in Theoretical Computer Science, Entering the 21th Century*, pages 393–408. World Scientific, 2001.

[GT04] Martin Grohe and György Turán. Learnability and definability in trees and similar structures. *Theory of Computing Systems*, 37(1):193–220, 2004. doi:10.1007/s00224-003-1112-8.

[HKS06] Ian Horrocks, Oliver Kutz, and Ulrike Sattler. The even more irresistible SROIQ. In *Proceedings of the 10th International Conference on Principles of Knowledge Representation and Reasoning (KR'06)*, page 57–67. AAAI Press, 2006. URL: http://www.aaai.org/Library/KR/2006/kr06-009.php.

[Kaz08] Yevgeny Kazakov. RIQ and SROIQ are harder than SHOIQ. In Gerhard Brewka and Jérôme Lang, editors, *Proceedings of the 11th International Conference on Principles of Knowledge Representation and Reasoning (KR'08)*, pages 274–284. AAAI Press, 2008. URL: http://www.aaai.org/Library/KR/2008/kr08-027.php.

[KLMT13] Mélanie König, Michel Leclère, Marie-Laure Mugnier, and Michaël Thomazo. Sound, complete, and minimal query rewriting for existential rules. In Francesca Rossi, editor, *Proceedings of the 23rd International Joint Conference on Artificial Intelligence (IJCAI'13)*, pages 3017–3021. IJCAI/AAAI, 2013. URL: http://www.aaai.org/ocs/index.php/IJCAI/IJCAI13/paper/view/6641.

[KM20] Emanuel Kieronski and Adam Malinowski. The triguarded fragment with transitivity. In Elvira Albert and Laura Kovács, editors, *Proceedings of the 23rd International Conference on Logic for Programming, Artificial Intelligence and Reasoning (LPAR'20)*, volume 73 of *EPiC Series in Computing*, pages 334–353. EasyChair, 2020. doi:10.29007/z359.

[KR11] Markus Krötzsch and Sebastian Rudolph. Extending decidable existential rules by joining acyclicity and guardedness. In Toby Walsh, editor, *Proceedings of the 22nd International Joint Conference on Artificial Intelligence (IJCAI'11)*, pages 963–968. IJCAI/AAAI, 2011. doi:10.5591/978-1-57735-516-8/IJCAI11-166.

[KR21] Emanuel Kieronski and Sebastian Rudolph. Finite model theory of the triguarded fragment and related logics. In *Proceedings of the 36th Annual ACM/IEEE Symposium on Logic in Computer Science (LICS'21)*, pages 1–13. IEEE, 2021. doi:10.1109/LICS52264.2021.9470734.

[Mar09] B. Marnette. Generalized schema-mappings: from termination to tractability. In J. Paredaens and J. Su, editors, *Proceedings of the 28th ACM SIGMOD-SIGACT-SIGART Symposium on Principles of Database Systems (PODS'09)*, pages 13–22. ACM, 2009. doi:10.1145/1559795.1559799.

[Mor75] Michael Mortimer. On languages with two variables. *Mathematical Logic Quarterly*, 21(1):135–140, 1975.

[OS12] Magdalena Ortiz and Mantas Simkus. Reasoning and query answering in description logics. In Thomas Eiter and Thomas Krennwallner, editors, *Lecture Notes of the 8th International Reasoning Web Summer School (RW'12)*, volume 7487 of *LNCS*, pages 1–53. Springer, 2012. doi:10.1007/978-3-642-33158-9_1.

[PH23] Ian Pratt-Hartmann. *Fragments of First-Order Logic*, volume 56 of *Oxford Logic Guides*. Oxford University Press, 2023. doi:10.1093/oso/9780192867964.001.0001.

[Rab69] Michael O. Rabin. Decidability of second-order theories and automata on infinite trees. *Transactions of the American Mathematical Society*, 141:1–35, 1969. URL: http://www.jstor.org/stable/1995086.

[RG10] Sebastian Rudolph and Birte Glimm. Nominals, inverses, counting, and conjunctive queries or: Why infinity is your friend! *Journal of Artificial Intelligence Research*, 39:429–481, 2010. doi:10.1613/jair.3029.

[RK13] Sebastian Rudolph and Markus Krötzsch. Flag & check: Data access with monadically defined queries. In Richard Hull and Wenfei Fan, editors, *Proceedings of the 32nd ACM SIGMOD-SIGACT-SIGART Symposium on Principles of Database Systems (PODS'13)*, pages 151–162. ACM, 2013. doi:10.1145/2463664.2465227.

[RKH08a] Sebastian Rudolph, Markus Krötzsch, and Pascal Hitzler. All elephants are bigger than all mice. In Franz Baader, Carsten Lutz, and Boris Motik, editors, *Proceedings of the 21st International Workshop on Description Logics (DL2008)*, volume 353 of *CEUR Workshop Proceedings*. CEUR-WS.org, 2008. URL: `http://ceur-ws.org/Vol-353/RudolphKraetzschHitzler.pdf`.

[RKH08b] Sebastian Rudolph, Markus Krötzsch, and Pascal Hitzler. Cheap boolean role constructors for description logics. In Steffen Hölldobler, Carsten Lutz, and Heinrich Wansing, editors, *Proceedings of the 11th European Conference on Logics in Artificial Intelligence (JELIA'08)*, volume 5293 of *LNCS*, pages 362–374. Springer, 2008. `doi:10.1007/978-3-540-87803-2_30`.

[Ros07] Riccardo Rosati. The limits of querying ontologies. In Thomas Schwentick and Dan Suciu, editors, *Proceedings of the 11th International Conference on Database Theory (ICDT'07)*, volume 4353 of *LNCS*, pages 164–178. Springer, 2007. `doi:10.1007/11965893_12`.

[RRV17] Juan L. Reutter, Miguel Romero, and Moshe Y. Vardi. Regular queries on graph databases. *Theory of Computing Systems*, 61(1):31–83, 2017. `doi:10.1007/s00224-016-9676-2`.

[RS84] Neil Robertson and P.D Seymour. Graph minors. iii. planar tree-width. *Journal of Combinatorial Theory, Series B*, 36(1):49–64, 1984. `doi:10.1016/0095-8956(84)90013-3`.

[RS18] Sebastian Rudolph and Mantas Simkus. The triguarded fragment of first-order logic. In Gilles Barthe, Geoff Sutcliffe, and Margus Veanes, editors, *Proceedings of the 22nd International Conference on Logic for Programming, Artificial Intelligence and Reasoning (LPAR-22)*, volume 57 of *EPiC Series in Computing*, pages 604–619. EasyChair, 2018. `doi:10.29007/m8ts`.

[Rud11] Sebastian Rudolph. Foundations of description logics. In Axel Polleres, Claudia d'Amato, Marcelo Arenas, Siegfried Handschuh, Paula Kroner, Sascha Ossowski, and Peter F. Patel-Schneider, editors, *Lecture Notes of the 7th International Reasoning Web Summer School (RW'11)*, volume 6848 of *LNCS*, pages 76–136. Springer, 2011. `doi:10.1007/978-3-642-23032-5_2`.

[Sko29] Thoralf Skolem. Über einige Grundlagenfragen der Mathematik. *Skrifter utgitt av det Norske Videnskaps-Akademi i Oslo, I. Matematisk-naturvidenskabelig Klasse*, 7:1–49, 1929.

[SM96] Eric Salvat and Marie-Laure Mugnier. Sound and complete forward and backward chainings of graph rules. In Peter W. Eklund, Gerard Ellis, and Graham Mann, editors, *Proceedings of the 4th International Conference on Conceptual Structures (ICCS'96)*, volume 1115 of *LNCS*, pages 248–262. Springer, 1996. `doi:10.1007/3-540-61534-2_16`.

[ST04] Wieslaw Szwast and Lidia Tendera. The guarded fragment with transitive guards. *Annals of Pure and Applied Logic*, 128(1-3):227–276, 2004. `doi:10.1016/j.apal.2004.01.003`.

[StC13] Luc Segoufin and Balder ten Cate. Unary negation. *Logical Methods in Computer Science*, 9(3), 2013. `doi:10.2168/LMCS-9(3:25)2013`.

[Tar43] Alfred Tarski. The semantic conception of truth and the foundations of semantics. *Philosophy and Phenomenological Research*, 4(3):341–376, 1943. `doi:10.2307/2102968`.

[Tra50] Boris Abramovich Trakhtenbrot. The impossibility of an algorithm for the decidability problem on finite classes. *Doklady Akademii Nauk SSSR*, 70(4):569–572, 1950.

[Var96] Moshe Y. Vardi. Why is modal logic so robustly decidable? In Neil Immerman and Phokion G. Kolaitis, editors, *Proceedings of a DIMACS Workshop on Descriptive Complexity and Finite Models*, volume 31 of *DIMACS Series in Discrete Mathematics and Theoretical Computer Science*, pages 149–183. DIMACS/AMS, 1996. `doi:10.1090/DIMACS/031/05`.